COMP2870 Theoretical Foundations: Linear Algebra

Thomas Ranner

19 October 2026

Table of contents

1 Introduction **5**
- 1.1 Contents of this submodule . . . 5
  - 1.1.1 Topics . . . 5
  - 1.1.2 Learning objectives . . . 6
- 1.2 Textbooks and other resources . . . 6
- 1.3 Programming . . . 7
- 1.4 Reminder of matrices and vectors . . . 7
- 1.5 Comments on these notes . . . 9

2 Floating point number systems **10**
- 2.1 Finite precision number systems . . . 10
- 2.2 Normalised systems . . . 11
- 2.3 Errors, machine precision and unit roundoff . . . 15
- 2.4 Other "Features" of finite precision arithmetic . . . 18
- 2.5 Is this all academic? . . . 19
- 2.6 Summary . . . 19
  - 2.6.1 Further reading . . . 19

3 Introduction to systems of linear equations **20**
- 3.1 Systems of linear equations . . . 20
- 3.2 Matrix inverse . . . 23
- 3.3 The determinant . . . 25
- 3.4 Special types of matrices . . . 28
- 3.5 Summary . . . 30
- 3.6 Further reading . . . 30

4 Direct solvers for systems of linear equations **31**
- 4.1 Reminder of the problem . . . 31
- 4.2 Elementary row operations . . . 32
- 4.3 Gaussian elimination . . . 34
  - 4.3.1 The algorithm . . . 35
  - 4.3.2 Python version . . . 37
- 4.4 Solving triangular systems of equations . . . 46
- 4.5 Combining Gaussian elimination and backward substitution . . . 52
- 4.6 The cost of Gaussian elimination . . . 54

4.7 LU factorisation . . . 55
4.7.1 Computing L and U . . . 57
4.8 Effects of finite precision arithmetic . . . 59
4.8.1 Gaussian elimination with pivoting . . . 63
4.8.2 Python code for Gaussian elimination with pivoting . . . 65
4.9 Summary . . . 71
4.10 Further reading . . . 71

5 Iterative solutions of linear equations 72
5.1 Iterative methods . . . 72
5.2 Jacobi iteration . . . 74
5.3 Gauss-Seidel iteration . . . 76
5.4 Python version of iterative methods . . . 79
5.5 Sparse Matrices . . . 83
5.5.1 Python experiments . . . 83
5.6 Convergence of an iterative method . . . 90
5.7 Summary . . . 92
5.8 Further reading . . . 92

6 Complex numbers 94
6.1 Basic definitions . . . 94
6.2 Calculations with complex numbers . . . 96
6.3 A geometric picture . . . 98
6.4 Euler's formula . . . 102
6.5 Solving polynomial equations . . . 103
6.6 Complex vectors and matrices . . . 104
6.7 Summary . . . 107

7 Eigenvectors and eigenvalues 108
7.1 Key definitions . . . 108
7.2 Why does a computer scientist need to know about eigenvalues . . . 110
7.3 How to find eigenvalues and eigenvectors . . . 113
7.4 Important theory . . . 116
7.5 Similar matrices . . . 116
7.6 Eigenvalues of symmetric matrices . . . 120
7.7 Summary . . . 121

8 Eigenvectors and eigenvalues: practical solutions 122
8.1 Recall: the eigenvalue problem . . . 122
8.2 The grand strategy . . . 122
8.3 QR algorithm . . . 123
8.3.1 The Gram-Schmidt process . . . 124
8.3.2 Applying the QR algorithm by hand . . . 130

8.3.3 Python QR factorisation using Gram-Schmidt 131
8.4 Finding eigenvalues and eigenvectors 132
8.5 Correctness and convergence 135
8.6 Summary 138

1 Introduction

1.1 Contents of this submodule

This part of the module will deal with numerical algorithms that involve matrices. The study of this type of problem is called *linear algebra*. We will approach these problems using a combination of theoretical ideas and practical solutions, thinking through the lens of real-world applications. As a consequence, to succeed in linear algebra, you will do some programming (using Python) and some pen-and-paper theoretical work, too.

1.1.1 Topics

We will have seven double lectures, three tutorials, and four labs. We break up the topics as follows:

Lectures

1. Introduction and motivation, key problem statements (Week 4, Mon)
2. When can we solve systems of linear equations? (Week 4, Wed)
3. Direct methods for systems of linear equations (Week 5, Mon)
4. Iterative solution of linear equations (Week 5, Wed)
5. Complex numbers (Week 6, Mon)
6. Eigenvalues and eigenvectors (Week 6, Wed)
7. Practical solutions for eigenvalues and eigenvectors / Summary (Week 7, Mon)

There are notes here for an additional lecture on Floating point numbers which instead we cover briefly in the lectures and in further detail in the first Lab session.

Labs

1. Floating point numbers (Week 4)
2. When can we solve systems of linear equations? (Week 5)
3. Systems of linear equations (Week 6)
4. Eigenvalues and eigenvectors (Week 7)

Tutorials

1. Linear independence, span, basis (Week 5)

2. Solution of linear systems (Week 6)
3. Eigenvalue and eigenvectors (Week 7)

Later this term, week 10, you will complete a project based on the things you've learnt from this section of the module.

1.1.2 Learning objectives

At the start of each section of the notes, a number of learning objectives are listed. These can be used as a guide to the important topics in each section and can be used as a revision aid. Additionally, we expect that you can state and apply for the following algorithms in an exam setting:

1. Gaussian elimination (without pivoting)
2. Gaussian elimination with pivoting
3. Jacobi iteration
4. Gauss-Seidel iteration
5. QR factorisation through the Gram-Schmidt process

1.2 Textbooks and other resources

There are many textbooks and other external resources which could help your learning:

- Introduction to Linear Algebra (Fifth Edition), Gilbert Strang, Wellesley-Cambridge Press, 2016. with MIT course material. *Strongly recommended book and YouTube lecture series*
- Scientific Computing: An Introductory Survey, T.M. Heath, McGraw-Hill, 2002. Some lecture notes based on the book
- Engineering Mathematics, K.A. Stroud, Macmillan, 2001. *available online*
- Numerical Recipes in C++/C/FORTRAN: The Art of Scientific Computing, W.H. Press, S.A. Teukolsky, W.T. Vetterling and B.P. Flannery, Cambridge University Press, 2002/1993/1993. *A very practical view of the methods we develop in this module which could be used for library development*

1.3 Programming

This section of the notes links theoretical material with practical applications. We will have weekly lab sessions where you will see how the methods mentioned in the lecture notes work when implemented. More instructions on how we will do this will be covered in the lab sessions themselves.

An important theoretical aspect of translating the algorithms in linear algebra to computer implementations is the use of floating-point numbers. You will have already met floating-point numbers in your first year studies where you met how computer store numbers. You will already know that computer cannot store every possible real number exactly. The inexactness of floating-point numbers has real consequences when performing linear algebra computations.

Exploring and understanding the material in (Chapter 2) is really helpful for understanding many of the choices that we make when forming methods in linear algebra.

We will make extensive use of `Python` and `numpy` during this section of the module. It may help you to revise some of your notes from last year on these topics.

1.4 Reminder of matrices and vectors

There are two important objects we will work with that were defined in your first-year Theoretical Foundations module (COMP1870).

Definition 1.1. A *matrix* is a rectangular array of numbers called *entries* or *elements* of the matrix. A matrix with m rows and n columns is called an $m \times n$ matrix or m-by-n matrix. We may additionally say that the matrix is of order $m \times n$. If $m = n$, then we say that the matrix is *square*.

Example 1.1. A, B and C are 3×4 matrices :

$$A = \begin{pmatrix} 10 & 1 & 0 & 9 \\ 12.4 & 6 & 1 & 0 \\ 1 & 3.14 & 1 & 0 \end{pmatrix} \quad B = \begin{pmatrix} 0 & 6 & 3 & 1 \\ 1 & 4 & 1 & 0 \\ 7 & 0 & 10 & 20 \end{pmatrix} \quad C = \begin{pmatrix} 4 & 1 & 8 & -1 \\ 1.5 & 1 & 3 & 4 \\ 6 & -4 & 2 & 8 \end{pmatrix}$$

Exercise 1.1.

1. Compute, if defined, $A + B$, $B + C$.
2. Compute, if defined, AB, BA, BC (here, by writing matrices next to each other we mean the matrix product).

When considering systems of linear equations, the entries of the matrix will always be real numbers (later we will explore using complex numbers (Chapter 6) too)

Definition 1.2. A *column vector*, often just called a *vector*, is a matrix with a single column. A matrix with a single row is a *row vector*. The entries of a vector are called *components*. A vector with n rows is called an n-vector.

Example 1.2. $\vec{a}$ is a row vector, $\vec{b}$ and $\vec{c}$ are (column) vectors.

$$\vec{a} = \begin{pmatrix} 0 & 1 & 7 \end{pmatrix} \quad \vec{b} = \begin{pmatrix} 0 \\ 1 \\ 3.1 \\ 7 \end{pmatrix} \quad \vec{c} = \begin{pmatrix} 4 \\ 6 \\ -4 \\ 0 \end{pmatrix}.$$

Exercise 1.2.

1. Compute, if defined, $\vec{b} + \vec{c}$, $0.25\vec{c}$.
2. What is the meaning of $\vec{b}^T\vec{c}$? (Here, we are interpreting the vectors as matrices).
3. Compute, if defined, $B\vec{b}$.

Example 1.3 (Some special matrices)**.** It will be helpful to remember some special matrices during this section of the module.

A **rotation matrix** is a 2×2 matrix given by

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}. \tag{1.1}$$

This matrix describes an anti-clockwise rotation by θ (in radians).

The **projection operator** maps vectors onto either lines or planes which pass through the origin ($\vec{0}$).

- For projection onto a line $L = \{\lambda\vec{a} : \lambda \in \mathbb{R}\}$, with $\|\vec{a}\| = 1$ (i.e. magnitude 1), we can write this transformation as:

  $$\vec{x} \mapsto \vec{a} \cdot \vec{x}\vec{a} = (\vec{a} \otimes \vec{a})\vec{x},$$

  where $\otimes$ is the *outer product*. For two n-vectors $\vec{a}$ and $\vec{b}$, $\vec{a} \otimes \vec{b}$ is an $n \times n$ with entries:

  $$(\vec{a} \otimes \vec{b})_{ij} = a_i b_j.$$

- For projection onto a plane Π with normal vector $\vec{a} \in \mathbb{R}^3$, $\|\vec{a}\| = 1$, that is $\Pi = \{\vec{x} \in \mathbb{R}^3 : \vec{x} \cdot \vec{a} = 0\}$, we can write the projection operator as:

  $$\vec{x} \mapsto (I_3 - \vec{a} \otimes \vec{a})\vec{x} = \vec{x} - (\vec{x} \cdot \vec{a})\vec{a}.$$

1.5 Comments on these notes

There are two versions of these notes available: as an online website and as a pdf. I expect minor adjustments to be made to both throughout the progress of delivering the materials.

You can always find the latest versions at:

- https://comp2870-2627.github.io/linear-algebra-notes/ (html version)
- https://comp2870-2627.github.io/linear-algebra-notes/COMP2870-Theoretical-Foundations--Linear-Algebra.pdf (pdf version)

Please either email me (T.Ranner@leeds.ac.uk) or use the github repository to report any corrections.

```
This version of the notes is a41c757+dirty.
```

Microsoft copilot has been used to proof read these notes.

Copyright 2024-2026, Thomas Ranner and University of Leeds, licensed under CC BY 4.0.

2 Floating point number systems

Module learning objective

- Explain practical challenges working with floating-point numbers;
- Recognise situations where care is needed when working with floating-point numbers.

This topic will not be presented explicitly in lectures; instead, you will study the material yourself in Lab Session 1.

2.1 Finite precision number systems

Computers store numbers with **finite precision**, i.e. using a finite set of bits (binary digits), typically 32 or 64 of them. You learnt how to store numbers as floating-point numbers last year in the module COMP1860 Building our Digital World.

You will recall that many numbers cannot be stored exactly.

- Some numbers cannot be represented precisely using **any** finite positional representation of digits: e.g. $\sqrt{2} = 1.41421\ldots$, $\pi = 3.14159\ldots$, etc.
- Some cannot be represented precisely in a given number base:
  e.g. $\frac{1}{9} = 0.111\ldots$ (decimal), $\frac{1}{5} = 0.00110011\ldots$ (binary).
- Others can be represented by a finite number of digits but only using more than are available: e.g. 1.526374856437 cannot be stored exactly using 10 decimal digits.

The inaccuracies inherent in finite precision arithmetic must be modelled in order to understand:

- how the numbers are represented (and the nature of associated limitations);
- the errors in their representation;
- the errors which occur when arithmetic operations are applied to them.

The examples shown here will be in **decimal** but the issues apply to any base, *e.g.* **binary**.

This is important when trying to solve problems with floating-point numbers, so that we learn how to avoid the key pitfalls.

2.2 Normalised systems

To understand how this works in practice, we introduce an abstract way to think about the practicalities of floating-point numbers, but our examples will have fewer digits.

Any finite precision number can be written using the floating point representation:

$$x = \pm 0.b_1 b_2 b_3 \ldots b_{t-1} b_t \times \beta^e.$$

- The digits b_i are integers satisfying $0 \leq b_i \leq \beta - 1$.
- The **mantissa**, $b_1 b_2 b_3 \ldots b_{t-1} b_t$, contains t digits.
- β is the **base** (always a positive integer).
- e is the integer **exponent** and is bounded ($L \leq e \leq U$).

(β, t, L, U) fully defines a finite precision number system.

Normalised finite precision systems will be considered here for which

$$b_1 \neq 0 \quad (0 < b_1 \leq \beta - 1).$$

Example 2.1.

1. In the case $(\beta, t, L, U) = (10, 4, -49, 50)$ (base 10),

$$10000 = .1000 \times 10^5, \quad 22.64 = .2264 \times 10^2, \quad 0.0000567 = .5670 \times 10^{-4}$$

2. In the case $(\beta, t, L, U) = (2, 6, -7, 8)$ (binary),

$$10000 = .100000 \times 2^5, \quad 1011.11 = .101111 \times 2^4,$$

$$0.000011 = .110000 \times 2^{-4}.$$

3. **Zero** is always taken to be a special case e.g., $0 = \pm .00 \ldots 0 \times \beta^0$.

Our familiar floating point numbers can (almost) be represented using this format too:

1. The IEEE single-precision standard is $(\beta, t, L, U) = (2, 24, -125, 128)$. This is available via `numpy.float32` or `numpy.single` .
2. The IEEE double-precision standard is $(\beta, t, L, U) = (2, 53, -1021, 1024)$. This is available via `numpy.float64` or `numpy.double`.

Note these values differ from values in other places so we use the $1.b_1 b_2 \ldots$ normalisation instead of the usual after the binary point notation.

```python
import numpy as np

for dtype in [np.float16, np.float32, np.float64]:
    value = dtype(1.1)
    print(dtype.__name__, value, type(value))
```

```
float16 1.1 <class 'numpy.float16'>
float32 1.1 <class 'numpy.float32'>
float64 1.1 <class 'numpy.float64'>
```

IEEE 754 floating point formats include signed zeros, subnormal numbers, infinities and NaN values too. We do not deal with these here ideas in this lecture.

You can create double-precision floating point numbers without using `numpy` (i.e. 64 bit with the same β, t, L, U as above) by calling the `float` function, or just giving the number in decimal format:

```python
python_value = 5.1
numpy_value = np.float64(5.1)

print(type(python_value))
print(type(numpy_value))
print(python_value == numpy_value)
```

```
<class 'float'>
<class 'numpy.float64'>
True
```

On most current platforms, Python's built-in float type uses IEEE 754 binary64 arithmetic. However, the Python language specification does not require every implementation and platform to use precisely the same underlying representation.

Example 2.2. Consider the number system given by $(\beta, t, L, U) = (10, 2, -1, 2)$ which gives

$$x = \pm .b_1 b_2 \times 10^e \text{ where } -1 \leq e \leq 2.$$

a. How many non-zero numbers can be represented by this normalised system?

- The sign can be positive or negative
- b_1 can take on the values 1 to 9 (9 options)

- b_2 can take on the values 0 to 9 (10 options)
- e can take on the values $-1, 0, 1, 2$ (4 options)

Overall this gives:

$$2 \times 9 \times 10 \times 4 \text{ options } = 720 \text{ options.}$$

b. What are the two largest positive numbers in this system?

The largest value uses + as a sign, $b_1 = 9$, $b_2 = 9$ and $e = 2$ which gives

$$+0.99 \times 10^2 = 99.$$

The second largest value uses + as a sign, $b_1 = 9$, $b_2 = 8$ and $e = 2$ which gives

$$+0.98 \times 10^2 = 98.$$

c. What are the two smallest positive numbers?

The smallest positive number has + sign, $b_1 = 1$, $b_2 = 0$ and $e = -1$ which gives

$$+0.10 \times 10^{-1} = 0.01.$$

The second smallest positive number has + sign, $b_1 = 1$, $b_2 = 1$ and $e = -1$ which gives

$$+0.11 \times 10^{-1} = 0.011.$$

d. What is the smallest possible difference between two numbers in this system?

The smallest difference will be between numbers of the form $+0.10 \times 10^{-1}$ and $+0.11 \times 10^{-1}$ which gives

$$0.11 \times 10^{-1} - 0.10 \times 10^{-1} = 0.011 - 0.010 = 0.001.$$

Exercise 2.1. Consider the number system given by $(\beta, t, L, U) = (10, 3, -3, 3)$ which gives

$$x = \pm .b_1 b_2 b_3 \times 10^e \text{ where } -3 \leq e \leq 3.$$

a. How many non-zero numbers can be represented by this normalised system?

b. What are the two largest positive numbers in this system?

c. What are the two smallest positive numbers?

d. What is the smallest possible difference between two numbers in this system?

e. What is the smallest possible difference between two numbers x and y in this system for which $x < 100 < y$?

Example 2.3 (What about in Python)**.** We find that even with double-precision floating point numbers, we see some apparently surprising behaviour when working with decimals:

```
a = np.double(0.0)

# add 0.1 ten times to get 1?
for _ in range(10):
    a = a + np.double(0.1)
    print(a)

print("Is a = 1?", a == 1.0)
```

```
0.1
0.2
0.30000000000000004
0.4
0.5
0.6
0.7
0.7999999999999999
0.8999999999999999
0.9999999999999999
Is a = 1? False
```

Why is this output not a surprise?

We also see that even adding up numbers can have different results depending on the order in which they are added:

```
x = np.double(1e30)
y = np.double(-1e30)
z = np.double(1.0)

print(f"{(x + y) + z=:.16f}")
```

```
(x + y) + z=1.0000000000000000
```

```
print(f"{x + (y + z)=:.16f}")
```

```
x + (y + z)=0.0000000000000000
```

2.3 Errors, machine precision and unit roundoff

From now on $fl(x)$ will be used to represent the (approximate) stored value of x. The error in this representation can be expressed in two ways.

$$\text{Absolute error} = |fl(x) - x|$$
$$\text{Relative error} = \frac{|fl(x) - x|}{|x|}.$$

The number $fl(x)$ is said to approximate x to t **significant digits** (or figures) if t is the largest non-negative integer for which

$$\text{Relative error} < 0.5 \times \beta^{1-t}.$$

More generally, the number of correct significant digits is related to the size of the relative error. A relative error of β^{-d} indicates approximately d correct base-β significant digits.

In the number system given by (β, t, L, U), the nearest (larger) representable number to $x = 0.b_1b_2b_3 \ldots b_{t-1}b_t \times \beta^e$ is

$$\tilde{x} = x + .\underbrace{000\ldots01}_{t \text{ digits}} \times \beta^e = x + \beta^{e-t}$$

Any number $y \in (x, \tilde{x})$ is stored as either x or $\tilde{x}$ by **rounding** to the nearest representable number, so

- the largest possible error is $\frac{1}{2}\beta^{e-t}$,
- which means that $|y - fl(y)| \leq \frac{1}{2}\beta^{e-t}$.

It follows from $y > x \geq .100 \ldots 00 \times \beta^e = \beta^{e-1}$ that

$$\frac{|y - fl(y)|}{|y|} < \frac{1}{2}\frac{\beta^{e-t}}{\beta^{e-1}} = \frac{1}{2}\beta^{1-t},$$

and this provides a bound on the **relative error**: for any y

$$\frac{|y - fl(y)|}{|y|} < \frac{1}{2}\beta^{1-t}.$$

The last term is known as **unit roundoff** and is often written as u. A related quantity is **machine precision epsilon** and is often written as eps:

$$eps = \beta^{1-t} = 2u.$$

This is obtained in Python with

```
eps = np.finfo(np.double).eps
print(eps)
```

```
2.220446049250313e-16
```

Example 2.4.

1. The number system $(\beta, t, L, U) = (10, 2, -1, 2)$ gives

$$eps = \beta^{1-t} = 10^{1-2} = 0.1.$$

2. The number system $(\beta, t, L, U) = (10, 3, -3, 3)$ gives

$$eps = \beta^{1-t} = 10^{1-3} = 0.01.$$

3. The number system $(\beta, t, L, U) = (10, 7, 2, 10)$ gives

$$eps = \beta^{1-t} = 10^{1-7} = 0.000001.$$

4. For some common types in Python, we see the following values:

```
for dtype in [np.float16, np.float32, np.float64]:
    print(dtype.__name__, np.finfo(dtype).eps)
```

```
float16 0.000977
float32 1.1920929e-07
float64 2.220446049250313e-16
```

Machine precision epsilon (*eps*) gives us an upper bound for the error in the representation of a floating-point number in a particular system. We note that this is different to the smallest possible numbers that we can store!

```
eps = np.finfo(np.double).eps
print(f"{eps=}")

smaller = eps
for _ in range(5):
    smaller = smaller / 10.0
    print(f"{smaller=}")
```

```
eps=np.float64(2.220446049250313e-16)
smaller=np.float64(2.2204460492503132e-17)
smaller=np.float64(2.220446049250313e-18)
smaller=np.float64(2.220446049250313e-19)
smaller=np.float64(2.2204460492503132e-20)
smaller=np.float64(2.2204460492503133e-21)
```

Arithmetic operations are usually carried out as though infinite precision is available, after which the result is rounded to the nearest representable number.

This approach means that arithmetic cannot be completely trusted and the usual rules do not necessarily apply!

Example 2.5. Consider the number system $(\beta, t, L, U) = (10, 2, -1, 2)$ and take

$$x = .10 \times 10^2, \quad y = .49 \times 10^0, \quad z = .51 \times 10^0.$$

- In exact arithmetic $x + y = 10 + 0.49 = 10.49$ and $x + z = 10 + 0.51 = 10.51$.
- In this number system rounding gives

$$fl(x + y) = .10 \times 10^2 = x, \qquad fl(x + z) = .11 \times 10^2 \neq x.$$

(Note that $\frac{y}{x} < u$ but $\frac{z}{x} > u$.)

Evaluate the following expression in this number system.

$$x + (y + y), \quad (x + y) + y, \quad x + (z + z), \quad (x + z) + z.$$

(Also note the benefits of adding the *smallest* terms first!)

Exercise 2.2 (More examples)**.**

1. Verify that a similar problem arises for the numbers

$$x = .85 \times 10^0, \quad y = .3 \times 10^{-2}, \quad z = .6 \times 10^{-2},$$

in the system $(\beta, t, L, U) = (10, 2, -3, 3)$.

2. Given the number system $(\beta, t, L, U) = (10, 3, -3, 3)$ and $x = .100 \times 10^3$, find non-zero numbers y and z from this system for which $fl(x + y) = x$ and $fl(x + z) > x$.

It is sometimes helpful to think of machine precision epsilon in another way: **Machine precision epsilon** is the smallest positive number eps such that $1 + eps > 1$, i.e. it is the difference between 1 and the next largest representable number.

Examples:

1. For the number system $(\beta, t, L, U) = (10, 2, -1, 2)$,

$$\begin{array}{rll} & .11 \times 10^1 & \leftarrow \text{next number} \\ - & .10 \times 10^1 & \leftarrow 1 \\ \hline & .01 \times 10^1 & \leftarrow 0.1 \end{array}$$

so $eps = 0.1$.

2. Verify that this approach gives the previously calculated value for eps in the number system given by $(\beta, t, L, U) = (10, 3, -3, 3)$.

3. Here is the Python version:

```python
one = np.float64(1.0)
next_number = np.nextafter(one, np.inf)

print(f"{one=}")
print(f"{next_number=}")
print(f"machine epsilon = {next_number - one}")
print(f"unit roundoff = {(next_number - one) / 2}")
```

```
one=np.float64(1.0)
next_number=np.float64(1.0000000000000002)
machine epsilon = 2.220446049250313e-16
unit roundoff = 1.1102230246251565e-16
```

2.4 Other "Features" of finite precision arithmetic

When working with floating-point numbers there are other things we need to worry about, too!

Overflow The number is too large to be represented, e.g. multiply the largest representable number by 10. This gives `inf` (infinity) with `numpy.float64`s and is usually "fatal".

Underflow The number is too small to be represented, e.g. divide the smallest representable number by 10. This gives 0 and may not be immediately obvious. You may want to read up on subnormal numbers to understand how this works in practice.

Divide by zero when using `numpy` floating point numbers (matching IEEE) gives a result of `inf`, but $\frac{0}{0}$ gives `nan` (not a number), but Python floats instead raise a `ZeroDivisionError`.

Divide by `inf` gives 0.0 with no warning and `inf / inf` produces `NaN`.

2.5 Is this all academic?

No! There are many examples of major software errors that have occurred due to programmers not understanding the issues associated with computer arithmetic…

- In February 1991, a basic rounding error within software for the US Patriot missile system caused it to fail, contributing to the loss of 28 lives.
- In June 1996, the European Space Agency's Ariane 5 Rocket exploded shortly after lift-off: the error was due to failing to handle overflow correctly.
- In October 2020, a driverless car drove straight into a wall due to faulty handling of a floating-point error.

2.6 Summary

- There is inaccuracy in almost all computer arithmetic.
- Care must be taken to minimise its effects, for example:
  - add the smallest terms in an expression first;
  - avoid taking the difference of two very similar terms;
  - even checking whether $a = b$ is dangerous!
- The usual mathematical rules no longer apply.
- There is no point in trying to compute a solution to a problem to a greater accuracy than can be stored by the computer.

2.6.1 Further reading

- David Goldberg, What every computer scientist should know about floating-point arithmetic, ACM Computing Surveys, Volume 23, Issue 1, March 1991.
- John D Cook, Floating point error is the least of my worries, *online*, November 2011.

3 Introduction to systems of linear equations

Module learning objective

- Formulate a system of linear equations in matrix form $A\vec{x} = \vec{b}$.
- Compute the determinant of small matrices by and determine whether a square matrix is invertible using its determinant.
- Explain the difference between solving a system directly and computing a matrix inverse.
- Recognise common matrix structures such as triangular and sparse matrices.

3.1 Systems of linear equations

Given an $n \times n$ matrix A and an n-vector $\vec{b}$, find the n-vector $\vec{x}$ which satisfies:

$$A\vec{x} = \vec{b}. \tag{3.1}$$

We can also write (3.1) as a system of linear equations:

$$\begin{aligned}
\text{Equation 1:} \quad & a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n = b_1 \\
\text{Equation 2:} \quad & a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + \cdots + a_{2n}x_n = b_2 \\
& \vdots \\
\text{Equation i:} \quad & a_{i1}x_1 + a_{i2}x_2 + a_{i3}x_3 + \cdots + a_{in}x_n = b_i \\
& \vdots \\
\text{Equation n:} \quad & a_{n1}x_1 + a_{n2}x_2 + a_{n3}x_3 + \cdots + a_{nn}x_n = b_n.
\end{aligned}$$

Notes:

- The values a_{ij} are known as **coefficients**.
- The **right-hand side** values b_i are known and are given to you as part of the problem.
- $x_1, x_2, x_3, \ldots, x_n$ are **not** known and are what you need to find to solve the problem.

Many computational algorithms require the solution of linear equations, e.g. in fields such as

- scientific computation;
- network design and optimisation;
- graphics and visualisation;
- machine learning.

Typically, these systems are *very* large, with millions or billions of unknowns in some scientific and engineering applications.

It is therefore important that this problem can be solved

- accurately: we are allowed to make small errors but not big errors;
- efficiently: we need to find the answer quickly;
- reliably: we need to know that our algorithm will give us an answer that we are happy with.

In this part of the module we will consider *square* systems of equations for which the number of equations is equal to the number of unknowns. In this case the following statements are equivalent:

- The linear system $A\vec{x} = \vec{b}$ has a unique solution, for any right-hand side vector $\vec{b}$.
- There exists a matrix (let's call it A^{-1} the inverse of A) such that $A^{-1}A = I_n$, and we say that the matrix A is invertible.
- The matrix A is non-singular.

We will see further statements also equivalent to these later in this section of the module.

Example 3.1 (Temperature in a sealed room)**.** Suppose we wish to estimate the temperature distribution inside an object:

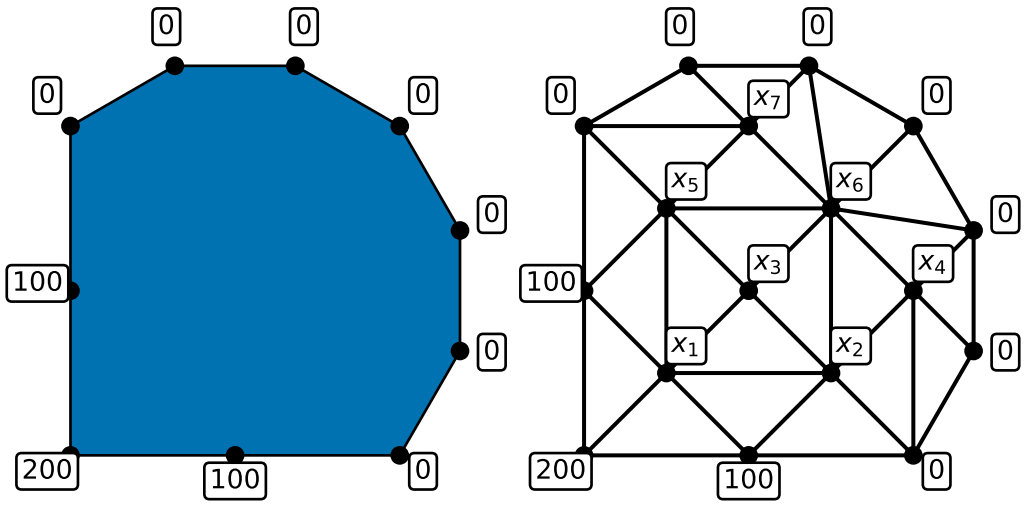


Figure 3.1: Image showing temperature sample points and relations in a room.

We can place a network of points inside the object and use the following model: the temperature at each interior point is the average of its neighbours.

This example leads to the system:

$$\begin{pmatrix} 1 & -1/6 & -1/6 & 0 & -1/6 & 0 & 0 \\ -1/6 & 1 & -1/6 & -1/6 & 0 & -1/6 & 0 \\ -1/4 & -1/4 & 1 & 0 & -1/4 & -1/4 & 0 \\ 0 & -1/5 & 0 & 1 & 0 & -1/5 & 0 \\ -1/6 & 0 & -1/6 & 0 & 1 & -1/6 & -1/6 \\ 0 & -1/8 & -1/8 & -1/8 & -1/8 & 1 & -1/8 \\ 0 & 0 & 0 & 0 & -1/5 & -1/5 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{pmatrix} = \begin{pmatrix} 400/6 \\ 100/6 \\ 0 \\ 0 \\ 100/6 \\ 0 \\ 0 \end{pmatrix}.$$

Example 3.2 (Traffic network)**.** Suppose we wish to monitor the flow of traffic in a city centre:

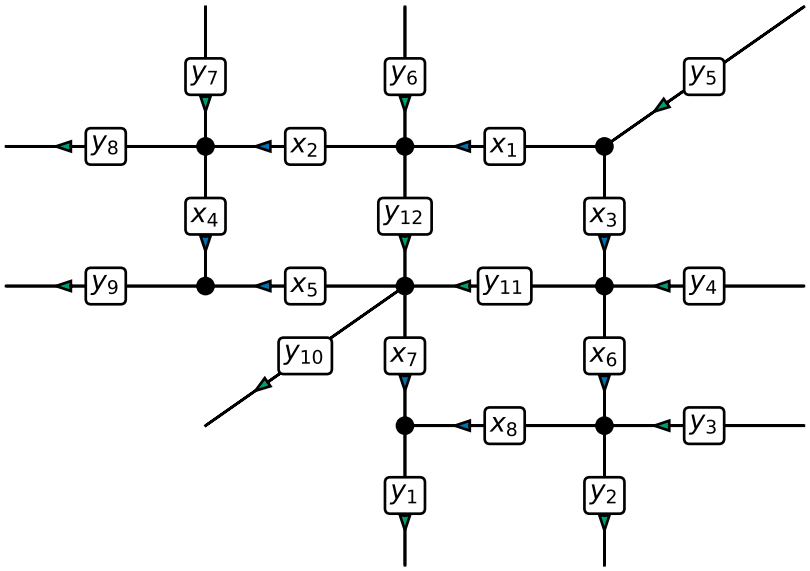


Figure 3.2: Example network showing traffic flow in a city

As the above example shows, it is not necessary to monitor every single road. If we know all of the y values, we can calculate the x values!

This example leads to the system:

$$\begin{pmatrix} 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \\ x_8 \end{pmatrix} = \begin{pmatrix} y_5 \\ y_{12} - y_6 \\ y_8 - y_7 \\ y_{11} - y_4 \\ y_{11} + y_{12} - y_{10} \\ y_9 \\ y_2 - y_3 \\ y_1 \end{pmatrix}.$$

3.2 Matrix inverse

One way to solve a system of linear equations is to compute the inverse of A, A^{-1}, directly, then the solution can be obtained through matrix multiplication: $\vec{x} = A^{-1}\vec{b}$.

In two dimensions this is a simple procedure. Denote by

$$A = \begin{pmatrix} a & b \\ c & d \end{pmatrix},$$

then

$$A^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}.$$

Indeed, we see that

$$\begin{aligned} A^{-1}A &= \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} \begin{pmatrix} a & b \\ c & d \end{pmatrix} \\ &= \frac{1}{ad - bc} \begin{pmatrix} da - bc & db - bd \\ -ca + ac & -bc + ad \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}. \end{aligned}$$

Example 3.3. The inverse of the matrix

$$\begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$$

is

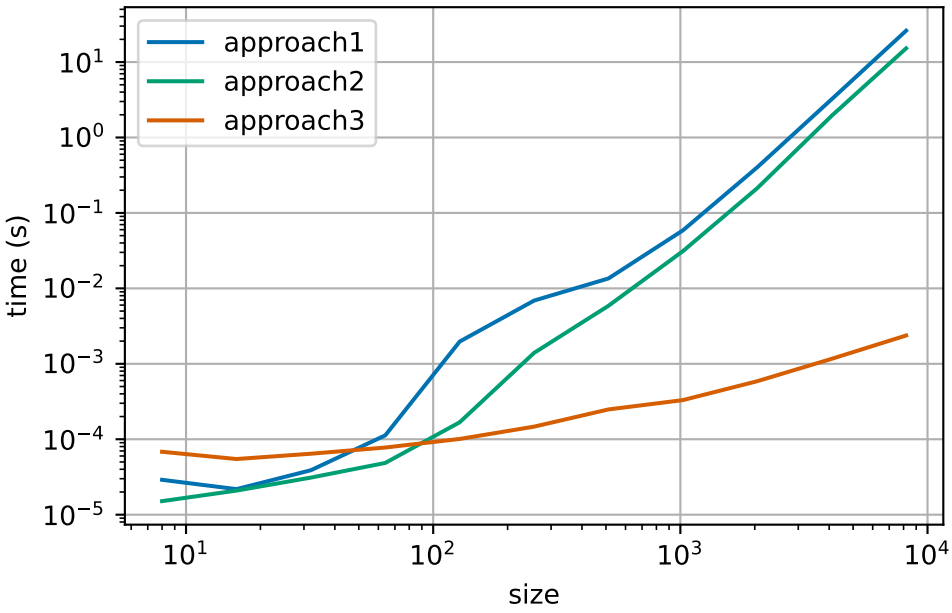
$$\frac{1}{3 \times 3 - 1 \times 1} \begin{pmatrix} 3 & -1 \\ -1 & 3 \end{pmatrix} = \begin{pmatrix} \frac{3}{8} & -\frac{1}{8} \\ -\frac{1}{8} & \frac{3}{8} \end{pmatrix}.$$

There are general formulae for computing larger matrix inverses but it turns out using these to solve systems of linear equations is a bad idea! Here we compare two simple and one more specialised implementations:

```python
def approach1(A, b):
    """
    Solve the system of linear equations Ax = b by applying the inverse of A
    """
    A_inv = np.linalg.inv(A)
    x = A_inv @ b
    return x


def approach2(A, b):
    """
    Solve the system of linear equations Ax = b through numpy's linear solver
    """
    x = np.linalg.solve(A, b)
    return x


def approach3(A_sparse, b):
    """
    Solve the system of linear equations Ax = b through scipy's sparse linear
    solver
    """
    x = sp.sparse.linalg.spsolve(A_sparse, b)
    return x
```



We see that using `np.linalg.solve` to actually solve the equation instead of computing the inverse and then applying the inverse is always faster for the problems considered here. We also see that the specialised solver can do much better than either approach here.

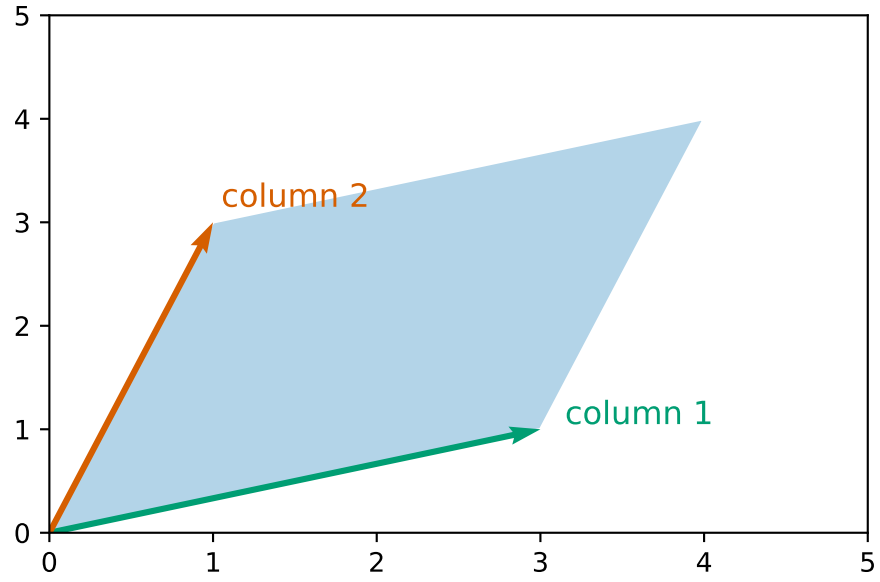
3.3 The determinant

One easy way to test if a matrix is invertible (or that a system of linear equations has a unique solution) is to use the determinant. The determinant of a matrix A can be interpreted geometrically as the signed area, volume, or hypervolume of the parallelogram, parallelepiped, or higher-dimensional analogue generated by the columns of A.

Example 3.4. For A given by

$$A = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$$

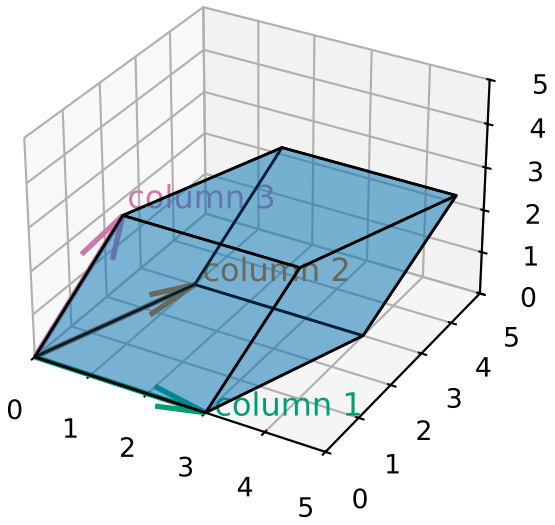
the shape is a parallelogram that looks like this:



For A given by

$$A = \begin{pmatrix} 3 & 1 & 1 \\ 0 & 3 & 1 \\ 0 & 0 & 3 \end{pmatrix},$$

the shape is a parallelepiped given that looks like this:



If we are given an $n \times n$ matrix A, it is invertible if, and only if, the area/volume/hypervolume of the associated parallelogram/parallelepiped is nonzero. The quantity measuring the signed

area, volume, or hypervolume of the associated parallelepiped is called the determinant of A, denoted $\det A$:

Definition 3.1. Let A be a square $n \times n$ matrix.

If $n = 2$,

$$\det A = \det \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} = a_{11}a_{22} - a_{21}a_{12}. \tag{3.2}$$

If $n = 3$,

$$\begin{aligned} \det A &= \det \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} \\ &= a_{11}(a_{22}a_{33} - a_{23}a_{32}) - a_{12}(a_{21}a_{33} - a_{23}a_{31}) + a_{13}(a_{21}a_{32} - a_{22}a_{31}). \end{aligned}$$

For general n, the determinant can be found by, for example, Laplace expansions

$$\det A = \sum_{j=1}^{n} (-1)^{j+1} a_{1,j} M_{1,j},$$

where $a_{1,j}$ is the entry of the first row and jth column of A and $M_{1,j}$ is the the determinant of the submatrix obtained by removing the first row and the jth column from A.

Example 3.5.

1. Let A be given by

$$A = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}.$$

   Then we can compute that

$$\det A = 3 \times 3 - 1 \times 1 = 9 - 1 = 8.$$

2. Let A be given by

$$A = \begin{pmatrix} 3 & 1 & 1 \\ 0 & 3 & 1 \\ 0 & 0 & 3 \end{pmatrix}.$$

   Then we can compute that

$$\begin{aligned} \det A &= 3 \times (3 \times 3 - 0 \times 1) - 1 \times (0 \times 3 - 0 \times 1) + 1 \times (0 \times 0 - 3 \times 0) \\ &= 3 \times 9 - 1 \times 0 + 1 \times 0 = 27. \end{aligned}$$

Exercise 3.1. Compute the determinant of

$$\begin{pmatrix} 2 & -1 & 3 \\ 3 & 2 & -4 \\ 5 & 1 & 1 \end{pmatrix}.$$

This gives us an additional criterion to determine if our systems have a unique solution: A square matrix A is invertible if and only if

$$\det A \neq 0.$$

We have been inaccurate in what we have said before. Strictly speaking the determinant we have defined here is the *signed* area/volume/hypervolume of the associated parallelepiped, and we can recover the actual volume by taking $|\det A|$.

We do have the following rule for manipulating determinants too:

- Let A, B be square $n \times n$-matrices then $\det(AB) = \det A \det B$.
- Let A be a square $n \times n$-matrix and c a real number then $\det(cA) = c^n \det A$.
- Let A be an invertible square matrix, then $\det(A^{-1}) = \frac{1}{\det A}$.

Exercise 3.2. Prove these three rules for manipulating determinants for 2×2 matrices by direct calculation.

Remark 3.1. Computing determinants numerically is an important topic in its own right but we do not focus on this in this module.

You can use `numpy` to compute determinants

```
A = np.array([[3.0, 1.0], [0.0, 3.0]])
np.linalg.det(A)
```

```
np.float64(9.000000000000002)
```

3.4 Special types of matrices

The general matrix A before the examples is known as a **full** matrix: any of its components a_{ij} might be nonzero.

Almost always, the problem being solved leads to a matrix with a particular structure of entries: Some entries may be known to be zero. If this is the case then it is often possible to use this knowledge to improve the efficiency of the algorithm (in terms of both speed and/or storage).

Example 3.6 (Triangular matrix)**.** One common (and important) structure takes the form

$$A = \begin{pmatrix} a_{11} & 0 & 0 & \cdots & 0 \\ a_{21} & a_{22} & 0 & \cdots & 0 \\ a_{31} & a_{32} & a_{33} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{pmatrix}.$$

- A is a **lower triangular** matrix. Every entry above the leading diagonal is zero:

$$a_{ij} = 0 \quad \text{for} \quad j > i.$$

- The *transpose* of this matrix is an **upper triangular** matrix and can be treated in a very similar manner.
- Note that the determinant of a triangular matrix is simply the product of diagonal coefficients:

$$\det A = a_{11} a_{22} \cdots a_{nn} = \prod_{i=1}^{n} a_{ii}.$$

Example 3.7 (Sparse matrices)**. Sparse matrices** are prevalent in any application which relies on some form of *graph* structure (see both the temperature, Example 3.1, and traffic network examples, Example 3.2).

- The a_{ij} typically represents some form of "communication" between vertices i and j of the graph, so the element is only nonzero if the vertices are connected.
- There is no generic pattern for these entries, though there is usually one that is specific to the problem solved.
- Usually, $a_{ii} \neq 0$ - the diagonal is nonzero.
- A "large" portion of the matrix is zero.
  - A full $n \times n$ matrix may have n^2 nonzero entries.
  - A sparse $n \times n$ has αn nonzero entries, where $\alpha \ll n$.
- Many special techniques exist for handling sparse matrices, some of which can be used automatically within Python (`scipy.sparse` documentation)

What is the significance of these special examples?

- In the next section we will discuss a general numerical algorithm for the solution of linear systems of equations.

- This will involve **reducing** the problem to one involving a **triangular matrix**, which, as we show below, is relatively easy to solve.
- In subsequent lectures, we will see that, for *sparse* matrix systems, alternative solution techniques are available.

3.5 Summary

- We have introduced the terminology for systems of linear equations and given some application areas where examples may arise.
- We have seen that we can solve simple systems of linear equations using the matrix inverse but this is not advised for larger systems.
- Finally, we have introduced the determinant as a way of checking that systems of linear equations have a unique solution.

3.6 Further reading

- Gregory Gundersen Why shouldn't I invert that matrix?, 2020.

4 Direct solvers for systems of linear equations

Module learning objective

- Apply Gaussian elimination and backward substitution to solve linear systems;
- Apply LU factorisation to solve linear systems efficiently;
- Evaluate the computational cost and numerical stability of direct linear solvers, including the role of pivoting in finite-precision arithmetic.

This section deals with the first method that we will use to solve systems of linear equations. We will see that by using the approach of Gaussian elimination (and variations of that method), we can solve any system of linear equations that has a solution.

Gaussian elimination should look very familiar to you – this looks a lot like solving simultaneous equations in high school. Here, we are trying to codify this approach into an algorithm that a computer can apply without any further human input in a way that always 'works' (or at least works when it should!).

4.1 Reminder of the problem

Recall the problem is to solve a set of n **linear** equations for n unknown values x_j, for $j = 1, 2, \ldots, n$.

Notation:

$$\begin{aligned}
\text{Equation } 1: &\quad a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n = b_1 \\
\text{Equation } 2: &\quad a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + \cdots + a_{2n}x_n = b_2 \\
&\vdots \\
\text{Equation } i: &\quad a_{i1}x_1 + a_{i2}x_2 + a_{i3}x_3 + \cdots + a_{in}x_n = b_i \\
&\vdots \\
\text{Equation } n: &\quad a_{n1}x_1 + a_{n2}x_2 + a_{n3}x_3 + \cdots + a_{nn}x_n = b_n.
\end{aligned}$$

We can also write the system of linear equations in *general matrix-vector* form:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_n \end{pmatrix}.$$

Recall that the $n \times n$ matrix A represents the coefficients that multiply the unknowns in each equation (row), while the n-vector $\vec{b}$ represents the right-hand-side values.

Our strategy will be to reduce the system to a triangular system of matrices, which is then easy to solve!

4.2 Elementary row operations

Consider equation p of the above system:

$$a_{p1}x_1 + a_{p2}x_2 + a_{p3}x_3 + \cdots + a_{pn}x_n = b_p,$$

and equation q:

$$a_{q1}x_1 + a_{q2}x_2 + a_{q3}x_3 + \cdots + a_{qn}x_n = b_q.$$

Note three things…

- The order in which we choose to write the n equations is irrelevant
- We can multiply any equation by an arbitrary real number ($k \neq 0$ say):

$$ka_{p1}x_1 + ka_{p2}x_2 + ka_{p3}x_3 + \cdots + ka_{pn}x_n = kb_p.$$

- We can add any two equations:

$$ka_{p1}x_1 + ka_{p2}x_2 + ka_{p3}x_3 + \cdots + ka_{pn}x_n = kb_p$$

added to

$$a_{q1}x_1 + a_{q2}x_2 + a_{q3}x_3 + \cdots + a_{qn}x_n = b_q$$

yields

$$(ka_{p1} + a_{q1})x_1 + (ka_{p2} + a_{q2})x_2 + \cdots + (ka_{pn} + a_{qn})x_n = kb_p + b_q.$$

Example 4.1. Consider the system

$$2x_1 + 3x_2 = 4 \tag{4.1}$$
$$-3x_1 + 2x_2 = 7. \tag{4.2}$$

Then we have:

$$\begin{aligned} 4 \times (4.1) \to &\qquad 8x_1 + 12x_2 = 16 \\ -1.5 \times (4.2) \to &\qquad 4.5x_1 - 3x_2 = -10.5 \\ (4.1) + (4.2) \to &\qquad -x_1 + 5x_2 = 11 \\ (4.2) + 1.5 \times (4.1) \to &\qquad 0 + 6.5x_2 = 13. \end{aligned}$$

Exercise 4.1. Consider the system

$$x_1 + 2x_2 = 1 \tag{4.3}$$
$$4x_1 + x_2 = -3. \tag{4.4}$$

Work out the result of these elementary row operations:

$$\begin{aligned} 2 \times (4.3) \to \\ 0.25 \times (4.4) \to \\ (4.4) + (-1) \times (4.3) \to \\ (4.4) + (-4) \times (4.3) \to \end{aligned}$$

For a system written in matrix form our three observations mean the following:

- We can swap any two rows of the matrix (and corresponding right-hand side entries). For example:

$$\begin{pmatrix} 2 & 3 \\ -3 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 4 \\ 7 \end{pmatrix} \Rightarrow \begin{pmatrix} -3 & 2 \\ 2 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 7 \\ 4 \end{pmatrix}$$

- We can multiply any row of the matrix (and corresponding right-hand side entry) by a scalar. For example:

$$\begin{pmatrix} 2 & 3 \\ -3 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 4 \\ 7 \end{pmatrix} \Rightarrow \begin{pmatrix} 1 & \frac{3}{2} \\ -3 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 2 \\ 7 \end{pmatrix}$$

- We can replace row q by row $q + k\times$ row p. For example:

$$\begin{pmatrix} 2 & 3 \\ -3 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 4 \\ 7 \end{pmatrix} \Rightarrow \begin{pmatrix} 2 & 3 \\ 0 & 6.5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 4 \\ 13 \end{pmatrix}$$

  (here we replaced row 2 by row $2 + 1.5\times$ row 1)

$$\begin{pmatrix} 1 & 2 \\ 4 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ -3 \end{pmatrix} \Rightarrow \begin{pmatrix} 1 & 2 \\ 0 & -7 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ -7 \end{pmatrix}$$

  (here we replaced row 2 by row $2 + (-4)\times$ row 1)

Our strategy for solving systems of linear equations using Gaussian elimination is based on the following ideas:

- The three types of operation described above are called **elementary row operations** (ERO).
- We will apply a sequence of ERO to reduce an arbitrary system to a triangular form, which, we will see, can be easily solved.
- The algorithm for reducing a general matrix to upper triangular form is known as **forward elimination** or (more commonly) as **Gaussian elimination**.

4.3 Gaussian elimination

The algorithm of Gaussian elimination is an ancient method that you may have already met at school – perhaps by a different name. The details of the method may seem confusing at first. We are following the ideas of elimination from systems of simultaneous equations, but in a way that a computer understands. Thinking like a computer is an important concept for computer scientists. You will have seen different ideas of how to solve systems of simultaneous equations where the first step is to "look at the equations to decide the easiest first step". When there are 10^9 equations, it is not effective for a computer to try to find an easy way through the problem. The computer must be given a simple set of instructions to follow: this will be our algorithm.

The method is so old that in fact we have evidence of Chinese mathematicians using forms of Gaussian elimination in 179 CE (From Wikipedia):

The method of Gaussian elimination appears in the Chinese mathematical text Chapter Eight: Rectangular Arrays of The Nine Chapters on the Mathematical Art. Its use is illustrated in eighteen problems, with two to five equations. The first reference to the book by this title is dated to 179 CE, but parts of it were written as early as approximately 150 BCE. It was commented on by Liu Hui in the 3rd century.

The method in Europe stems from the notes of Isaac Newton. In 1670, he wrote that all the algebra books known to him lacked a lesson for solving simultaneous equations, which Newton then supplied. Carl Friedrich Gauss in 1810 devised a notation for symmetric elimination that was adopted in the 19th century by professional hand computers to solve the normal equations of least-squares problems. The algorithm that is taught in high school was named for Gauss only in the 1950s as a result of confusion over the history of the subject.

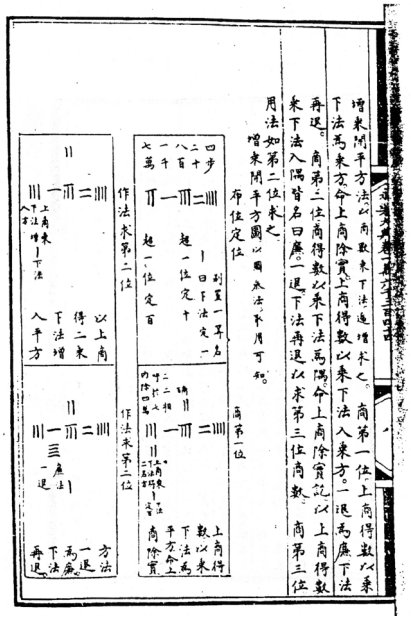


Figure 4.1: Image from Nine Chapter of the Mathematical art. By Yang Hui(1238-1298) - mybook, Public Domain, https://commons.wikimedia.org/w/index.php?curid=10317744

4.3.1 The algorithm

The following algorithm systematically introduces zeros into the system of equations, below the diagonal.

1. Subtract multiples of row 1 from the rows below it to eliminate (make zero) non-zero entries in column 1.
2. Subtract multiples of the new row 2 from the rows below it to eliminate non-zero entries in column 2.
3. Repeat for row $3, 4, \ldots, n-1$.

After row $n-1$, all entries below the diagonal have been eliminated, so A is now upper triangular and the resulting system can be solved by backward substitution.

Example 4.2. Use Gaussian elimination to reduce the following system of equations to upper triangular form:

$$\begin{pmatrix} 2 & 1 & 4 \\ 1 & 2 & 2 \\ 2 & 4 & 6 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 12 \\ 9 \\ 22 \end{pmatrix}.$$

First, use the first row to eliminate the first column below the diagonal:

- (row 2) $-0.5\times$ (row 1) gives

$$\begin{pmatrix} 2 & 1 & 4 \\ \mathbf{0} & 1.5 & 0 \\ 2 & 4 & 6 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 12 \\ 3 \\ 22 \end{pmatrix}$$

- (row 3) $-$ (row 1) then gives

$$\begin{pmatrix} 2 & 1 & 4 \\ \mathbf{0} & 1.5 & 0 \\ \mathbf{0} & 3 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 12 \\ 3 \\ 10 \end{pmatrix}$$

Now use the second row to eliminate the second column below the diagonal.

- (row 3) $-2\times$ (row 2) gives

$$\begin{pmatrix} 2 & 1 & 4 \\ \mathbf{0} & 1.5 & 0 \\ \mathbf{0} & \mathbf{0} & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 12 \\ 3 \\ 4 \end{pmatrix}$$

Exercise 4.2. Use Gaussian elimination to reduce the following system of linear equations to upper triangular form.

$$\begin{pmatrix} 4 & -1 & -1 \\ 2 & 4 & 2 \\ 1 & 2 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 9 \\ -6 \\ 3 \end{pmatrix}.$$

The key idea is that:

- Each row i is used to eliminate the entries in column i below the diagonal (a_{ii}), i.e. it forces $a_{ji} = 0$ for $j > i$.
- The elimination is done by subtracting a multiple of row i from row j:

$$(\text{row } j) \leftarrow (\text{row } j) - \frac{a_{ji}}{a_{ii}}(\text{row } i).$$

- Our update formula guarantees that a_{ji} becomes zero because

$$a_{ji} \leftarrow a_{ji} - \frac{a_{ji}}{a_{ii}} a_{ii} = a_{ji} - a_{ji} = 0.$$

4.3.2 Python version

We start with some helper code which determines the size of the system we are working with:

```
def system_size(A, b):
    """
    Validate the dimensions of a linear system and return its size.

    This function checks whether the given coefficient matrix `A` is square
    and whether its dimensions are compatible with the right-hand side vector
    `b`. If the dimensions are valid, it returns the size of the system.

    Parameters
    ----------
    A : numpy.ndarray
        A 2D array of shape ``(n, n)`` representing the coefficient matrix of
        the linear system.
    b : numpy.ndarray
        An array of shape ``(n, 1)`` representing the right-hand side vector.

```

```
    Returns
    -------
    int
        The size of the system, i.e., the number of variables `n`.

    Raises
    ------
    ValueError
        If `A` is not square or if the size of `b` does not match the
        number of rows in `A`.
    """

    # Validate that A is a 2D square matrix
    if A.ndim != 2:
        raise ValueError(f"Matrix A must be 2D, but got {A.ndim}D array")

    n, m = A.shape
    if n != m:
        raise ValueError(f"Matrix A must be square, got A.shape={A.shape}")

    if b.shape[0] != n:
        raise ValueError(
            f"System shapes are not compatible: A.shape={A.shape}, "
            f"b.shape={b.shape}"
        )

    return n
```

Then we can implement the elementary row operations

```
def row_swap(A, b, p, q):
    """
    Swap two rows in a linear system of equations in place.

    This function swaps the rows `p` and `q` of the coefficient matrix `A`
    and the right-hand side vector `b` for a linear system of equations
    of the form ``Ax = b``. The operation is performed **in place**, modifying
    the input arrays directly.

    Parameters
    ----------
    A : numpy.ndarray
```

```
        A 2D NumPy array of shape ``(n, n)`` representing the coefficient
        matrix of the linear system.
    b : numpy.ndarray
        A 2D NumPy array of shape ``(n, 1)`` representing the right-hand side
        vector of the system.
    p : int
        The index of the first row to swap. Must satisfy ``0 <= p < n``.
    q : int
        The index of the second row to swap. Must satisfy ``0 <= q < n``.

    Returns
    -------
    None
        This function modifies `A` and `b` directly and does not return
        anything.
    """
    # get system size
    n = system_size(A, b)
    # swap rows of A
    for j in range(n):
        A[p, j], A[q, j] = A[q, j], A[p, j]
    # swap rows of b
    b[p, 0], b[q, 0] = b[q, 0], b[p, 0]


def row_scale(A, b, p, k):
    """
    Scale a row of a linear system by a constant factor in place.

    This function multiplies all entries in row `p` of the coefficient matrix
    `A` and the corresponding entry in the right-hand side vector `b` by a
    scalar `k`. The operation is performed **in place**, modifying the input
    arrays directly.

    Parameters
    ----------
    A : numpy.ndarray
        A 2D NumPy array of shape ``(n, n)`` representing the coefficient
        matrix of the linear system.
    b : numpy.ndarray
        A 2D NumPy array of shape ``(n, 1)`` representing the right-hand side
        vector of the system.
```

```
    p : int
        The index of the row to scale. Must satisfy ``0 <= p < n``.
    k : float
        The scalar multiplier applied to the entire row.

    Returns
    -------
    None
        This function modifies `A` and `b` directly and does not return
        anything.
    """
    n = system_size(A, b)

    # scale row p of A
    for j in range(n):
        A[p, j] = k * A[p, j]
    # scale row p of b
    b[p, 0] = b[p, 0] * k


def row_add(A, b, p, k, q):
    """
    Perform an in-place row addition operation on a linear system.

    This function applies the elementary row operation:

    ``row_p ← row_p + k * row_q``

    where `row_p` and `row_q` are rows in the coefficient matrix `A` and the
    right-hand side vector `b`. It updates the entries of `A` and `b`
    **in place**, directly modifying the original data.

    Parameters
    ----------
    A : numpy.ndarray
        A 2D NumPy array of shape ``(n, n)`` representing the coefficient
        matrix of the linear system.
    b : numpy.ndarray
        A 2D NumPy array of shape ``(n, 1)`` representing the right-hand side
        vector of the system.
    p : int
        The index of the row to be updated (destination row). Must satisfy
```

```
        ``0 <= p < n``.
    k : float
        The scalar multiplier applied to `row_q` before adding it to `row_p`.
    q : int
        The index of the source row to be scaled and added. Must satisfy
        ``0 <= q < n``.
    """
    n = system_size(A, b)

    # Perform the row operation
    for j in range(n):
        A[p, j] = A[p, j] + k * A[q, j]

    # Update the corresponding value in b
    b[p, 0] = b[p, 0] + k * b[q, 0]
```

Let's test we are ok so far:

Test 1: swapping rows

```
A = np.array([[2.0, 3.0], [-3.0, 2.0]])
b = np.array([[4.0], [7.0]])

print("starting arrays:")
print_array(A)
print_array(b)
print()

print("swapping rows 0 and 1")
row_swap(A, b, 0, 1)  # remember NumPy arrays are indexed starting from zero!
print()

print("new arrays:")
print_array(A)
print_array(b)
print()
```

```
starting arrays:
A = [  2.0,  3.0 ]
    [ -3.0,  2.0 ]
b = [  4.0 ]
    [  7.0 ]
```

```
swapping rows 0 and 1

new arrays:
A = [ -3.0,  2.0 ]
    [  2.0,  3.0 ]
b = [  7.0 ]
    [  4.0 ]
```

Test 2: scaling one row

```
A = np.array([[2.0, 3.0], [-3.0, 2.0]])
b = np.array([[4.0], [7.0]])

print("starting arrays:")
print_array(A)
print_array(b)
print()

print("row 0 |-> 0.5 * row 0")
row_scale(A, b, 0, 0.5)
print()

print("new arrays:")
print_array(A)
print_array(b)
print()
```

```
starting arrays:
A = [  2.0,  3.0 ]
    [ -3.0,  2.0 ]
b = [  4.0 ]
    [  7.0 ]

row 0 |-> 0.5 * row 0

new arrays:
A = [  1.0,  1.5 ]
    [ -3.0,  2.0 ]
b = [  2.0 ]
    [  7.0 ]
```

Test 3: replacing a row by that adding a multiple of another row

```
A = np.array([[2.0, 3.0], [-3.0, 2.0]])
b = np.array([[4.0], [7.0]])

print("starting arrays:")
print_array(A)
print_array(b)
print()

print("row 1 |-> row 1 + 1.5 * row 0")
# remember NumPy arrays are indexed starting from zero!
row_add(A, b, 1, 1.5, 0)
print()

print("new arrays:")
print_array(A)
print_array(b)
print()
```

```
starting arrays:
A = [  2.0,  3.0 ]
    [ -3.0,  2.0 ]
b = [  4.0 ]
    [  7.0 ]

row 1 |-> row 1 + 1.5 * row 0

new arrays:
A = [  2.0,  3.0 ]
    [  0.0,  6.5 ]
b = [  4.0 ]
    [ 13.0 ]
```

Now we can define our Gaussian elimination function. We update the values in-place to avoid extra memory allocations.

```
def gaussian_elimination(A, b, verbose=False):
    """
    Perform Gaussian elimination to reduce a linear system to upper triangular
    form.

```

```
    This function performs **forward elimination** to transform the
    coefficient matrix `A` into an upper triangular matrix, while applying
    the same operations to the right-hand side vector `b`. This is the
    first step in solving a linear system of equations of the form
    ``Ax = b`` using Gaussian elimination.

    Parameters
    ----------
    A : numpy.ndarray
        A 2D NumPy array of shape ``(n, n)`` representing the coefficient
        matrix of the system.
    b : numpy.ndarray
        A 2D NumPy array of shape ``(n, 1)`` representing the right-hand side
        vector.
    verbose : bool, optional
        If ``True``, prints detailed information about each elimination step,
        including the row operations performed and the intermediate forms of
        `A` and `b`. Default is ``False``.

    Returns
    -------
    None
        This function modifies `A` and `b` **in place** and does not return
        anything.
    """
    # find shape of system
    n = system_size(A, b)

    # perform forward elimination
    for i in range(n - 1):
        # eliminate column i
        if verbose:
            print(f"eliminating column {i}")
        for j in range(i + 1, n):
            # row j
            factor = A[j, i] / A[i, i]
            if verbose:
                print(f"  row {j} |-> row {j} - {factor} * row {i}")
            row_add(A, b, j, -factor, i)

        if verbose:
            print()
```

```
            print("new system")
            print_array(A)
            print_array(b)
            print()
```

We can try our code on Example 1:

```
A = np.array([[2.0, 1.0, 4.0], [1.0, 2.0, 2.0], [2.0, 4.0, 6.0]])
b = np.array([[12.0], [9.0], [22.0]])

print("starting system:")
print_array(A)
print_array(b)
print()

print("performing Gaussian Elimination")
gaussian_elimination(A, b, verbose=True)
print()

print("final system:")
print_array(A)
print_array(b)
print()

# test that A is really upper triangular
print("Is A really upper triangular?", np.allclose(A, np.triu(A)))
```

```
starting system:
A = [  2.0,  1.0,  4.0 ]
    [  1.0,  2.0,  2.0 ]
    [  2.0,  4.0,  6.0 ]
b = [ 12.0 ]
    [  9.0 ]
    [ 22.0 ]

performing Gaussian Elimination
eliminating column 0
  row 1 |-> row 1 - 0.5 * row 0
  row 2 |-> row 2 - 1.0 * row 0

new system
```

```
A = [  2.0,  1.0,  4.0 ]
    [  0.0,  1.5,  0.0 ]
    [  0.0,  3.0,  2.0 ]
b = [ 12.0 ]
    [  3.0 ]
    [ 10.0 ]

eliminating column 1
  row 2 |-> row 2 - 2.0 * row 1

new system
A = [  2.0,  1.0,  4.0 ]
    [  0.0,  1.5,  0.0 ]
    [  0.0,  0.0,  2.0 ]
b = [ 12.0 ]
    [  3.0 ]
    [  4.0 ]


final system:
A = [  2.0,  1.0,  4.0 ]
    [  0.0,  1.5,  0.0 ]
    [  0.0,  0.0,  2.0 ]
b = [ 12.0 ]
    [  3.0 ]
    [  4.0 ]

Is A really upper triangular? True
```

4.4 Solving triangular systems of equations

A general *lower triangular* system of equations has $a_{ij} = 0$ for $j > i$ and takes the form:

$$\begin{pmatrix} a_{11} & 0 & 0 & \cdots & 0 \\ a_{21} & a_{22} & 0 & \cdots & 0 \\ a_{31} & a_{32} & a_{33} & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_n \end{pmatrix}.$$

Note that the first equation is

$$a_{11}x_1 = b_1.$$

Then x_i can be found by calculating

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j \right)$$

for each row $i = 1, 2, \ldots, n$ in turn.

- Each calculation only involves values x_j that have already been computed ($j < i$).
- The matrix A **must** have non-zero diagonal entries i.e. $a_{ii} \neq 0$ for $i = 1, 2, \ldots, n$.
- **Upper triangular** systems of equations can be solved similarly.

Example 4.3. Solve the lower triangular system of equations given by

$$\begin{aligned} 2x_1 &= 2 \\ x_1 + 2x_2 &= 7 \\ 2x_1 + 4x_2 + 6x_3 &= 26 \end{aligned}$$

or, equivalently,

$$\begin{pmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ 2 & 4 & 6 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 2 \\ 7 \\ 26 \end{pmatrix}.$$

The solution can be calculated systematically from

$$\begin{aligned} x_1 &= \frac{b_1}{a_{11}} = \frac{2}{2} = 1 \\ x_2 &= \frac{b_2 - a_{21}x_1}{a_{22}} = \frac{7 - 1 \times 1}{2} = \frac{6}{2} = 3 \\ x_3 &= \frac{b_3 - a_{31}x_1 - a_{32}x_2}{a_{33}} = \frac{26 - 2 \times 1 - 4 \times 3}{6} = \frac{12}{6} = 2 \end{aligned}$$

which gives the solution $\vec{x} = (1, 3, 2)^T$.

Exercise 4.3. Solve the upper triangular linear system given by

$$\begin{aligned} 2x_1 + \quad x_2 + \quad 4x_3 &= \quad 12 \\ 1.5x_2 \quad\quad &= \quad 3. \\ 2x_3 &= \quad 4 \end{aligned}$$

Remark 4.1.

- It is simple to solve a lower (upper) triangular system of equations (provided the diagonal is non-zero).
- This process is often referred to as **forward** (**backward**) **substitution**.
- A general system of equations (i.e. a full matrix A) can be solved rapidly once it has been reduced to upper triangular form. The entire process is called Gaussian elimination with backward substitution.

We can define functions to solve both upper and lower triangular form systems of linear equations:

```
def forward_substitution(A, b):
    """
    Solve a lower triangular system of linear equations using forward
    substitution.

    This function solves the system of equations:

    .. math::
        A x = b

    where `A` is a **lower triangular matrix** (all elements above the main
    diagonal are zero). The solution vector `x` is computed sequentially by
    solving each equation starting from the first row.

    Parameters
    ----------
    A : numpy.ndarray
        A 2D NumPy array of shape ``(n, n)`` representing the lower triangular
        coefficient matrix of the system.
    b : numpy.ndarray
        A 1D NumPy array of shape ``(n,)`` or a 2D NumPy array of shape
        ``(n, 1)`` representing the right-hand side vector.

```

```
    Returns
    -------
    x : numpy.ndarray
        A NumPy array of shape ``(n,)`` containing the solution vector.
    """
    # get size of system
    n = system_size(A, b)

    # check is lower triangular
    if not np.allclose(A, np.tril(A)):
        raise ValueError("Matrix A is not lower triangular")

    # create solution variable
    x = np.empty_like(b)

    # perform forwards solve
    for i in range(n):
        partial_sum = 0.0
        for j in range(0, i):
            partial_sum += A[i, j] * x[j]
        x[i] = 1.0 / A[i, i] * (b[i] - partial_sum)

    return x


def backward_substitution(A, b):
    """
    Solve an upper triangular system of linear equations using backward
    substitution.

    This function solves the system of equations:

    .. math::
        A x = b

    where `A` is an **upper triangular matrix** (all elements below the main
    diagonal are zero). The solution vector `x` is computed starting from the
    last equation and proceeding backward.

    Parameters
    ----------
    A : numpy.ndarray
```

```
        A 2D NumPy array of shape ``(n, n)`` representing the upper triangular
        coefficient matrix of the system.
    b : numpy.ndarray
        A 1D NumPy array of shape ``(n,)`` or a 2D NumPy array of shape
        ``(n, 1)`` representing the right-hand side vector.

    Returns
    -------
    x : numpy.ndarray
        A NumPy array of shape ``(n,)`` containing the solution vector.
    """
    # get size of system
    n = system_size(A, b)

    # check is upper triangular
    if not np.allclose(A, np.triu(A)):
        raise ValueError("Matrix A is not upper triangular")

    # create solution variable
    x = np.empty_like(b)

    # perform backward solve
    for i in range(n - 1, -1, -1):  # iterate over rows backwards
        partial_sum = 0.0
        for j in range(i + 1, n):
            partial_sum += A[i, j] * x[j]
        x[i] = 1.0 / A[i, i] * (b[i] - partial_sum)

    return x
```

And we can then test it out!

```
A = np.array([[2.0, 0.0, 0.0], [1.0, 2.0, 0.0], [2.0, 4.0, 6.0]])
b = np.array([[2.0], [7.0], [26.0]])

print("The system is given by:")
print_array(A)
print_array(b)
print()

print("Solving the system using forward substitution")
x = forward_substitution(A, b)
```

```
print()

print("The solution using forward substitution is:")
print_array(x)
print()

print("Does x really solve the system?", np.allclose(A @ x, b))
```

```
The system is given by:
A = [  2.0,  0.0,  0.0 ]
    [  1.0,  2.0,  0.0 ]
    [  2.0,  4.0,  6.0 ]
b = [  2.0 ]
    [  7.0 ]
    [ 26.0 ]

Solving the system using forward substitution

The solution using forward substitution is:
x = [  1.0 ]
    [  3.0 ]
    [  2.0 ]

Does x really solve the system? True
```

We can also do a backward substitution test:

```
A = np.array([[2.0, 1.0, 4.0], [0.0, 1.5, 0.0], [0.0, 0.0, 2.0]])
b = np.array([[12.0], [3.0], [4.0]])

print("The system is given by:")
print_array(A)
print_array(b)
print()

print("Solving the system using backward substitution")
x = backward_substitution(A, b)
print()

print("The solution using backward substitution is:")
print_array(x)
```

```
print()

print("Does x really solve the system?", np.allclose(A @ x, b))
```

```
The system is given by:
A = [  2.0,  1.0,  4.0 ]
    [  0.0,  1.5,  0.0 ]
    [  0.0,  0.0,  2.0 ]
b = [ 12.0 ]
    [  3.0 ]
    [  4.0 ]

Solving the system using backward substitution

The solution using backward substitution is:
x = [  1.0 ]
    [  2.0 ]
    [  2.0 ]

Does x really solve the system? True
```

4.5 Combining Gaussian elimination and backward substitution

Our grand strategy can now come together so we have a method to solve systems of linear equations:

Given a system of linear equations $A\vec{x} = \vec{b}$;

- First perform Gaussian elimination to give an equivalent system of equations in upper triangular form;
- Then use backward substitution to produce a solution $\vec{x}$

We can use our code to test this:

```
A = np.array([[2.0, 1.0, 4.0], [1.0, 2.0, 2.0], [2.0, 4.0, 6.0]])
b = np.array([[12.0], [9.0], [22.0]])

print("starting system:")
print_array(A)
print_array(b)
print()
```

```

print("performing Gaussian Elimination")
gaussian_elimination(A, b, verbose=True)
print()

print("upper triangular system:")
print_array(A)
print_array(b)
print()

print("Solving the system using backward substitution")
x = backward_substitution(A, b)
print()

print("solution using backward substitution:")
print_array(x)
print()

A = np.array([[2.0, 1.0, 4.0], [1.0, 2.0, 2.0], [2.0, 4.0, 6.0]])
b = np.array([[12.0], [9.0], [22.0]])
print("Does x really solve the original system?", np.allclose(A @ x, b))
```

```
starting system:
A = [  2.0,  1.0,  4.0 ]
    [  1.0,  2.0,  2.0 ]
    [  2.0,  4.0,  6.0 ]
b = [ 12.0 ]
    [  9.0 ]
    [ 22.0 ]

performing Gaussian Elimination
eliminating column 0
  row 1 |-> row 1 - 0.5 * row 0
  row 2 |-> row 2 - 1.0 * row 0

new system
A = [  2.0,  1.0,  4.0 ]
    [  0.0,  1.5,  0.0 ]
    [  0.0,  3.0,  2.0 ]
b = [ 12.0 ]
    [  3.0 ]
    [ 10.0 ]
```

```
eliminating column 1
  row 2 |-> row 2 - 2.0 * row 1

new system
A = [  2.0,  1.0,  4.0 ]
    [  0.0,  1.5,  0.0 ]
    [  0.0,  0.0,  2.0 ]
b = [ 12.0 ]
    [  3.0 ]
    [  4.0 ]


upper triangular system:
A = [  2.0,  1.0,  4.0 ]
    [  0.0,  1.5,  0.0 ]
    [  0.0,  0.0,  2.0 ]
b = [ 12.0 ]
    [  3.0 ]
    [  4.0 ]

Solving the system using backward substitution

solution using backward substitution:
x = [  1.0 ]
    [  2.0 ]
    [  2.0 ]

Does x really solve the original system? True
```

Exercise 4.4. Use Gaussian elimination followed by backward substitution to solve the system:

$$\begin{pmatrix} 2 & 1 & -1 \\ 4 & 5 & 7 \\ -8 & -3 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 6 \\ 30 \\ -22 \end{pmatrix}.$$

4.6 The cost of Gaussian elimination

Gaussian elimination (GE) is unnecessarily expensive when it is applied to many systems of equations with the same matrix A but different right-hand sides $\vec{b}$.

- The forward elimination process is the most computationally expensive part at $O(n^3)$ but is exactly the same for any choice of $\vec{b}$.
- In contrast, the solution of the resulting upper triangular system only requires $O(n^2)$ operations.

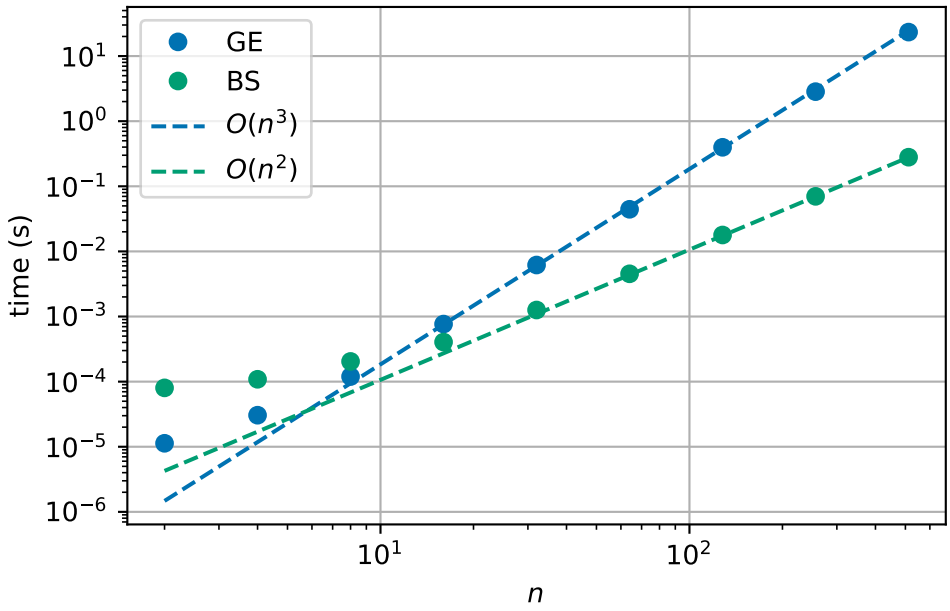


Figure 4.2: Runtime cost of applying Gaussian elimination (GE) and backward solution (BS).

We can use this information to improve the way in which we solve multiple systems of equations with the same matrix A but different right-hand sides $\vec{b}$.

4.7 LU factorisation

Our next algorithm, called LU factorisation, is a way to try to speed up Gaussian elimination by reusing information. This can be used when we solve systems of equations with the same matrix A but different right-hand sides $\vec{b}$ - this is more common than you would think!

Recall the elementary row operations (EROs) from above. Note that the EROs can be produced by left multiplication with a suitable matrix:

- Row swap:

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} = \begin{pmatrix} a & b & c \\ g & h & i \\ d & e & f \end{pmatrix}$$

- Row swap:

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{pmatrix} = \begin{pmatrix} a & b & c & d \\ i & j & k & l \\ e & f & g & h \\ m & n & o & p \end{pmatrix}$$

- Multiply row by α:

$$\begin{pmatrix} \alpha & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} = \begin{pmatrix} \alpha a & \alpha b & \alpha c \\ d & e & f \\ g & h & i \end{pmatrix}$$

- $\alpha \times$ row $p +$ row q:

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ \alpha & 0 & 1 \end{pmatrix} \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} = \begin{pmatrix} a & b & c \\ d & e & f \\ \alpha a + g & \alpha b + h & \alpha c + i \end{pmatrix}$$

Since Gaussian elimination (GE) is just a sequence of EROs and each ERO is just multiplication by a suitable matrix, say E_k, forward elimination applied to the system $A\vec{x} = \vec{b}$ can be expressed as

$$(E_m \cdots E_1)A\vec{x} = (E_m \cdots E_1)\vec{b},$$

here m is the number of EROs required to reduce the upper triangular form.

Let $U = (E_m \cdots E_1)A$ and $L = (E_m \cdots E_1)^{-1}$. Now the original system $A\vec{x} = \vec{b}$ is equivalent to

$$LU\vec{x} = \vec{b} \tag{4.5}$$

where U is *upper triangular* (by construction) and L may be shown to be lower triangular (provided the EROs do not include any row swaps).

Once L and U are known it is easy to solve (4.5)

- Solve $L\vec{z} = \vec{b}$ in $O(n^2)$ operations.
- Solve $U\vec{x} = \vec{z}$ in $O(n^2)$ operations.

L and U may be found in $O(n^3)$ operations by performing GE and saving the E_i matrices: however, it is more convenient to find them directly (also $O(n^3)$ operations).

4.7.1 Computing L and U

Consider a general 4×4 matrix A and its factorisation LU:

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ l_{21} & 1 & 0 & 0 \\ l_{31} & l_{32} & 1 & 0 \\ l_{41} & l_{42} & l_{43} & 1 \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ 0 & u_{22} & u_{23} & u_{24} \\ 0 & 0 & u_{33} & u_{34} \\ 0 & 0 & 0 & u_{44} \end{pmatrix}$$

For the first column,

$$\begin{aligned} a_{11} &= (1,0,0,0)(u_{11},0,0,0)^T &&= u_{11} && \to u_{11} = a_{11} \\ a_{21} &= (l_{21},1,0,0)(u_{11},0,0,0)^T &&= l_{21}u_{11} && \to l_{21} = a_{21}/u_{11} \\ a_{31} &= (l_{31},l_{32},1,0)(u_{11},0,0,0)^T &&= l_{31}u_{11} && \to l_{31} = a_{31}/u_{11} \\ a_{41} &= (l_{41},l_{42},l_{43},1)(u_{11},0,0,0)^T &&= l_{41}u_{11} && \to l_{41} = a_{41}/u_{11} \end{aligned}$$

The second, third and fourth columns follow in a similar manner, giving all the entries in L and U.

Remark 4.2.

- L is assumed to have 1's on the diagonal, to ensure that the factorisation is unique.
- The process involves division by the diagonal entries u_{11}, u_{22}, etc., so they **must** be non-zero.
- In general the factors l_{ij} and u_{ij} are calculated for each column j in turn, i.e.,

```
for j in range(n):
  for i in range(j+1):
      # Compute factors u_{ij}
      ...
  for i in range(j+1, n):
      # Compute factors l_{ij}
      ...
```

Example 4.4. Use LU factorisation to solve the linear system of equations given by

$$\begin{pmatrix} 2 & 1 & 4 \\ 1 & 2 & 2 \\ 2 & 4 & 6 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 12 \\ 9 \\ 22 \end{pmatrix}.$$

This can be rewritten in the form $A = LU$ where

$$\begin{pmatrix} 2 & 1 & 4 \\ 1 & 2 & 2 \\ 2 & 4 & 6 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ l_{21} & 1 & 0 \\ l_{31} & l_{32} & 1 \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{pmatrix}.$$

Column 1 of A gives

$$\begin{aligned} 2 &= u_{11} && \rightarrow && u_{11} = 2 \\ 1 &= l_{21}u_{11} && \rightarrow && l_{21} = 0.5 \\ 2 &= l_{31}u_{11} && \rightarrow && l_{31} = 1. \end{aligned}$$

Column 2 of A gives

$$\begin{aligned} 1 &= u_{12} && \rightarrow && u_{12} = 1 \\ 2 &= l_{21}u_{12} + u_{22} && \rightarrow && u_{22} = 1.5 \\ 4 &= l_{31}u_{12} + l_{32}u_{22} && \rightarrow && l_{32} = 2. \end{aligned}$$

Column 3 of A gives

$$\begin{aligned} 4 &= u_{13} && \rightarrow && u_{13} = 4 \\ 2 &= l_{21}u_{13} + u_{23} && \rightarrow && u_{23} = 0 \\ 6 &= l_{31}u_{13} + l_{32}u_{23} + u_{33} && \rightarrow && u_{33} = 2. \end{aligned}$$

Solve the lower triangular system $L\vec{z} = \vec{b}$:

$$\begin{pmatrix} 1 & 0 & 0 \\ 0.5 & 1 & 0 \\ 1 & 2 & 1 \end{pmatrix} \begin{pmatrix} z_1 \\ z_2 \\ z_3 \end{pmatrix} = \begin{pmatrix} 12 \\ 9 \\ 22 \end{pmatrix} \rightarrow \begin{pmatrix} z_1 \\ z_2 \\ z_3 \end{pmatrix} = \begin{pmatrix} 12 \\ 3 \\ 4 \end{pmatrix}$$

Solve the upper triangular system $U\vec{x} = \vec{z}$:

$$\begin{pmatrix} 2 & 1 & 4 \\ 0 & 1.5 & 0 \\ 0 & 0 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 12 \\ 3 \\ 4 \end{pmatrix} \rightarrow \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \\ 2 \end{pmatrix}.$$

Exercise 4.5. Rewrite the matrix A as the product of lower and upper triangular matrices where

$$A = \begin{pmatrix} 4 & 2 & 0 \\ 2 & 3 & 1 \\ 0 & 1 & 2.5 \end{pmatrix}.$$

Remark. The first example gives

$$\begin{pmatrix} 2 & 1 & 4 \\ 1 & 2 & 2 \\ 2 & 4 & 6 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0.5 & 1 & 0 \\ 1 & 2 & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 & 4 \\ 0 & 1.5 & 0 \\ 0 & 0 & 2 \end{pmatrix}$$

Note that

- the matrix U is the same as the fully eliminated upper triangular form produced by Gaussian elimination;
- L contains the multipliers that were used at each stage to eliminate the rows.

4.8 Effects of finite precision arithmetic

Example 4.5. Consider the following linear system of equations

$$\begin{pmatrix} 0 & 2 & 1 \\ 2 & 1 & 0 \\ 1 & 2 & 0 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 7 \\ 4 \\ 5 \end{pmatrix}$$

Problem. We cannot eliminate the entries below the diagonal in the first column using multiples of row 1 to rows 2 and 3, respectively.

Solution. Swap the order of the equations!

- Swap rows 1 and 2:

$$\begin{pmatrix} 2 & 1 & 0 \\ 0 & 2 & 1 \\ 1 & 2 & 0 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 4 \\ 7 \\ 5 \end{pmatrix}$$

- Now apply Gaussian elimination

- row 2 $\to$ row 2 $-0\times$ row 1 and row 3 $\to$ row 3 $-\frac{1}{2}$ row 1:

$$\begin{pmatrix} 2 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 1.5 & 0 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 4 \\ 7 \\ 3 \end{pmatrix}$$

- row 3 $\to$ row 3 $-\frac{1.5}{2}$ row 2

$$\begin{pmatrix} 2 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & -0.75 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 4 \\ 7 \\ -2.25 \end{pmatrix}.$$

Example 4.6. Consider another system of equations

$$\begin{pmatrix} 2 & 1 & 1 \\ 4 & 2 & 1 \\ 2 & 2 & 0 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \\ 2 \end{pmatrix}$$

- Apply Gaussian elimination as usual:
  - row 2 $\to$ row 2 $-2\times$ row 1

$$\begin{pmatrix} 2 & 1 & 1 \\ 0 & 0 & -1 \\ 2 & 2 & 0 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 3 \\ -1 \\ 2 \end{pmatrix}.$$

  - row 3 $\to$ row 3 $-1\times$ row 1

$$\begin{pmatrix} 2 & 1 & 1 \\ 0 & 0 & -1 \\ 0 & 1 & -1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 3 \\ -1 \\ -1 \end{pmatrix}.$$

- *Problem.* We cannot eliminate the second column below the diagonal by adding a multiple of row 2 to row 3.
- Again, this problem may be overcome simply by swapping the order of the equations – this time, swapping rows 2 and 3:

$$\begin{pmatrix} 2 & 1 & 1 \\ 0 & 1 & -1 \\ 0 & 0 & -1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 3 \\ -1 \\ -1 \end{pmatrix}$$

- We can now continue the Gaussian elimination process as usual.

In general. Gaussian elimination requires row swaps to avoid breaking down when there is a zero in the "pivot" position. Avoiding zero-division might be a familiar aspect of Gaussian elimination, but there is an additional reason to apply pivoting when working with floating point numbers:

Example 4.7. Consider using Gaussian elimination to solve the linear system of equations given by

$$\begin{pmatrix} \varepsilon & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 2+\varepsilon \\ 3 \end{pmatrix}$$

where $\varepsilon \neq 1$.

- The true, unique solution is $(x_1, x_2)^T = (1, 2)^T$.
- If $\varepsilon \neq 0$, Gaussian elimination gives

$$\begin{pmatrix} \varepsilon & 1 \\ 0 & 1 - \frac{1}{\varepsilon} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 2+\varepsilon \\ 3 - \frac{2+\varepsilon}{\varepsilon} \end{pmatrix}$$

- Problems occur not only when $\varepsilon = 0$ but also when it is very small, i.e. when $\frac{1}{\varepsilon}$ is very large, which will introduce very significant rounding errors into the computation.

Use Gaussian elimination to solve the linear system of equations given by

$$\begin{pmatrix} 1 & 1 \\ \varepsilon & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 3 \\ 2+\varepsilon \end{pmatrix}$$

where $\varepsilon \neq 1$.

- The true solution is still $(x_1, x_2)^T = (1, 2)^T$.
- Gaussian elimination now gives

$$\begin{pmatrix} 1 & 1 \\ 0 & 1 - \varepsilon \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 3 \\ 2 - 2\varepsilon \end{pmatrix}$$

- The problems due to small values of ε have disappeared.

This is a genuine problem we see in the code versions too!

```python
print("without row swapping:")
for eps in [1.0e-2, 1.0e-4, 1.0e-6, 1.0e-8, 1.0e-10, 1.0e-12, 1.0e-14]:
    A = np.array([[eps, 1.0], [1.0, 1.0]])
    b = np.array([[2.0 + eps], [3.0]])

    gaussian_elimination(A, b)
    x = backward_substitution(A, b)
    print(f"{eps=:.1e}", end=", ")
    print_array(x.T, "x.T", end=", ")

    A = np.array([[eps, 1.0], [1.0, 1.0]])
    b = np.array([[2.0 + eps], [3.0]])
    print("Solution?", np.allclose(A @ x, b))
print()

print("with row swapping:")
for eps in [1.0e-2, 1.0e-4, 1.0e-6, 1.0e-8, 1.0e-10, 1.0e-12, 1.0e-14]:
    A = np.array([[1.0, 1.0], [eps, 1.0]])
    b = np.array([[3.0], [2.0 + eps]])

    gaussian_elimination(A, b)
    x = backward_substitution(A, b)
    print(f"{eps=:.1e}", end=", ")
    print_array(x.T, "x.T", end=", ")

    A = np.array([[1.0, 1.0], [eps, 1.0]])
    b = np.array([[3.0], [2.0 + eps]])
    print("Solution?", np.allclose(A @ x, b))
print()
```

```
without row swapping:
eps=1.0e-02, x.T = [  1.0,  2.0 ], Solution? True
eps=1.0e-04, x.T = [  1.0,  2.0 ], Solution? True
eps=1.0e-06, x.T = [  1.0,  2.0 ], Solution? True
eps=1.0e-08, x.T = [  1.0,  2.0 ], Solution? True
eps=1.0e-10, x.T = [  1.0,  2.0 ], Solution? True
eps=1.0e-12, x.T = [  1.00009,  2.00000 ], Solution? False
eps=1.0e-14, x.T = [  1.0214,  2.0000 ], Solution? False

with row swapping:
eps=1.0e-02, x.T = [  1.0,  2.0 ], Solution? True
eps=1.0e-04, x.T = [  1.0,  2.0 ], Solution? True
```

```
eps=1.0e-06, x.T = [  1.0,  2.0 ], Solution? True
eps=1.0e-08, x.T = [  1.0,  2.0 ], Solution? True
eps=1.0e-10, x.T = [  1.0,  2.0 ], Solution? True
eps=1.0e-12, x.T = [  1.0,  2.0 ], Solution? True
eps=1.0e-14, x.T = [  1.0,  2.0 ], Solution? True
```

Remark 4.3.

- Writing the equations in a different order has removed the previous problem.
- The diagonal entries are now always *relatively* larger.
- The interchange of the order of equations is a simple example of **row pivoting**. This strategy avoids excessive rounding errors in the computations.

4.8.1 Gaussian elimination with pivoting

Key idea:

- Before eliminating entries in column j:
  - find the entry in column j, below the diagonal, of maximum magnitude;
  - if that entry is larger in magnitude than the diagonal entry then swap its row with row j.
- Then eliminate column j as before.

This algorithm will always work when the matrix A is non-singular. Conversely, if all of the possible pivot values are zero this implies that the matrix is singular and a unique solution does not exist. At each elimination step the row multipliers used are guaranteed to be at most one in magnitude so any errors in the representation of the system cannot be amplified by the elimination process. As always, solving $A\vec{x} = \vec{b}$ requires that the entries in $\vec{b}$ are also swapped in the appropriate way.

Pivoting can be applied in an equivalent way to LU factorisation. The sequence of pivots is independent of the vector $\vec{b}$ and can be recorded and reused. The constraint imposed on the row multipliers means that for LU factorisation every entry in L satisfies $|l_{ij}| \leq 1$.

Example 4.8. Consider the linear system of equations given by

$$\begin{pmatrix} 10 & -7 & 0 \\ -3 & 2.1-\varepsilon & 6 \\ 5 & -1 & 5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 7 \\ 9.9+\varepsilon \\ 11 \end{pmatrix}$$

where $0 \leq \varepsilon \ll 1$, and solve it using

1. Gaussian elimination without pivoting
2. Gaussian elimination with pivoting.

The exact solution is $\vec{x} = (0, -1, 2)^T$ for any ε in the given range.

1. Solve the system using Gaussian elimination with no pivoting.

Eliminating the first column gives

$$\begin{pmatrix} 10 & -7 & 0 \\ 0 & -\varepsilon & 6 \\ 0 & 2.5 & 5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 7 \\ 12+\varepsilon \\ 7.5 \end{pmatrix}$$

and then the second column gives

$$\begin{pmatrix} 10 & -7 & 0 \\ 0 & -\varepsilon & 6 \\ 0 & 0 & 5+15/\varepsilon \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 7 \\ 12+\varepsilon \\ 7.5+2.5(12+\varepsilon)/\varepsilon \end{pmatrix}$$

which leads to

$$x_3 = \frac{3+\frac{12+\varepsilon}{\varepsilon}}{2+\frac{6}{\varepsilon}} \qquad x_2 = \frac{(12+\varepsilon)-6x_3}{-\varepsilon} \qquad x_1 = \frac{7+7x_2}{10}.$$

There are many divisions by ε, so we will have problems if ε is (very) small.

2. Solve the system using Gaussian elimination with pivoting.

The first stage is identical (because $a_{11} = 10$ is largest).

$$\begin{pmatrix} 10 & -7 & 0 \\ 0 & -\varepsilon & 6 \\ 0 & 2.5 & 5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 7 \\ 12+\varepsilon \\ 7.5 \end{pmatrix}$$

but now $|a_{22}| = \varepsilon$ and $|a_{32}| = 2.5$ so we swap rows 2 and 3 to give

$$\begin{pmatrix} 10 & -7 & 0 \\ 0 & 2.5 & 5 \\ 0 & -\varepsilon & 6 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 7 \\ 7.5 \\ 12+\varepsilon \end{pmatrix}$$

Now we may eliminate column 2:

$$\begin{pmatrix} 10 & -7 & 0 \\ 0 & 2.5 & 5 \\ 0 & 0 & 6+2\varepsilon \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 7 \\ 7.5 \\ 12+4\varepsilon \end{pmatrix}$$

which leads to the exact answer:

$$x_3 = \frac{12+4\varepsilon}{6+2\varepsilon} = 2 \qquad x_2 = \frac{7.5-5x_3}{2.5} = -1 \qquad x_1 = \frac{7+7x_2}{10} = 0.$$

4.8.2 Python code for Gaussian elimination with pivoting

```
def gaussian_elimination_with_pivoting(A, b, verbose=False):
    """
    Perform Gaussian elimination with partial pivoting to reduce a linear
    system to upper triangular form.

    This function performs **forward elimination** to transform the
    coefficient matrix `A` into an upper triangular matrix, while applying
    the same operations to the right-hand side vector `b`. Before
    eliminating each column, rows are swapped so that the entry of largest
    absolute value is used as the pivot (**partial pivoting**), improving
    numerical stability compared to :func:`gaussian_elimination`.

    Parameters
    ----------
    A : numpy.ndarray
        A 2D NumPy array of shape ``(n, n)`` representing the coefficient
        matrix of the system.
    b : numpy.ndarray
        A 2D NumPy array of shape ``(n, 1)`` representing the right-hand side
        vector.
    verbose : bool, optional
        If ``True``, prints detailed information about each elimination step,
        including the row swaps and row operations performed and the
        intermediate forms of `A` and `b`. Default is ``False``.

    Returns
    -------
    None
        This function modifies `A` and `b` **in place** and does not return
```

```
        anything.
    """
    # find shape of system
    n = system_size(A, b)

    # perform forwards elimination
    for i in range(n - 1):
        # eliminate column i
        if verbose:
            print(f"eliminating column {i}")

        # find largest entry in column i
        largest = abs(A[i, i])
        j_max = i
        for j in range(i + 1, n):
            if abs(A[j, i]) > largest:
                largest, j_max = abs(A[j, i]), j

        # swap rows j_max and i
        row_swap(A, b, i, j_max)
        if verbose:
            print(f"swapped system ({i} <-> {j_max})")
            print_array(A)
            print_array(b)
            print()

        for j in range(i + 1, n):
            # row j
            factor = A[j, i] / A[i, i]
            if verbose:
                print(f"row {j} |-> row {j} - {factor} * row {i}")
            row_add(A, b, j, -factor, i)

        if verbose:
            print("new system")
            print_array(A)
            print_array(b)
            print()
```

Gaussian elimination without pivoting following by back substitution:

```python
eps = 1.0e-14
A = np.array([[10.0, -7.0, 0.0], [-3.0, 2.1 - eps, 6.0], [5.0, -1.0, 5.0]])
b = np.array([[7.0], [9.9 + eps], [11.0]])

print("starting system:")
print_array(A)
print_array(b)
print()

print("performing Gaussian elimination without pivoting")
gaussian_elimination(A, b, verbose=True)
print()

print("upper triangular system:")
print_array(A)
print_array(b)
print()

print("performing backward substitution")
x = backward_substitution(A, b)
print()

print("solution using backward substitution:")
print_array(x)
print()

A = np.array([[10.0, -7.0, 0.0], [-3.0, 2.1 - eps, 6.0], [5.0, -1.0, 5.0]])
b = np.array([[7.0], [9.9 + eps], [11.0]])
print("Does x solve the original system?", np.allclose(A @ x, b))
```

```
starting system:
A = [ 10.0, -7.0,  0.0 ]
    [ -3.0,  2.1,  6.0 ]
    [  5.0, -1.0,  5.0 ]
b = [  7.0 ]
    [  9.9 ]
    [ 11.0 ]

performing Gaussian elimination without pivoting
eliminating column 0
  row 1 |-> row 1 - -0.3 * row 0
  row 2 |-> row 2 - 0.5 * row 0
```

```
new system
A = [ 10.0, -7.0,  0.0 ]
    [  0.0, -0.0,  6.0 ]
    [  0.0,  2.5,  5.0 ]
b = [  7.0 ]
    [ 12.0 ]
    [  7.5 ]

eliminating column 1
  row 2 |-> row 2 - -244760849313613.9 * row 1

new system
A = [ 10.0, -7.0,  0.0 ]
    [  0.0, -0.0,  6.0 ]
    [  0.0,  0.0, 1468565095881688.5 ]
b = [  7.0 ]
    [ 12.0 ]
    [ 2937130191763377.0 ]


upper triangular system:
A = [ 10.0, -7.0,  0.0 ]
    [  0.0, -0.0,  6.0 ]
    [  0.0,  0.0, 1468565095881688.5 ]
b = [  7.0 ]
    [ 12.0 ]
    [ 2937130191763377.0 ]

performing backward substitution

solution using backward substitution:
x = [ -0.030435 ]
    [ -1.043478 ]
    [  2.000000 ]

Does x solve the original system? False
```

Gaussian elimination with pivoting following by back substitution:

```
1 eps = 1.0e-14
2 A = np.array([[10.0, -7.0, 0.0], [-3.0, 2.1 - eps, 6.0], [5.0, -1.0, 5.0]])
```

```
b = np.array([[7.0], [9.9 + eps], [11.0]])

print("starting system:")
print_array(A)
print_array(b)
print()

print("performing Gaussian elimination with pivoting")
gaussian_elimination_with_pivoting(A, b, verbose=True)
print()

print("upper triangular system:")
print_array(A)
print_array(b)
print()

print("performing backward substitution")
x = backward_substitution(A, b)
print()

print("solution using backward substitution:")
print_array(x)
print()

A = np.array([[10.0, -7.0, 0.0], [-3.0, 2.1 - eps, 6.0], [5.0, -1.0, 5.0]])
b = np.array([[7.0], [9.9 + eps], [11.0]])
print("Does x solve the original system?", np.allclose(A @ x, b))
```

```
starting system:
A = [ 10.0, -7.0,  0.0 ]
    [ -3.0,  2.1,  6.0 ]
    [  5.0, -1.0,  5.0 ]
b = [  7.0 ]
    [  9.9 ]
    [ 11.0 ]

performing Gaussian elimination with pivoting
eliminating column 0
swapped system (0 <-> 0)
A = [ 10.0, -7.0,  0.0 ]
    [ -3.0,  2.1,  6.0 ]
    [  5.0, -1.0,  5.0 ]
```

```
b = [  7.0 ]
    [  9.9 ]
    [ 11.0 ]

row 1 |-> row 1 - -0.3 * row 0
row 2 |-> row 2 - 0.5 * row 0
new system
A = [ 10.0, -7.0,  0.0 ]
    [  0.0, -0.0,  6.0 ]
    [  0.0,  2.5,  5.0 ]
b = [  7.0 ]
    [ 12.0 ]
    [  7.5 ]

eliminating column 1
swapped system (1 <-> 2)
A = [ 10.0, -7.0,  0.0 ]
    [  0.0,  2.5,  5.0 ]
    [  0.0, -0.0,  6.0 ]
b = [  7.0 ]
    [  7.5 ]
    [ 12.0 ]

row 2 |-> row 2 - -4.085620730620576e-15 * row 1
new system
A = [ 10.0, -7.0,  0.0 ]
    [  0.0,  2.5,  5.0 ]
    [  0.0,  0.0,  6.0 ]
b = [  7.0 ]
    [  7.5 ]
    [ 12.0 ]


upper triangular system:
A = [ 10.0, -7.0,  0.0 ]
    [  0.0,  2.5,  5.0 ]
    [  0.0,  0.0,  6.0 ]
b = [  7.0 ]
    [  7.5 ]
    [ 12.0 ]

performing backward substitution
```

```
solution using backward substitution:
x = [  0.0 ]
    [ -1.0 ]
    [  2.0 ]

Does x solve the original system? True
```

4.9 Summary

We have introduced some methods for solving systems of linear equations based manipulating the system into an upper triangular form and then applying backwards substitution. The methods require a fixed amount of work but that amount will tend to be large for large systems. We have seen that care must be taken when working with floating point numbers to ensure accurate results. Next we will see an alternative approach which often works much more quickly in a range of different practical scenarios.

4.10 Further reading

Some basic reading:

- Joseph F. Grcar. How ordinary elimination became Gaussian elimination. Historia Mathematica. Volume 38, Issue 2, May 2011. (More history)

Some implementations:

- Numpy `numpy.linalg.solve`
- Scipy `scipy.linalg.lu`
- LAPACK Gaussian elimination (uses LU factorisation): `dgesv()`
- LAPACK LU Factorisation: `dgetrf()`.

5 Iterative solutions of linear equations

Module learning objective

- Derive and apply Jacobi and Gauss-Seidel iterative methods from a matrix splitting of a linear system;
- Implement iterative solvers for dense and sparse matrices and estimate their computational complexity;
- Evaluate convergence behaviour and justify the choice of stopping criteria for practical computations.

5.1 Iterative methods

In the previous section we looked at what are known as *direct* methods for solving systems of linear equations. In exact arithmetic, they produce the solution after a finite number of operations, but this fixed amount of work may be **very** large.

For a general $n \times n$ system of linear equations $A\vec{x} = \vec{b}$, the computational cost of direct methods is typically $O(n^3)$. The amount of storage required for these approaches is $O(n^2)$ which is dominated by the cost of storing the matrix A. As n becomes larger the storage and computation work required limit the practicality of direct approaches.

As an alternative, we will propose some **iterative methods**. Iterative methods produce a sequence $\{\vec{x}^{(k)}\}$ of approximations to the solution of the linear system of equations $A\vec{x} = \vec{b}$. The iteration is defined recursively and is typically of the form:

$$\vec{x}^{(k+1)} = \vec{F}(\vec{x}^{(k)}),$$

where $\vec{x}^{(k)}$ is now a vector of values and $\vec{F}$ is some vector function (which comes as part of the definition of the method). We will need to choose a starting value $\vec{x}^{(0)}$ but there is often a reasonable approximation which can be used. Once all this is defined, we still need to decide when we need to stop!

Remark 5.1. We use a value in brackets in the superscript to denote the iteration number to avoid confusion between the iteration number and the component of the vector:

$$\vec{x}^{(k)} = (\vec{x}_1^{(k)}, \vec{x}_2^{(k)}, \dots, \vec{x}_n^{(k)}).$$

The methods we consider will have the form:

$$\vec{F}(\vec{x}^{(k)}) = \vec{x}^{(k)} + P(\vec{b} - A\vec{x}^{(k)}). \tag{5.1}$$

for some matrix P. We call $\vec{r}^{(k)} = \vec{b} - A\vec{x}^{(k)}$ the **residual**.

Example 5.1 (Some terrible examples)**.** These are examples of potential iterative methods which would not work very well!

1. Consider

$$\vec{F}(\vec{x}^{(k)}) = \vec{x}^{(k)} + A^{-1}(\vec{b} - A\vec{x}^{(k)}).$$

   Each iteration is very expensive to compute since it requires applying A^{-1} but it converges in just one step since

$$\begin{aligned} A\vec{x}^{(k+1)} &= A\vec{x}^{(k)} + AA^{-1}(\vec{b} - A\vec{x}^{(k)}) \\ &= A\vec{x}^{(k)} + \vec{b} - A\vec{x}^{(k)} \\ &= \vec{b}. \end{aligned}$$

   This example uses $P = A^{-1}$.

2. Consider

$$\vec{F}(\vec{x}^{(k)}) = \vec{x}^{(k)}.$$

   Each iteration is very cheap to compute but very inaccurate - it never converges!

   This example uses $P = O$ the zero matrix.

The key point here is that we want a method that is both computationally cheap and converges quickly to the solution. To achieve this we need to choose P so that:

- P is easy to compute, or the matrix-vector product $\vec{r}^{(k)} \mapsto P\vec{r}^{(k)}$ is easy to compute,
- P approximates A^{-1} well enough that the algorithm converges in few iterations.

5.2 Jacobi iteration

One straightforward choice for P in (5.1) is given by the Jacobi method where we take $P = D^{-1}$ where D is the diagonal of A:

$$D_{ii} = A_{ii} \quad \text{and} \quad D_{ij} = 0 \text{ for } i \neq j.$$

The **Jacobi iteration** is given by

$$\vec{x}^{(k+1)} = \vec{x}^{(k)} + D^{-1}(\vec{b} - A\vec{x}^{(k)})$$

D is a *diagonal matrix*, so D^{-1} is trivial to form (as long as the diagonal entries are all non-zero):

$$(D^{-1})_{ii} = \frac{1}{D_{ii}} \quad \text{and} \quad (D^{-1})_{ij} = 0 \text{ for } i \neq j.$$

Remark.

- The cost of one iteration is $O(n^2)$ for a full matrix, and this is dominated by the matrix-vector product $A\vec{x}^{(k)}$.
- This cost can be reduced to $O(n)$ if the matrix A is sparse - this is when iterative methods are especially attractive (Example 3.7).
- The amount of work also depends on the number of iterations required to get a "satisfactory" solution.
  - The number of iterations depends on the matrix;
    - Fewer iterations are needed for a less accurate solution;
  - A good initial estimate $\vec{x}^{(0)}$ reduces the required number of iterations.
- Unfortunately, the iteration might not converge!

The Jacobi iteration updates all components using the values from iteration k. It updates all elements of $\vec{x}^{(k)}$ *simultaneously* to get $\vec{x}^{(k+1)}$. Writing the method out component by component gives

$$
\begin{aligned}
x_1^{(k+1)} &= x_1^{(k)} + \frac{1}{A_{11}}\left(b_1 - \sum_{j=1}^{n} A_{1j}x_j^{(k)}\right) \\
x_2^{(k+1)} &= x_2^{(k)} + \frac{1}{A_{22}}\left(b_2 - \sum_{j=1}^{n} A_{2j}x_j^{(k)}\right) \\
&\vdots \qquad\quad \vdots \\
x_n^{(k+1)} &= x_n^{(k)} + \frac{1}{A_{nn}}\left(b_n - \sum_{j=1}^{n} A_{nj}x_j^{(k)}\right).
\end{aligned}
$$

Note that once the first step has been taken, $x_1^{(k+1)}$ is already known, but the Jacobi iteration does not make use of this information!

Example 5.2. Take two iterations of Jacobi iteration to approximate the solution of the following system using the initial guess $\vec{x}^{(0)} = (1,1)^T$:

$$
\begin{pmatrix} 2 & 1 \\ -1 & 4 \end{pmatrix}\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 3.5 \\ 0.5 \end{pmatrix}
$$

Starting from $\vec{x}^{(0)} = (1,1)^T$, the first iteration is

$$
\begin{aligned}
x_1^{(1)} &= x_1^{(0)} + \frac{1}{A_{11}}\left(b_1 - A_{11}x_1^{(0)} - A_{12}x_2^{(0)}\right) \\
&= 1 + \frac{1}{2}(3.5 - 2\times 1 - 1\times 1) = 1.25 \\
x_2^{(1)} &= x_2^{(0)} + \frac{1}{A_{22}}\left(b_2 - A_{21}x_1^{(0)} - A_{22}x_2^{(0)}\right) \\
&= 1 + \frac{1}{4}(0.5 - (-1)\times 1 - 4\times 1) = 0.375.
\end{aligned}
$$

So we have $\vec{x}^{(1)} = (1.25, 0.375)^T$. Then the second iteration is

$$
\begin{aligned}
x_1^{(2)} &= x_1^{(1)} + \frac{1}{A_{11}}\left(b_1 - A_{11}x_1^{(1)} - A_{12}x_2^{(1)}\right) \\
&= 1.25 + \frac{1}{2}(3.5 - 2\times 1.25 - 1\times 0.375) = 1.5625 \\
x_2^{(2)} &= x_2^{(1)} + \frac{1}{A_{22}}\left(b_2 - A_{21}x_1^{(1)} - A_{22}x_2^{(1)}\right) \\
&= 0.375 + \frac{1}{4}(0.5 - (-1)\times 1.25 - 4\times 0.375) = 0.4375.
\end{aligned}
$$

So we have $\vec{x}^{(2)} = (1.5625, 0.4375)$.

Note that the only difference between the formulae for Iteration 1 and 2 is the iteration number, the superscript in brackets. The exact solution is given by $\vec{x} = (1.5, 0.5)^T$.

We note that we can also slightly simplify the way the Jacobi iteration is written. We can expand A into $A = L + D + U$, where L and U are the parts of the matrix from below and above the diagonal respectively:

$$L_{ij} = \begin{cases} A_{ij} & \text{if } i > j \\ 0 & \text{if } i \leq j, \end{cases} \qquad U_{ij} = \begin{cases} A_{ij} & \text{if } i < j \\ 0 & \text{if } i \geq j. \end{cases}$$

Then we can calculate that:

$$\begin{aligned} \vec{x}^{(k+1)} &= \vec{x}^{(k)} + D^{-1}(\vec{b} - A\vec{x}^{(k)}) \\ &= \vec{x}^{(k)} + D^{-1}(\vec{b} - (L + D + U)\vec{x}^{(k)}) \\ &= \vec{x}^{(k)} - D^{-1}D\vec{x}^{(k)} + D^{-1}(\vec{b} - (L + U)\vec{x}^{(k)}) \\ &= \vec{x}^{(k)} - \vec{x}^{(k)} + D^{-1}(\vec{b} - (L + U)\vec{x}^{(k)}) \\ &= D^{-1}(\vec{b} - (L + U)\vec{x}^{(k)}). \end{aligned}$$

In this formulation, we do not explicitly form the residual as part of the computations. In practical situations, this may be a simpler formulation we can use if we have knowledge of the coefficients of A, rather than just the matrix-vector product.

5.3 Gauss-Seidel iteration

As an alternative to Jacobi iteration, the iteration might use $x_i^{(k+1)}$ as soon as it is calculated (rather than using the previous iteration), giving

$$
\begin{aligned}
x_1^{(k+1)} &= x_1^{(k)} + \frac{1}{A_{11}}\left(b_1 - \sum_{j=1}^{n} A_{1j}x_j^{(k)}\right) \\
x_2^{(k+1)} &= x_2^{(k)} + \frac{1}{A_{22}}\left(b_2 - A_{21}x_1^{(k+1)} - \sum_{j=2}^{n} A_{2j}x_j^{(k)}\right) \\
x_3^{(k+1)} &= x_3^{(k)} + \frac{1}{A_{33}}\left(b_3 - \sum_{j=1}^{2} A_{3j}x_j^{(k+1)} - \sum_{j=3}^{n} A_{3j}x_j^{(k)}\right) \\
&\vdots \qquad \vdots \\
x_i^{(k+1)} &= x_i^{(k)} + \frac{1}{A_{ii}}\left(b_i - \sum_{j=1}^{i-1} A_{ij}x_j^{(k+1)} - \sum_{j=i}^{n} A_{ij}x_j^{(k)}\right) \\
&\vdots \qquad \vdots \\
x_n^{(k+1)} &= x_n^{(k)} + \frac{1}{A_{nn}}\left(b_n - \sum_{j=1}^{n-1} A_{nj}x_j^{(k+1)} - A_{nn}x_n^{(k)}\right).
\end{aligned}
$$

Consider the system $A\vec{x} = b$ with the matrix A split as $A = L + D + U$, where D is the diagonal of A, L contains the elements below the diagonal, and U contains the elements above the diagonal. The componentwise iteration above can be written in matrix form as

$$\vec{x}^{(k+1)} = \vec{x}^{(k)} + (D + L)^{-1}(\vec{b} - A\vec{x}^{(k)}).$$

That is, we use $P = (D + L)^{-1}$ in (5.1).

In practice, we do not form the inverse of $D + L$ explicitly. Since $D + L$ is lower triangular, systems involving $D + L$ can be solved efficiently using forward substitution.

Example 5.3. Take two iterations of Gauss-Seidel iteration to approximate the solution of the following system using the initial guess $\vec{x}^{(0)} = (1, 1)^T$:

$$\begin{pmatrix} 2 & 1 \\ -1 & 4 \end{pmatrix}\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 3.5 \\ 0.5 \end{pmatrix}$$

Starting from $\vec{x}^{(0)} = (1, 1)^T$ we have

Iteration 1:

$$
\begin{aligned}
x_1^{(1)} &= x_1^{(0)} + \frac{1}{A_{11}}(b_1 - A_{11}x_1^{(0)} - A_{12}x_2^{(0)}) \\
&= 1 + \frac{1}{2}(3.5 - 2 \times 1 - 1 \times 1) = 1.25 \\
x_2^{(1)} &= x_2^{(0)} + \frac{1}{A_{22}}(b_2 - A_{21}x_1^{(1)} - A_{22}x_2^{(0)}) \\
&= 1 + \frac{1}{4}(0.5 - (-1) \times 1.25 - 4 \times 1) = 0.4375.
\end{aligned}
$$

Iteration 2:

$$
\begin{aligned}
x_1^{(2)} &= x_1^{(1)} + \frac{1}{A_{11}}(b_1 - A_{11}x_1^{(1)} - A_{12}x_2^{(1)}) \\
&= 1.25 + \frac{1}{2}(3.5 - 2 \times 1.25 - 1 \times 0.4375) = 1.53125 \\
x_2^{(2)} &= x_2^{(1)} + \frac{1}{A_{22}}(b_2 - A_{21}x_1^{(2)} - A_{22}x_2^{(1)}) \\
&= 0.4375 + \frac{1}{4}(0.5 - (-1) \times 1.53125 - 4 \times 0.4375) = 0.5078125.
\end{aligned}
$$

Again, note the changes in the iteration number on the right-hand side of these equations, especially the differences against the Jacobi method.

- What happens if the initial estimate is altered to $\vec{x}^{(0)} = (2, 1)^T$.

Remark.

- On the examples Example 5.2 and Example 5.3, both methods converge, but relatively slowly. They might not converge at all!
- We will discuss convergence and when to stop later.
- The Gauss-Seidel iteration generally out-performs the Jacobi iteration.
- Performance can depend on the order in which the equations are written.
- Both iterative algorithms can be made faster and more efficient for sparse systems of equations (far more than direct methods).

Exercise 5.1. Take one iteration of (a) Jacobi iteration and (b) Gauss-Seidel iteration to approximate the solution of the following system using the initial guess $\vec{x}^{(0)} = (0, 1, 1)^T$:

$$
\begin{pmatrix} 1 & -2 & 4 \\ -1 & -2 & -1 \\ -1 & -5 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} -3 \\ 4 \\ 5 \end{pmatrix}.
$$

Exercise 5.2.

1. Show that the componentwise operations written above are equivalent to $P = (D+L)^{-1}$ in (5.1).
2. Then show that the componentwise operations are the same as forward substitution applied to (5.1) written as

$$\vec{x}^{(k+1)} = \vec{x}^{(k)} + \vec{z}^{(k)}, \qquad (D+L)\vec{z}^{(k)} = (\vec{b} - A\vec{x}^{(k)}).$$

5.4 Python version of iterative methods

```
def jacobi_iteration(A, b, x0, max_iter, verbose=False):
    """
    Solve a linear system Ax = b using the Jacobi iterative method.

    The Jacobi method is an iterative algorithm for solving the linear system:

    .. math::
        Ax = b

    starting from an initial guess ``x0``. At each iteration, the solution is
    updated according to:

    .. math::
        x_i^{(k+1)} = x_i^{(k)} + \\frac{1}{A_{ii}} \\left( b_i -
        \\sum_{j=0}^{n-1} A_{ij} x_j^{(k)} \\right)

    Parameters
    ----------
    A : numpy.ndarray
        A 2D NumPy array of shape ``(n, n)`` representing the coefficient
        matrix.
    b : numpy.ndarray
        A 1D or 2D NumPy array of shape ``(n,)`` or ``(n, 1)`` representing
        the right-hand side vector.
    x0 : numpy.ndarray
        Initial guess for the solution, same shape as ``b``.
    max_iter : int
        Maximum number of iterations to perform.
    verbose : bool, optional
        If ``True``, prints the value of the solution vector at each
```

```
        iteration. Default is ``False``.

    Returns
    -------
    x : numpy.ndarray
        Approximated solution vector after ``max_iter`` iterations.
    """
    n = system_size(A, b)

    x = x0.copy()
    xnew = np.empty_like(x)

    if verbose:
        print("starting value: ", end="")
        print_array(x.T, "x.T")

    for iter in range(max_iter):
        for i in range(n):
            Axi = 0.0
            for j in range(n):
                Axi += A[i, j] * x[j]
            xnew[i] = x[i] + 1.0 / (A[i, i]) * (b[i] - Axi)
        x = xnew.copy()

        if verbose:
            print(f"after {iter=}: ", end="")
            print_array(x.T, "x.T")

    return x


def gauss_seidel_iteration(A, b, x0, max_iter, verbose=False):
    """
    Solve a linear system Ax = b using the Gauss-Seidel iterative method.

    The Gauss-Seidel method is an iterative algorithm for solving the linear
    system:

    .. math::
        Ax = b

    starting from an initial guess ``x0``. At each iteration, the solution is
```

```
    updated sequentially using the most recently computed values:

    .. math::
        x_i^{(k+1)} = x_i^{(k)} + \\frac{1}{A_{ii}} \\left( b_i -
        \\sum_{j=0}^{i-1} A_{ij} x_j^{(k+1)} -
        \\sum_{j=i}^{n-1} A_{ij} x_j^{(k)} \\right)

    Parameters
    ----------
    A : numpy.ndarray
        A 2D NumPy array of shape ``(n, n)`` representing the coefficient
        matrix.
    b : numpy.ndarray
        A 1D or 2D NumPy array of shape ``(n,)`` or ``(n, 1)`` representing
        the right-hand side vector.
    x0 : numpy.ndarray
        Initial guess for the solution, same shape as ``b``.
    max_iter : int
        Maximum number of iterations to perform.
    verbose : bool, optional
        If ``True``, prints the value of the solution vector at each
        iteration. Default is ``False``.

    Returns
    -------
    x : numpy.ndarray
        Approximated solution vector after ``max_iter`` iterations.

    """
    n = system_size(A, b)

    x = x0.copy()
    xnew = np.empty_like(x)

    if verbose:
        print("starting value: ", end="")
        print_array(x.T, "x.T")

    for iter in range(max_iter):
        for i in range(n):
            Axi = 0.0
            for j in range(i):
```

```
                Axi += A[i, j] * xnew[j]
            for j in range(i, n):
                Axi += A[i, j] * x[j]
            xnew[i] = x[i] + 1.0 / (A[i, i]) * (b[i] - Axi)
        x = xnew.copy()

        if verbose:
            print(f"after {iter=}: ", end="")
            print_array(x.T, "x.T")

    return x
```

```
A = np.array([[2.0, 1.0], [-1.0, 4.0]])
b = np.array([[3.5], [0.5]])
x0 = np.array([[1.0], [1.0]])

print("jacobi iteration")
x = jacobi_iteration(A, b, x0, 5, verbose=True)
print()

print("gauss seidel iteration")
x = gauss_seidel_iteration(A, b, x0, 5, verbose=True)
print()
```

```
jacobi iteration
starting value: x.T = [  1.0,  1.0 ]
after iter=0: x.T = [  1.250,  0.375 ]
after iter=1: x.T = [  1.5625,  0.4375 ]
after iter=2: x.T = [  1.53125,  0.51562 ]
after iter=3: x.T = [  1.49219,  0.50781 ]
after iter=4: x.T = [  1.49609,  0.49805 ]

gauss seidel iteration
starting value: x.T = [  1.0,  1.0 ]
after iter=0: x.T = [  1.2500,  0.4375 ]
after iter=1: x.T = [  1.53125,  0.50781 ]
after iter=2: x.T = [  1.49609,  0.49902 ]
after iter=3: x.T = [  1.50049,  0.50012 ]
after iter=4: x.T = [  1.49994,  0.49998 ]
```

5.5 Sparse Matrices

We met sparse matrices as an example of a special matrix format when we first thought about systems of linear equations (Example 3.7). Sparse matrices are very common in applications and have a structure which is very useful when used with iterative methods. There are two main ways in which sparse matrices can be exploited in order to obtain benefits within iterative methods.

- The storage can be reduced from $O(n^2)$.
- The cost per iteration can be reduced from $O(n^2)$.

Recall that a sparse matrix is defined to be such that it has at most αn non-zero entries (where α is independent of n) – in other words, that the number of nonzero entries grows proportionally to n, rather than n^2. Typically this happens when we know there are at most α non-zero entries in any row.

The simplest way in which a sparse matrix is stored is using three arrays:

- an array of floating point numbers (`A_real` say) that stores the non-zero entries;
- an array of integers (`I_row` say) that stores the row number of the corresponding entry in the real array;
- an array of integers (`I_col` say) that stores the column numbers of the corresponding entry in the real array.

This requires just $3\alpha n$ units of storage - i.e. $O(n)$. This is called the COO (coordinate) format.

Given the above storage pattern, the following algorithm will execute a sparse matrix-vector multiplication ($\vec{z} = A\vec{y}$) in $O(n)$ operations:

```
z = np.zeros((n, 1))
for k in range(nonzero):
    z[I_row[k]] = z[I_row[k]] + A_real[k] * y[I_col[k]]
```

- Here `nonzero` is the number of non-zero entries in the matrix.
- Note that the cost of this operation is $O(n)$ as required.

5.5.1 Python experiments

First let's adapt our implementations to use this sparse matrix format:

```
def jacobi_iteration_sparse(
    A_real, I_row, I_col, b, x0, max_iter, verbose=False
):
    """
    Solve a sparse linear system Ax = b using the Jacobi iterative method.

    The system is represented in **sparse COO (coordinate) format** with:
    - `A_real`: non-zero values
    - `I_row`: row indices of non-zero entries
    - `I_col`: column indices of non-zero entries

    The Jacobi method updates the solution iteratively:

    .. math::
        x_i^{(k+1)} = x_i^{(k)} + \\frac{1}{A_{ii}} \\left( b_i -
        \\sum_{j=0}^{n-1} A_{ij} x_j^{(k)} \\right)

    Parameters
    ----------
    A_real : array-like
        Array of non-zero entries of the sparse matrix A.
    I_row : array-like
        Row indices corresponding to each entry in `A_real`.
    I_col : array-like
        Column indices corresponding to each entry in `A_real`.
    b : array-like
        Right-hand side vector of length `n`.
    x0 : numpy.ndarray
        Initial guess for the solution vector, same length as `b`.
    max_iter : int
        Maximum number of iterations to perform.
    verbose : bool, optional
        If ``True``, prints the solution vector after each iteration.
        Default is ``False``.

    Returns
    -------
    x : numpy.ndarray
        Approximated solution vector after `max_iter` iterations.
    """
    n, nonzero = system_size_sparse(A_real, I_row, I_col, b)

```

```
    x = x0.copy()
    xnew = np.empty_like(x)

    if verbose:
        print("starting value: ", end="")
        print_array(x.T, "x.T")

    # determine diagonal
    # D[i] should be A_{ii}
    D = np.zeros_like(x)
    for k in range(nonzero):
        if I_row[k] == I_col[k]:
            D[I_row[k]] = A_real[k]

    for iter in range(max_iter):
        # precompute Ax
        Ax = np.zeros_like(x)
        for k in range(nonzero):
            Ax[I_row[k]] = Ax[I_row[k]] + A_real[k] * x[I_col[k]]

        for i in range(n):
            xnew[i] = x[i] + 1.0 / D[i] * (b[i] - Ax[i])
        x = xnew.copy()

        if verbose:
            print(f"after {iter=}: ", end="")
            print_array(x.T, "x.T")

    return x


def gauss_seidel_iteration_sparse(
    A_real, I_row, I_col, b, x0, max_iter, verbose=False
):
    """
    Solve a sparse linear system Ax = b using the Gauss-Seidel iterative
    method.

    The system is represented in **sparse COO (coordinate) format** with:
    - `A_real`: non-zero values
    - `I_row`: row indices of non-zero entries
    - `I_col`: column indices of non-zero entries
```

```
    The Gauss-Seidel method updates the solution sequentially using the most
    recently computed values:

    .. math::
        x_i^{(k+1)} = x_i^{(k)} + \\frac{1}{A_{ii}} \\left( b_i -
        \\sum_{j=0}^{i-1} A_{ij} x_j^{(k+1)} -
        \\sum_{j=i}^{n-1} A_{ij} x_j^{(k)} \\right)

    Parameters
    ----------
    A_real : array-like
        Array of non-zero entries of the sparse matrix A.
    I_row : array-like
        Row indices corresponding to each entry in `A_real`.
    I_col : array-like
        Column indices corresponding to each entry in `A_real`.
    b : array-like
        Right-hand side vector of length `n`.
    x0 : numpy.ndarray
        Initial guess for the solution vector, same length as `b`.
    max_iter : int
        Maximum number of iterations to perform.
    verbose : bool, optional
        If ``True``, prints the solution vector after each iteration.
        Default is ``False``.

    Returns
    -------
    x : numpy.ndarray
        Approximated solution vector after `max_iter` iterations.
    """
    n, nonzero = system_size_sparse(A_real, I_row, I_col, b)

    x = x0.copy()

    if verbose:
        print("starting value: ", end="")
        print_array(x.T, "x.T")

    # determine diagonal and row_mapping
    # D[i] should be A_{ii}
```

```
    D = np.zeros_like(x)
    row_entries = [[] for _ in range(n)]
    for k in range(nonzero):
        # diagonals
        if I_row[k] == I_col[k]:
            D[I_row[k]] = A_real[k]
        # store which row
        row_entries[I_row[k]].append(k)

    for iter in range(max_iter):
        for i in range(n):
            # precompute ith component of Ax using latest x values
            Ax_i = 0.0
            for k in row_entries[i]:
                Ax_i = Ax_i + A_real[k] * x[I_col[k]]

            x[i] = x[i] + 1.0 / D[i] * (b[i] - Ax_i)

        if verbose:
            print(f"after {iter=}: ", end="")
            print_array(x.T, "x.T")

    return x
```

Then we can test the two different implementations of the methods:

```
# random matrix
n = 4
nonzero = 10
A_real, I_row, I_col, b = random_sparse_system(n, nonzero)
print("sparse matrix:")
print("A_real =", A_real)
print("I_row = ", I_row)
print("I_col = ", I_col)
print()

# convert to dense for comparison
A_dense = to_dense(A_real, I_row, I_col)
print("dense matrix:")
print_array(A_dense)
print()

```

```python
# starting guess
x0 = np.zeros((n, 1))

print("jacobi with sparse matrix")
x_sparse = jacobi_iteration_sparse(
    A_real, I_row, I_col, b, x0, max_iter=5, verbose=True
)
print()

print("jacobi with dense matrix")
x_dense = jacobi_iteration(A_dense, b, x0, max_iter=5, verbose=True)
print()

print("gauss seidel with sparse matrix")
x_sparse = gauss_seidel_iteration_sparse(
    A_real, I_row, I_col, b, x0, max_iter=5, verbose=True
)
print()

print("gauss seidel with dense matrix")
x_dense = gauss_seidel_iteration(A_dense, b, x0, max_iter=5, verbose=True)
print()
```

```
sparse matrix:
A_real = [-12.    14.   -12.    22.25   9.   -10.5  -12.     9.25 -10.5  -10.5
  14.    13.25   9.   -10.5    9.   -12.  ]
I_row =  [0 0 0 0 1 1 1 1 2 2 2 2 3 3 3 3]
I_col =  [3 2 1 0 3 2 0 1 3 1 0 2 3 2 1 0]

dense matrix:
A_dense = [ 22.25, -12.00, 14.00, -12.00 ]
          [ -12.00,  9.25, -10.50,  9.00 ]
          [ 14.00, -10.50, 13.25, -10.50 ]
          [ -12.00,  9.00, -10.50,  9.00 ]

jacobi with sparse matrix
starting value: x.T = [  0.0,  0.0,  0.0,  0.0 ]
after iter=0: x.T = [  0.55056, -0.45946,  0.47170, -0.50000 ]
after iter=1: x.T = [ -0.26370,  1.27671, -0.87035,  1.24386 ]
after iter=2: x.T = [  2.45761, -2.99976,  2.74775, -3.14372 ]
after iter=3: x.T = [ -4.4917,  8.9066, -6.9934,  8.9823 ]
after iter=4: x.T = [ 14.5989, -22.9645, 19.3938, -23.5546 ]
```

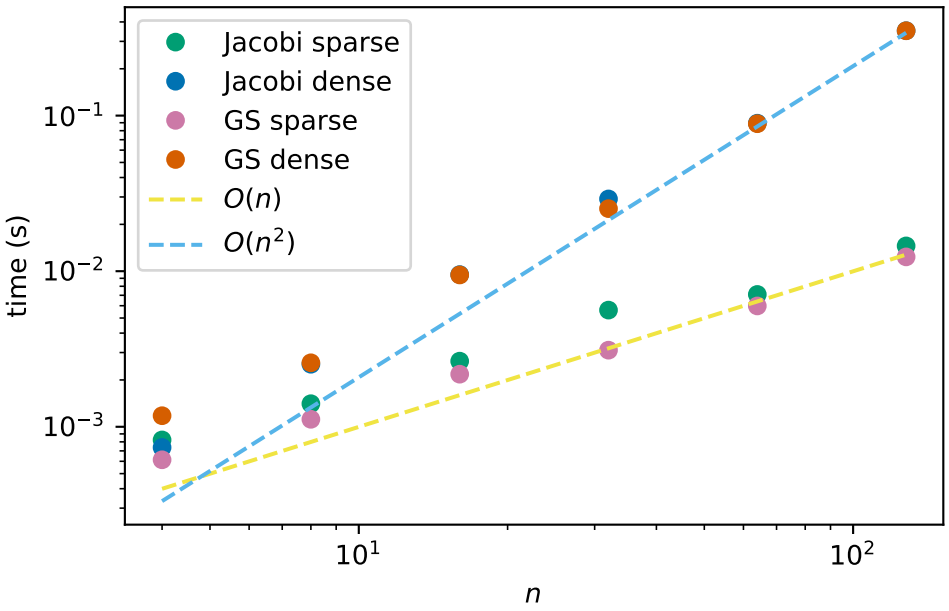
```
jacobi with dense matrix
starting value: x.T = [  0.0,  0.0,  0.0,  0.0 ]
after iter=0: x.T = [  0.55056, -0.45946,  0.47170, -0.50000 ]
after iter=1: x.T = [ -0.26370,  1.27671, -0.87035,  1.24386 ]
after iter=2: x.T = [  2.45761, -2.99976,  2.74775, -3.14372 ]
after iter=3: x.T = [ -4.4917,  8.9066, -6.9934,  8.9823 ]
after iter=4: x.T = [ 14.5989, -22.9645, 19.3938, -23.5546 ]

gauss seidel with sparse matrix
starting value: x.T = [  0.0,  0.0,  0.0,  0.0 ]
after iter=0: x.T = [  0.550562,  0.254783,  0.091876,  0.086488 ]
after iter=1: x.T = [  0.676808,  0.438703,  0.172769,  0.165272 ]
after iter=2: x.T = [  0.76759,  0.57165,  0.24463,  0.23721 ]
after iter=3: x.T = [  0.832876,  0.667916,  0.308950,  0.303026 ]
after iter=4: x.T = [  0.87982,  0.73779,  0.36688,  0.36332 ]

gauss seidel with dense matrix
starting value: x.T = [  0.0,  0.0,  0.0,  0.0 ]
after iter=0: x.T = [  0.550562,  0.254783,  0.091876,  0.086488 ]
after iter=1: x.T = [  0.676808,  0.438703,  0.172769,  0.165272 ]
after iter=2: x.T = [  0.76759,  0.57165,  0.24463,  0.23721 ]
after iter=3: x.T = [  0.832876,  0.667916,  0.308950,  0.303026 ]
after iter=4: x.T = [  0.87982,  0.73779,  0.36688,  0.36332 ]
```

We see that we get the same results!

Now let us see how long it takes to get a solution. The following plot shows the run times of using the two different implementations of the Jacobi method, each for 10 iterations. We see that, as expected, the run time of the dense formulation is $O(n^2)$ and the run time of the sparse formulation is $O(n)$.



We say "as expected" because we have already counted the number of operations per iteration and these implementations compute for a fixed number of iterations. In the next section, we look at alternative stopping criteria.

5.6 Convergence of an iterative method

We have discussed the construction of **iterations** which aim to find the solution of the equations $A\vec{x} = \vec{b}$ through a sequence of better and better approximations $\vec{x}^{(k)}$.

In general the iteration takes the form

$$\vec{x}^{(k+1)} = \vec{F}(\vec{x}^{(k)})$$

here $\vec{x}^{(k)}$ is a vector of values and $\vec{F}$ is some vector-valued function which we have defined.

How can we decide if this iteration has converged? We need $\vec{x} - \vec{x}^{(k)}$ to be small, but we do not have access to the exact solution $\vec{x}$ so we have to do something else!

How do we decide that a vector/array is small? The most common measure is to use the *Euclidean norm* of an array (which you met last year!). The Euclidean norm, or norm for short, is defined to be the square root of the sum of squares of the entries of the array:

$$\|\vec{y}\| = \sqrt{\sum_{i=1}^{n} y_i^2}.$$

where $\vec{y}$ is a vector with n entries.

Example 5.4. Consider the following sequence $\vec{x}^{(k)}$:

$$\begin{pmatrix} 1 \\ -1 \end{pmatrix}, \begin{pmatrix} 1.5 \\ 0.5 \end{pmatrix}, \begin{pmatrix} 1.75 \\ 0.25 \end{pmatrix}, \begin{pmatrix} 1.875 \\ 0.125 \end{pmatrix}, \begin{pmatrix} 1.9375 \\ -0.0625 \end{pmatrix}, \begin{pmatrix} 1.96875 \\ -0.03125 \end{pmatrix}, \ldots$$

- What is $\|\vec{x}^{(1)} - \vec{x}^{(0)}\|$?
- What is $\|\vec{x}^{(5)} - \vec{x}^{(4)}\|$?

Let $\vec{x} = \begin{pmatrix} 2 \\ 0 \end{pmatrix}$.

- What is $\|\vec{x} - \vec{x}^{(3)}\|$?
- What is $\|\vec{x} - \vec{x}^{(4)}\|$?
- What is $\|\vec{x} - \vec{x}^{(5)}\|$?

Rather than decide in advance how many iterations (of the Jacobi or Gauss-Seidel methods) to use, we can use the following stopping criteria:

- A maximum number of iterations.
- The *change* in values is small enough:

$$\|\vec{x}^{(k+1)} - \vec{x}^{(k)}\| < tol,$$

- The *norm of the residual* is small enough:

$$\|\vec{r}^{(k)}\| = \|\vec{b} - A\vec{x}^{(k)}\| < tol$$

In the second and third cases, we call *tol* the **convergence tolerance** and the choice of *tol* controls the accuracy of the solution.

Remark 5.2. We note that the residual $\vec{r}^{(k)}$ is different to the error $\vec{x} - \vec{x}^{(k)}$. The error cannot be computed in general since it requires knowledge of the exact solution $\vec{x}$. The residual can be computed and is therefore often used to monitor convergence.

The two are related as using the fact that the residual at the exact solution is $\vec{0}$:

$$\vec{r}^{(k)} = \vec{b} - A\vec{x}^{(k)} = \vec{b} - A\vec{x}^{(k)} - (\vec{b} - A\vec{x}) = A(\vec{x} - \vec{x}^{(k)}).$$

Therefore, the exact relation between the error and residual depends on the properties of A.

Exercise 5.3 (Discussion)**.** What is a good convergence tolerance?

In general there are two possible reasons that an iteration may fail to converge.

- It may **diverge** - this means that $\|\vec{x}^{(k)}\| \to \infty$ as k (the number of iterations) increases, e.g.:

$$\begin{pmatrix}1\\1\end{pmatrix}, \begin{pmatrix}4\\2\end{pmatrix}, \begin{pmatrix}16\\4\end{pmatrix}, \begin{pmatrix}64\\8\end{pmatrix}, \begin{pmatrix}256\\16\end{pmatrix}, \begin{pmatrix}1024\\32\end{pmatrix}, \ldots$$

- It may *neither* converge nor diverge, e.g.:

$$\begin{pmatrix}1\\1\end{pmatrix}, \begin{pmatrix}2\\0\end{pmatrix}, \begin{pmatrix}3\\1\end{pmatrix}, \begin{pmatrix}1\\0\end{pmatrix}, \begin{pmatrix}2\\1\end{pmatrix}, \begin{pmatrix}3\\0\end{pmatrix}, \ldots$$

In addition to testing for convergence it is also necessary to include tests for failure to converge.

- Divergence may be detected by monitoring $\|\vec{x}^{(k)}\|$.
- Impose a maximum number of iterations to ensure that the loop is not repeated forever!

5.7 Summary

Many complex computational problems simply cannot be solved with today's computers using direct methods. Iterative methods are used instead since they can massively reduce the computational cost and storage required to get a "good enough" solution.

These basic iterative methods are simple to describe and program but generally slow to converge to an accurate answer - typically $O(n)$ iterations are required! Their usefulness for general dense matrix systems is very limited. However, for sparse systems, they can offer dramatic reductions in both storage requirements and computational cost.

More advanced iterative methods do exist but are beyond the scope of this module - see Final year projects, MSc projects, PhD, and beyond!

5.8 Further reading

- Jason Brownlee: A gentle introduction to sparse matrices for machine learning, Machine learning mastery

Some software implementations:

- `scipy.sparse` custom routines specialised to sparse matrices
- SuiteSparse, a suite of sparse matrix algorithms geared toward the direct solution of sparse linear systems

- `scipy.sparse` iterative solvers: Solving linear problems
- PETSc: Linear system solvers - a high-performance linear algebra toolkit

6 Complex numbers

Module learning objective

- Use complex arithmetic and polar representations to analyse complex numbers algebraically and geometrically.
- Determine complex roots of polynomial equations using factorisation and the quadratic formula.
- Manipulate complex vectors and matrices including computing Hermitian products, conjugate transposes, and Hermitian matrices.

6.1 Basic definitions

A complex number is an element of a number system which extends our familiar real number system. Our motivation will be to find eigenvalues and eigenvectors which is the next topic in this module. To find eigenvalues, we need to solve polynomial equations and it will turn out to be useful to *always* be able to get a solution to any polynomial equation.

Example 6.1. Consider the polynomial equation

$$x^2 + 1 = 0. \tag{6.1}$$

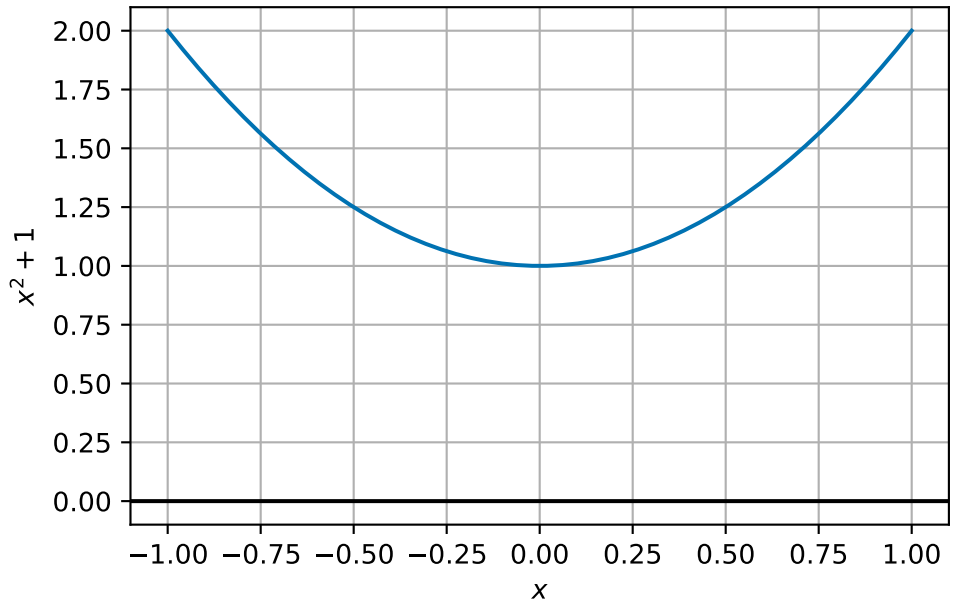


Figure 6.1: Plot of $y = x^2 + 1$.

We can see that this equation has no solution over the real numbers: There are no real values x such that $x^2 + 1 = 0$.

The key idea is to introduce a new symbol, which we will call i, or the **imaginary unit** which satisfies:

$$i^2 = -1 \qquad \sqrt{-1} = i.$$

By taking multiples of this imaginary unit, we can create many more new numbers, like $3i, \sqrt{5}i$ or $-12i$. These are examples of **imaginary numbers**.

A **complex number** is formed by combining a real part and an imaginary part while keeping the two components distinct. For example, $2 + 3i$, $\frac{1}{2} + \sqrt{5}i$ or $12 - 12i$.

Definition 6.1. Any number that can be written as $z = a + bi$, with a, b real numbers and i the imaginary unit, is called a *complex number*. In this format, we call a the **real part** of z and b the **imaginary part** of z.

We notice that all real numbers x must also be complex numbers since $x = x + 0i$.

Remark 6.1. Among the first recorded uses of complex numbers in European mathematics was by an Italian mathematician, Gerolamo Cardano, in around 1545. He later described complex numbers as being "as subtle as they are useless" and "mental torment".

The term imaginary was coined by René Descartes in 1637:

… sometimes only imaginary, that is one can imagine as many as I said in each equation, but sometimes there exist no quantity that matches that which we imagine.

Exercise 6.1. What are the real and imaginary parts of these numbers?

1. $3 + 6i$
2. $-3.5 + 2i$
3. 5
4. $7i$

6.2 Calculations with complex numbers

We can perform the basic operations, such as addition, subtraction, multiplication and division, on complex numbers.

Addition and subtraction are relatively straightforward: we treat the real and imaginary parts separately.

Example 6.2. We can compute that

$$(2 + 3i) + (12 - 12i) = (2 + 12) + (3 - 12)i = 14 - 9i$$
$$(2 + 3i) - (12 - 12i) = (2 - 12) + (3 - (-12))i = -10 + 15i.$$

Exercise 6.2. Compute $(3 + 6i) + (-3.5 + 2i)$ and $(3 + 6i) - (-3.5 + 2i)$.

For multiplying and dividing, we first say that applying these operations between complex and real numbers again follows the usual rules. Let x be a real number and $z = (a + bi)$ be a complex number, then

$$x \times (a + bi) = (a + bi) \times x = (x \times a) + (x \times b)i$$
$$\frac{a + bi}{x} = \frac{a}{x} + \frac{b}{x}i.$$

For multiplication between complex numbers, things are harder. We expand brackets and apply the rule that $i^2 = -1$:

Example 6.3.

$$\begin{aligned}
&(2+3i)\times(12-12i)\\
&=2\times 12+3i\times 12+2\times -12i+3i\times -12i && \text{(expand brackets)}\\
&=2\times 12+(12\times 3)i+(2\times -12)i+(3\times -12)\times i^2 && \text{(rearrange)}\\
&=24+36i-24i-36\times i^2 && \text{(compute products)}\\
&=24+36i-24i+36 && \text{(use } i^2=-1\text{)}\\
&=60+12i && \text{(collect terms).}
\end{aligned}$$

This leads us to a general formula:

$$(a+bi)\times(c+di)=(ac-bd)+(ad+bc)i.$$

Exercise 6.3. Compute $(3+6i)\times(-3.5+2i)$.

Division is harder - you may want to skip this on first reading since it is not so important for what follows in these notes. When we divide complex numbers, we try to rewrite the fraction to have a real denominator by "rationalising the denominator".

Example 6.4. Suppose we want to find $(2+3i)/(12-12i)$. Our idea is to find a number so that we can write

$$\frac{2+3i}{12-12i}=\frac{2+3i}{12-12i}\times\frac{z}{z}=\frac{(2+3i)z}{(12-12i)z}=\frac{\text{something}}{\text{something real}}.$$

The answer is to use $z=12+12i$ - that is the denominator with the sign of the imaginary part flipped (we will give this a name later on).

We can compute that

$$\begin{aligned}
&(12-12i)\times(12+12i)\\
&=(12\times 12)+(12\times 12i)+(-12i\times 12)+(-12i\times 12i) && \text{(expand bracket)}\\
&=12\times 12+(12\times 12)i+(-12\times 12)i+(-12\times 12)i^2 && \text{(rearrange)}\\
&=144+144i-144i-144i^2 && \text{(compute products)}\\
&=144+144i-144i+144 && \text{(use } i^2=-1\text{)}\\
&=288+0i && \text{(collect terms).}
\end{aligned}$$

So we have that $(12-12i)\times(12+12i)=288$ is a real number.

We continue by computing that

$$\begin{aligned}
&(2+3i)\times(12+12i)\\
&=(2\times 12)+(2\times 12i)+(3i\times 12)+(3i\times 12i)\\
&=2\times 12+(2\times 12)i+(3\times 12)i+(3\times 12)i^2\\
&=24+24i+36i+36i^2\\
&=24+24i+36i-36\\
&=-12+60i.
\end{aligned}$$

Thus we infer that

$$\begin{aligned}
\frac{2+3i}{12-12i}&=\frac{2+3i}{12-12i}\times\frac{12+12i}{12+12i}\\
&=\frac{(2+3i)(12+12i)}{(12-12i)(12+12i)}\\
&=\frac{-12+60i}{288}\\
&=\frac{-12}{288}+\frac{60}{288}i\\
&=-\frac{1}{24}+\frac{5}{24}i.
\end{aligned}$$

To check we have not done anything silly, we should also check that $(12+12i)/(12+12i)=1$. This is left as an exercise.

Exercise 6.4. Find $(3+6i)/(-3.5+2i)$ and $(12+12i)/(12+12i)$.

Remark 6.2. One thing to be careful of when considering products is that the identity $i^2=-1$ appears to break one rule of arithmetic of square roots:

$$i^2=(\sqrt{-1})^2=\sqrt{-1}\sqrt{-1}\neq\sqrt{(-1)\times(-1)}=\sqrt{1}=1.$$

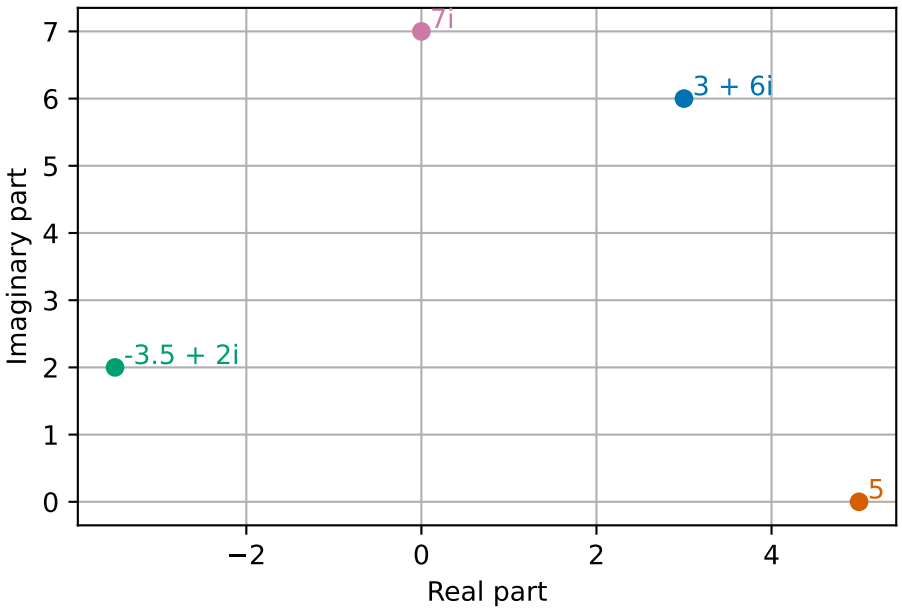
In fact, we have that $\sqrt{x}\sqrt{y}=\sqrt{xy}$ only if $x,y>0$.

6.3 A geometric picture

The idea of adding complex numbers by considering real and imaginary parts separately is reminiscent of adding two-dimensional vectors. For this reason, it is often helpful to think of complex numbers as points in *the complex plane*.

The complex plane is the two-dimensional space formed by considering the real and imaginary parts of a complex number as two different coordinate axes.

Example 6.5.



Adding complex numbers looks just like adding two dimensional vectors! We can also use this geometric picture to help with some further operations.

The *complex conjugate* of a complex number $z = a + bi$ is given by $\bar{z} = a - bi$. The complex conjugate is that number we used before when working out how to divide complex numbers.

Example 6.6. The complex conjugate of $2 + 3i$ is $2 - 3i$. The complex conjugate of $12 - 12i$ is $12 + 12i$.

Exercise 6.5. Find the complex conjugates of $3 + 6i$ and $-3.5 + 2i$.

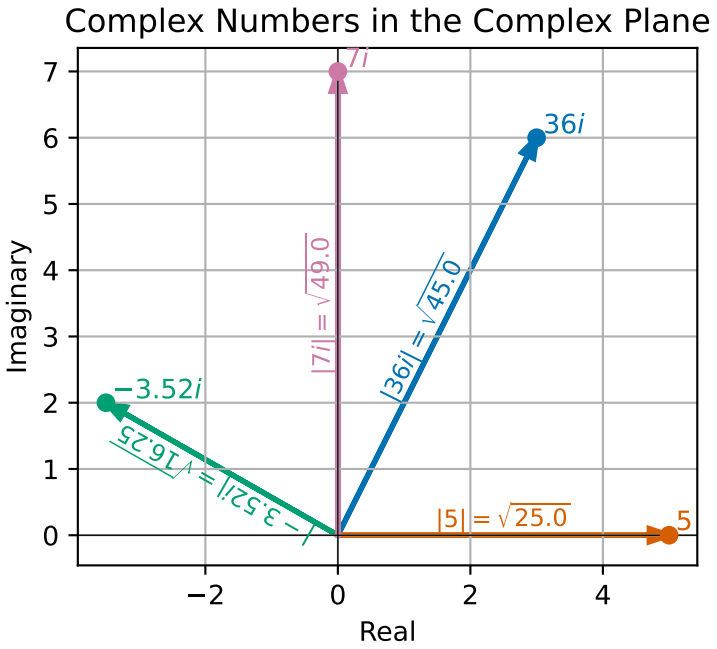
We have already seen that the complex conjugate of a complex number is helpful when performing division of complex numbers. The reason is that computing the product of a number and its conjugate always gives a real, positive number:

$$\begin{aligned}
&(a + bi) \times (a - bi) \\
&= (a \times a) + (a \times -bi) + (bi \times a) + (bi \times -bi) \\
&= (a \times a) + (a \times -b)i + (b \times a)i + (b \times -b)i^2 \\
&= (a \times a) + (a \times -b + b \times a)i - (b \times -b) \\
&= a^2 + b^2 + 0i.
\end{aligned}$$

In fact, we use this same calculation to define the **modulus** (sometimes called the **absolute value**) of a complex number $z = a + bi$

$$|z| = |a + bi| = \sqrt{a^2 + b^2} = \sqrt{z\bar{z}}. \tag{6.2}$$

Example 6.7.



Exercise 6.6. Find the value of

$$|3 + 6i| \quad \text{and} \quad |-3.5 + 2i|. \tag{6.3}$$

Consider two complex numbers $z = a + bi$ and $y = c + di$. Then, we have already seen that

$$zy = (a + bi) \times (c + di) = (ac - bd) + (ad + bc)i.$$

We can compute the square of the modulus of the product zy as

$$\begin{aligned} |zy|^2 &= (ac - bd)^2 + (ad + bc)^2 \\ &= a^2c^2 - 2abcd + b^2d^2 + a^2d^2 + 2abcd + b^2c^2 \\ &= a^2c^2 + b^2d^2 + a^2d^2 + b^2c^2 \\ &= (a^2 + b^2)(c^2 + d^2). \end{aligned}$$

Since moduli are non-negative, taking square roots gives $|zy| = |z||y|$.

In particular, if y has modulus 1, then $|zy| = |z|$. This means that zy and z are the same distance from the origin but 'point' in different directions. We can write the real and imaginary parts of y (which has $|y| = 1$) as $y = c + di = \cos(\theta) + i\sin(\theta)$, where θ is the angle between the positive real axis and the line between 0 and y. Then

$$\begin{aligned} zy &= (ac - bd) + (ad + bc)i \\ &= (a\cos(\theta) - b\sin(\theta)) + (a\sin(\theta) + b\cos(\theta))i. \end{aligned}$$

Recalling the example of a rotation matrix from Example 1.3, we see that multiplying by y is the same as rotating the complex point (z) by an angle of θ radians in the anticlockwise direction.

This motivates the use of *polar coordinates* for the complex plane. Polar coordinates are a different form of coordinates that replace the usual x and y-directions (up and across) by two values which represent the distance to the origin (that we call radius) and angle to the positive x-axis (that we call the angle). When talking about a complex number z represented in the complex plane, we know that the modulus $|z|$ represents the radius. The idea of θ above represents the angle of a complex number that we now call the **argument**.

Definition 6.2. Let z be a complex number. The **polar form** of z is $R(\cos\theta + i\sin\theta)$. We call R the modulus of z and θ is an argument of z.

The representation of the angle is unique up to adding integer multiples of 2π, since rotating a point by 2π about the origin leaves it unchanged. We always have $R = |z|$.

Example 6.8. Let $z = 12 - 12i$. Then

$$|z| = |12 - 12i| = \sqrt{12^2 + 12^2} = \sqrt{2 \times 144} = 12\sqrt{2}.$$

We have $\arg z = -\pi/4$ since

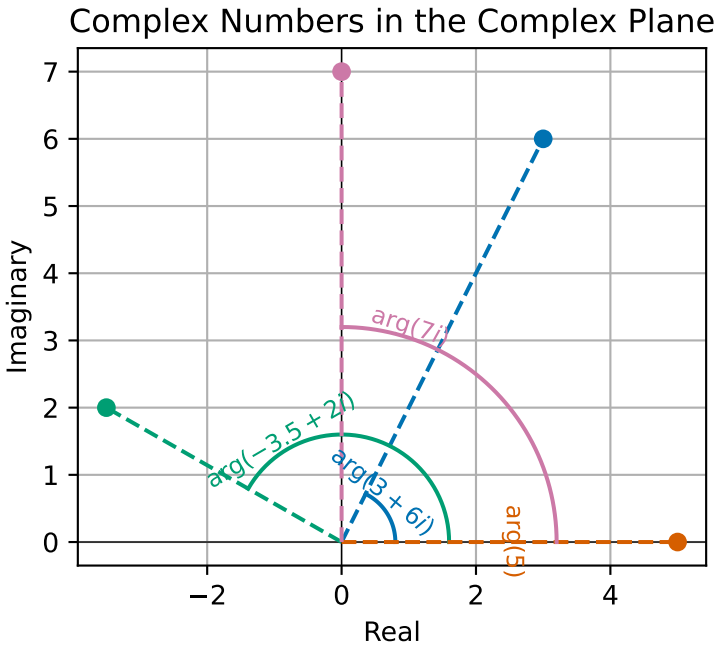
$$\cos(-\pi/4) = \frac{1}{\sqrt{2}}, \quad \text{and} \quad \sin(-\pi/4) = \frac{-1}{\sqrt{2}},$$

so

$$12\sqrt{2}(\cos(-\pi/4) + i\sin(-\pi/4)) = 12\sqrt{2}\left(\frac{1}{\sqrt{2}} + i\frac{-1}{\sqrt{2}}\right) = 12 - 12i.$$

Exercise 6.7. Compute the modulus and argument of 2, $3i$ and $4 + 4i$.

Example 6.9.



The polar representation of complex numbers then gives us a nice way to understand multiplication of complex numbers. If $y \neq 0$, then we can check that $\left|\frac{y}{|y|}\right| = 1$ and $\arg y = \arg \frac{y}{|y|}$. Then writing $zy = z\frac{y}{|y|}|y|$, we can use our calculations above to infer that multiplying by y corresponds to an anticlockwise rotation by $\arg y$ then scaling by $|y|$.

Exercise 6.8. Check that for any non-zero complex number y, that $\left|\frac{y}{|y|}\right| = 1$ and $\arg y = \arg \frac{y}{|y|}$.

6.4 Euler's formula

The polar form of complex numbers can be further developed thanks to two helpful identities. We only state the identities here. There are some examples on the worksheet to help your understanding here.

Euler's formula relates the polar form of a complex number to complex exponentials: Let x be a real number, then

$$\exp(ix) = \cos x + i \sin x.$$

The result can be shown by showing that the function $f(x) = \exp(-ix)(\cos x + i \sin x)$ is a constant function and that the constant value is 1. For the purposes of this part of the

module, you might want to think of $\exp(ix)$ simply as a shorthand for $\cos x + i \sin x$ when used in calculations.

For example, as a consequence of Euler's formula, we can compute powers of complex numbers very simply. Let z be given in polar form by $z = R\exp(i\theta)$ (that is R is the magnitude of z and θ is the argument of z). Then z^p is, by the rule of exponentiation, $z^p = R^p \exp(ip\theta)$.

Euler's formula is the key important step for showing Euler's identity:

$$\exp(i\pi) = -1.$$

Many mathematicians regard this as a particularly surprising and beautiful identity.

6.5 Solving polynomial equations

As we have mentioned above, we will be using complex numbers when solving polynomial equations to work out eigenvalues and eigenvectors of matrices later in the section. The reason complex numbers are useful here is this very important Theorem:

Theorem 6.1 (The Fundamental Theorem of Algebra)**.** *For any complex numbers $a_0, \ldots, a_n$ with $a_n \neq 0$, there is at least one complex number z which satisfies:*

$$a_n z^n + \cdots + a_1 z + a_0 = 0$$

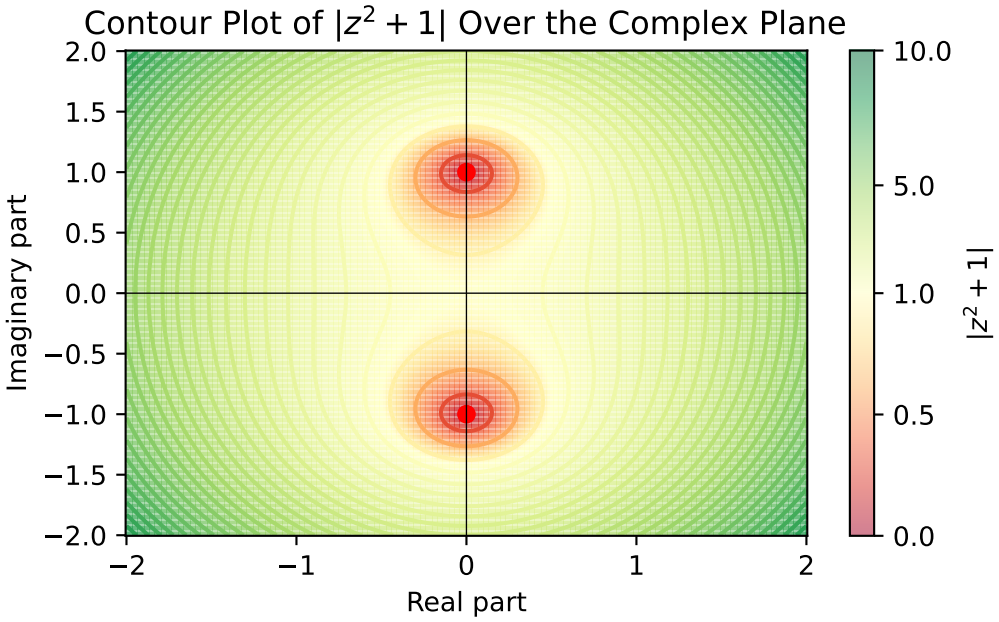
It is really important to note here that this is not true if we want z to be a real number. Let's revisit Example 6.1.

Example 6.10. Consider the polynomial equation

$$x^2 + 1 = 0. \tag{6.4}$$

We saw before that this equation has no solution over the real numbers, but the Fundamental Theorem of Algebra tells us there must be at least one solution which is a complex number. In fact it has two solutions - i and $-i$:

$$\begin{aligned} &i^2 + 1 = 0 \\ &(-i)^2 + 1 = (-1)^2 i^2 + 1 = i^2 + 1 = 0. \end{aligned}$$



Notice that along the real line (Imaginary part $= 0$), the value of the function is always above 1.

In general, to find complex roots of other quadratic equations, we can apply the quadratic formula:

Example 6.11. To find the values of z which satisfy $z^2 - 2z + 2 = 0$, we see:

$$z = \frac{+2 \pm \sqrt{(-2)^2 - 4 \times 1 \times 2}}{2} = \frac{2 \pm \sqrt{-4}}{2} = \frac{2 \pm 2\sqrt{-1}}{2} = 1 \pm i.$$

Exercise 6.9. Find the values of z which satisfy $z^2 - 4z + 20 = 0$.

We will see later that, although this approach can be used to compute the eigenvalues of a 2×2 matrix, it quickly becomes impractical for larger matrices.

6.6 Complex vectors and matrices

Last year you were introduced to vectors of real numbers. In the remaining part of Linear Algebra, we will need to work with complex matrices and vectors. The definitions are mostly what you would expect but with some slight subtleties.

Definition 6.3. A complex matrix is a matrix whose entries are complex numbers. A complex vector is a vector whose entries are complex numbers.

One thing we like to do with vectors is to take scalar products and to find their length. When working with complex vectors, we replace the scalar product with a *Hermitian product*. We write $\langle \vec{a}, \vec{b} \rangle$ for the Hermitian product of two complex n-vectors $\vec{a}$ and $\vec{b}$ and the value is given by

$$\langle \vec{a}, \vec{b} \rangle = \sum_{i=1}^{n} a_i \bar{b}_i. \tag{6.5}$$

We see that the Hermitian product is the same as taking the scalar product of $\vec{a}$ and the complex conjugate of $\vec{b}$. If the entries in $\vec{b}$ have no imaginary part (i.e. they are all real numbers), then the Hermitian product of $\vec{a}$ and $\vec{b}$ is equal to the scalar product of $\vec{a}$ and $\vec{b}$.

Example 6.12. Let $\vec{a} = (1+2i, 2+i, 3-4i)^T$ and $\vec{b} = (6+2i, 3+4i, 1+i)^T$ then:

$$\begin{aligned} \langle \vec{a}, \vec{b} \rangle &= (1+2i) \times (6-2i) + (2+i) \times (3-4i) + (3-4i) \times (1-i) \\ &= (10+10i) + (10-5i) + (-1-7i) = 19-2i. \end{aligned}$$

Exercise 6.10. Find the Hermitian product of $\vec{c} = (3+i, -2+2i, 4-i)^T$ and $\vec{d} = (-1+3i, 2+6i, 2i)^T$.

Recalling the definition of modulus of a complex number (6.2), we can also compute that for a complex n-vector $\vec{z}$ that

$$\langle \vec{z}, \vec{z} \rangle = \sum_{i=1}^{n} z_i \bar{z}_i = \sum_{i=1}^{n} |z_i|^2.$$

So for a complex vector, we define the Euclidean norm as

$$\|\vec{z}\| = \sqrt{\sum_{i=1}^{n} |z_i|^2} = \sqrt{\langle \vec{z}, \vec{z} \rangle}.$$

Recall that for a real *symmetric* $n \times n$ matrix A and any n-vectors $\vec{x}$ and $\vec{y}$ that

$$\begin{aligned}
A\vec{x} \cdot \vec{y} &= \sum_{i=1}^{n} (A\vec{x})_i y_i && \text{(definition of scalar product)} \\
&= \sum_{i=1}^{n} \left(\sum_{j=1}^{n} A_{ij} x_j \right) y_i && \text{(definition of matrix-vector product)} \\
&= \sum_{i=1}^{n} \sum_{j=1}^{n} A_{ji} x_j y_i && \text{(symmetry of } A\text{)} \\
&= \sum_{j=1}^{n} x_j \sum_{i=1}^{n} \left(A_{ji} y_i \right) && \text{(rearranging)} \\
&= \sum_{j=1}^{n} x_j (Ay)_j && \text{(definition of matrix-vector product)} \\
&= \vec{x} \cdot A\vec{y} && \text{(definition of scalar product).}
\end{aligned}$$

Replacing all terms with complex value and the scalar product with the Hermitian product, we get a similar calculation:

$$\begin{aligned}
\langle A\vec{x}, \vec{y} \rangle &= \sum_{i=1}^{n} (A\vec{x})_i \bar{y}_i && \text{(definition of Hermitian product)} \\
&= \sum_{i=1}^{n} \left(\sum_{j=1}^{n} A_{ij} x_j \right) \bar{y}_i && \text{(definition of matrix-vector product)} \\
&= \sum_{i=1}^{n} \sum_{j=1}^{n} A_{ji} x_j \bar{y}_i && \text{(symmetry of } A\text{)} \\
&= \sum_{j=1}^{n} x_j \sum_{i=1}^{n} \left(A_{ji} \bar{y}_i \right) && \text{(rearranging)} \\
&= \sum_{j=1}^{n} x_j (\bar{A}y)_j && \text{(definition of matrix-vector product)} \\
&= \langle \vec{x}, \bar{A}\vec{y} \rangle && \text{(definition of Hermitian).}
\end{aligned}$$

This is quite unsatisfactory (note the extra bar on A in the final equation), and we would like something that generalises the idea of a symmetric matrix to the Hermitian product case too.

The key idea is to generalise the definition of matrix transpose to the complex setting:

Definition 6.4. For a complex n-matrix A, we write A^H for the **conjugate transpose** (also called the **Hermitian transpose**) given by

$$(A^H)_{ij} = \bar{A}_{ji} = \bar{A}^T.$$

If $A^H = A$, then we say that A is **Hermitian**.

Repeating the above calculations, we can see that if A is Hermitian then

$$\langle A\vec{x}, \vec{y}\rangle = \langle \vec{x}, A\vec{y}\rangle.$$

6.7 Summary

In this section, we introduced complex numbers and their algebraic and geometric representations. Key concepts include:

- Definition of complex numbers as combinations of real and imaginary parts.
- Arithmetic operations: addition, subtraction, multiplication, and division.
- Geometric interpretation in the complex plane.
- Complex conjugates and their role in division and modulus.
- Polar representation and argument of complex numbers.
- Application of complex numbers in solving polynomial equations.
- Extension to complex vectors and matrices, including Hermitian products and Hermitian matrices.

These concepts form the foundation for understanding eigenvalues and eigenvectors in the context of complex matrices.

7 Eigenvectors and eigenvalues

Module learning objective

- Compute eigenvalues and eigenvectors of small matrices by hand using characteristic polynomials.
- Explain how similarity transformations preserve eigenvalues and motivate practical algorithms for larger matrices.
- Recognise matrices with favourable eigenvalue structure, such as triangular and symmetric matrices.

This section of the notes will introduce our second big linear algebra problem. Throughout, we will be considering a square $(n \times n)$ matrix.

7.1 Key definitions

We are thinking of a matrix A. We are interested in when the matrix-vector product $A\vec{x}$ is *parallel* to $\vec{x}$.

Definition 7.1. We say that any non-zero vector $\vec{x}$ is an *eigenvector* of A if $A\vec{x}$ is parallel to $\vec{x}$. Here by parallel, we mean that there exists a number λ (which can be positive, negative or zero, real or complex) such that

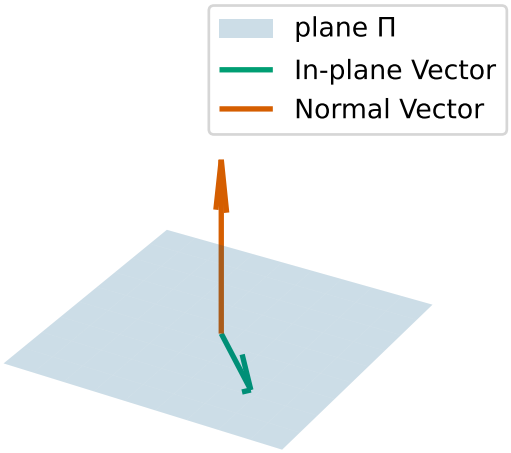
$$A\vec{x} = \lambda\vec{x}. \tag{7.1}$$

We call the associated number λ an *eigenvalue* of A.

We will later see that an $n \times n$ square matrix always has n eigenvalues (which may not always be distinct).

If there exists a nonzero vector $\vec{x}$ with $A\vec{x} = \vec{0}$, then $\vec{x}$ is an eigenvector associated with the eigenvalue 0. In fact, we know that 0 is an eigenvalue of A if, and only if, A is singular.

Example 7.1. Let P be the 3x3 matrix that represents projection onto a plane $\Pi = \{\vec{x} \in \mathbb{R}^3 : \vec{a} \cdot \vec{x} = 0\}$ (i.e. P maps any point in $\mathbb{R}^3$ to its nearest point in Π). What are the eigenvalues and eigenvectors of P?



- if $\vec{x}$ is in the plane Π, then $P\vec{x} = \vec{x}$. This means that $\vec{x}$ is an eigenvector and the associated eigenvalue is 1.
- if $\vec{y}$ is perpendicular to the plane Π, then $P\vec{y} = \vec{0}$. This means that $\vec{y}$ is an eigenvector and the associated eigenvalue is 0.

Let $\vec{y}$ be perpendicular to Π (so that $P\vec{y} = \vec{0}$ and $\vec{y}$ is an eigenvector of P), then for any number s, we can compute

$$P(s\vec{y}) = sP\vec{y} = s\vec{0} = \vec{0}.$$

This means that $s\vec{y}$ is also an eigenvector of P associated with the eigenvalue 0. As a consequence, when we compute eigenvectors, we need to take care to *normalise* the vector to ensure we get a unique answer.

We end up with a two-dimensional space of eigenvectors (i.e., the plane Π) associated with eigenvalue 1 and a one-dimensional space of eigenvectors (i.e., the line perpendicular to Π) associated with the eigenvalue 0.

Example 7.2. Let A be the permutation matrix which takes an input two-vector and outputs a two-vector with the components swapped. The matrix is given by

$$A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

What are the eigenvectors and eigenvalues of A?

- Let $\vec{x} = (1, 1)^T$, then swapping the components of $\vec{x}$ gives back the same vector $\vec{x}$. In equations, we can write $A\vec{x} = \vec{x}$. This means that $\vec{x}$ is an eigenvector and the eigenvalue is 1.
- Let $\vec{x} = (-1, 1)^T$, then swapping the components of $\vec{x}$ gives back $(1, -1)^T$ which we can see is $-\vec{x}$. In equations, we can write $A\vec{x} = -\vec{x}$. This means that $\vec{x}$ is an eigenvector of A and the associated eigenvalue is -1.

Remark. If we take the product of all eigenvalues of a $n \times n$ matrix A, we get the determinant of the matrix A:

$$\det A = \lambda_1 \cdot \lambda_2 \cdots \lambda_n.$$

If we add up all the eigenvalues of a $n \times n$ matrix, we get the *trace* of the matrix A. We can also find the trace by adding up the diagonal components of the matrix:

$$\lambda_1 + \cdots + \lambda_n = a_{11} + a_{22} + \cdots + a_{nn} = \text{trace}(A).$$

Let A and B be two $n \times n$ matrices. Then, we cannot use the eigenvalues of A and B to work out the eigenvalues of $A + B$ or the eigenvalues of AB, in general.

7.2 Why does a computer scientist need to know about eigenvalues

Eigenvalues and eigenvectors can be used to encode information about a dataset into a lower dimensional representation. Take, for example, the health record of a patient which might contain many dimensions of data (e.g. height, weight, resting heart rate, different blood test results, diet and exercise information). This dataset will be hard to compute with because these columns are not independent (more on this later in the Statistics section of the module) and the large dimension makes computation expensive. One way to reduce the dimensionality is through projecting the dataset onto a small number of directions which come from the eigenvectors of a related matrix (the covariance or correlation matrix). This is called principal component analysis (PCA) and you will learn more about this in a project later in this module.

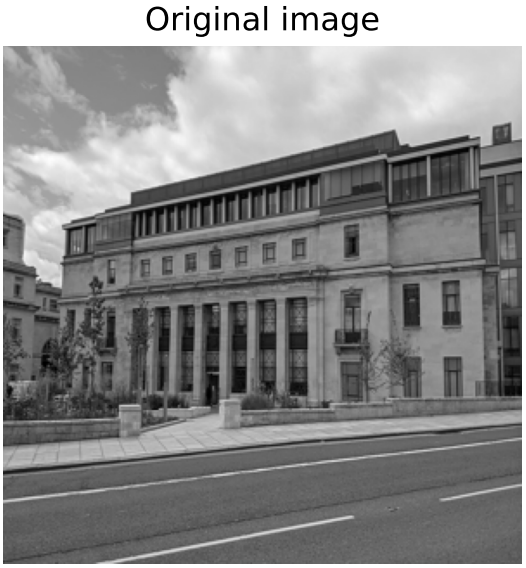
We demonstrate briefly here how PCA can work for compressing an image. Consider this image of the Bragg building:

Step 1: we load the image and do some normalisation:

```
img = Image.open("images/bragg-building.jpg").convert("L")
img = img.resize((300, 300))  # reduce size
img_array = np.asarray(img, dtype=np.float64) / 255.0

plt.title("Original image")
```

```
plt.imshow(img_array, cmap="gray")
plt.axis("off")
plt.show()
```



Step 2: we compute the covariance matrix

```
mean_vector = np.mean(img_array, axis=0)
centered_data = img_array - mean_vector
cov_matrix = np.cov(centered_data, rowvar=False)
```

Step 3: compute eigenvalues and eigenvectors

```
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)

# Sort eigenvalues and eigenvectors in descending order
sorted_indices = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[sorted_indices]
eigenvectors = eigenvectors[:, sorted_indices]
```

Step 4: reconstruct the image using the top k components

```python
def reconstruct_image(data, eigenvectors, mean_vector, k):
    """
    Reconstructs the image using the top k eigenvectors.
    """
    top_eigenvectors = eigenvectors[:, :k]
    compressed_data = data @ top_eigenvectors
    reconstructed = compressed_data @ top_eigenvectors.T + mean_vector
    return reconstructed
```

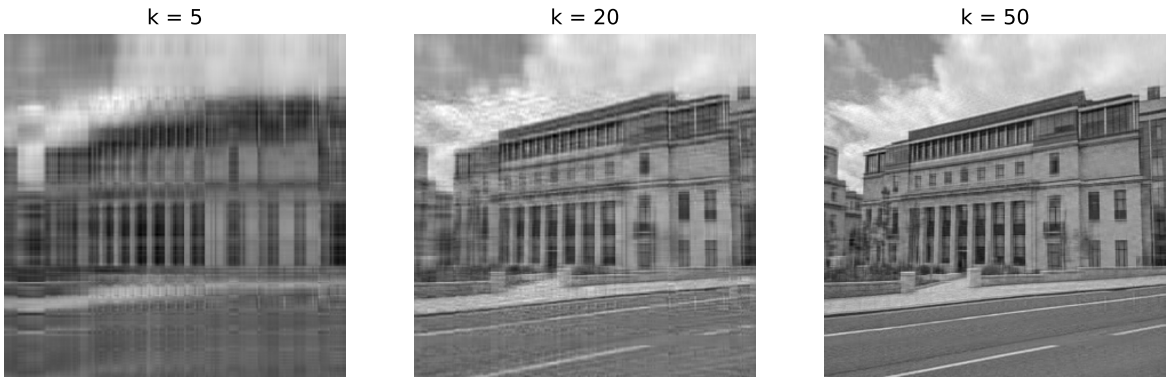
Step 5: display the results for different numbers of eigenvectors:

```python
k_values = [5, 20, 50]

plt.figure(figsize=(12, 8))
plt.subplot(1, len(k_values), 1)

for i, k in enumerate(k_values):
    reconstructed_img = reconstruct_image(
        centered_data, eigenvectors, mean_vector, k
    )
    plt.subplot(1, len(k_values), i + 1)
    plt.imshow(reconstructed_img, cmap="gray")
    plt.title(f"k = {k}")
    plt.axis("off")

plt.show()
```



We see that even with a very small number of values we can start to recognise our building!

7.3 How to find eigenvalues and eigenvectors

To compute eigenvalues and eigenvectors, we start from (7.1) and move everything to the left-hand side and use the identity matrix (I_n):

$$(A - \lambda I_n)\vec{x} = \vec{0}.$$

This tells us that if we want to find an eigenvalue of A, then we need to find a number λ such that $(A - \lambda I_n)$ can multiply a non-zero vector and give us back the zero-vector. This happens when $(A - \lambda I_n)$ is *singular*.

As we saw earlier one way to test if a matrix is singular, is if the determinant is 0. This gives us a test we can use to determine eigenvalues:

$$\det(A - \lambda I_n) = 0. \tag{7.2}$$

In fact, this equation no longer depends on the eigenvector $\vec{x}$, and if we can find solutions λ to this equation then λ is an eigenvalue of A.

We call (7.2) the *characteristic equation* or *eigenvalue equation*. We will see that (7.2) gives us a degree n polynomial equation in λ.

Once we have found an eigenvalue by solving the characteristic equation for a value λ^*, we need to find a vector $\vec{x}$ such that

$$(A - \lambda^*)\vec{x} = \vec{0}.$$

In general, this is possible using a variation of Gaussian elimination with pivoting, but we do not explore this method in this module. All the examples of finding eigenvectors we will see, you will be able to solve simply by inspection.

Example 7.3. Let A be the matrix given by

$$A = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}.$$

Then, we can compute that

$$\begin{aligned} \det(A - \lambda I_n) &= \det \begin{pmatrix} 3-\lambda & 1 \\ 1 & 3-\lambda \end{pmatrix} \\ &= (3-\lambda)(3-\lambda) - 1 \times 1 \\ &= \lambda^2 - 6\lambda + 8. \end{aligned}$$

So we want to find values λ such that

$$\det(A - \lambda I_n) = \lambda^2 - 6\lambda + 8 = 0.$$

By factorisation, we can read off that the values of λ are 4 and 2.

We can now start computing the associated eigenvectors.

To find the eigenvector associated with the eigenvalue 4. We see that

$$A - 4I_n = \begin{pmatrix} -1 & 1 \\ 1 & -1 \end{pmatrix}.$$

We can identify that $(A - 4I_n)(1,1)^T = \vec{0}$. So $(1,1)$ is an eigenvector associated with 4.

To find the eigenvector associated with the eigenvalue 2. We see that

$$A - 2I_n = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}.$$

We can identify that $(A - 2I_n)(-1,1)^T = \vec{0}$. So $(-1,1)^T$ is an eigenvector associated with 2.

We note that this example is actually surprisingly similar to Example 7.2. We see that the eigenvectors are actually the same! We can see that the matrices are related too:

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} + 3I_n = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}.$$

So we can compute that if $A\vec{x} = \lambda\vec{x}$ then

$$(A + 3I_n)\vec{x} = \lambda\vec{x} + 3\vec{x} = (\lambda + 3)\vec{x}.$$

So we see that $\vec{x}$ is also an eigenvector of $A + 3I_n$ and the associated eigenvalue is $\lambda + 3$.

Although this procedure gives us back eigenvalues there are cases where we have to be careful. We have seen one example above with a two-dimensional eigenspace associated with one eigenvector (Example 7.1). Here are two other cases we must be careful:

Example 7.4. Let Q denote the 2×2 matrix that rotates any vector anticlockwise by $\pi/2$ $(= 90^\circ)$:

$$Q = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}.$$

Our intuition says that there can be no vectors that when rotated by $\pi/2$ give something parallel to the input vector, but we can still compute:

$$\det(Q - \lambda I_n) = \det \begin{pmatrix} -\lambda & -1 \\ 1 & -\lambda \end{pmatrix} = \lambda^2 + 1.$$

So we can find eigenvalues by finding the values λ such that

$$\lambda^2 + 1 = 0.$$

We saw in the section on Complex Numbers (Chapter 6) that the solutions to this equation are $\pm i$. This means that a general algorithm for finding eigenvalues and eigenvectors need to handle complex numbers too.

Example 7.5. Let A be the 2×2 matrix given by

$$A = \begin{pmatrix} 3 & 1 \\ 0 & 3 \end{pmatrix}.$$

If we follow our procedure above we get a single repeated eigenvalue 3.

Looking at the shifted matrix, $A - 3I_n$:

$$A - 3I_n = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}.$$

We can identify one eigenvector $(1, 0)^T$, but there is no other eigenvector (in a different direction)! Indeed, we can compute that:

$$(A - 3I_n) \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} y \\ 0 \end{pmatrix}.$$

This tells us that if $(A - 3I_n)(x, y)^T = \vec{0}$ if, and only if, $y = 0$. Thus all eigenvectors have the form $(x, 0)^T$ and point in the same direction as $(1, 0)^T$.

Exercise 7.1. Find the eigenvalues and eigenvectors for the matrix:

$$A = \begin{pmatrix} 9 & -2 \\ -2 & 6 \end{pmatrix}.$$

Example 7.6 (Eigenvalues and eigenvectors of a triangular matrix)**.** Consider an upper triangular matrix (Example 3.6). When forming the characteristic equation, we see that most terms are simply zero:

$$\begin{aligned}
&\det \begin{pmatrix} 3-\lambda & 2 \\ 0 & 1-\lambda \end{pmatrix} \\
&= (3-\lambda)(1-\lambda) - 2 \times 0 = (3-\lambda)(1-\lambda) \\
&\det \begin{pmatrix} 3-\lambda & 2 & 4 \\ 0 & 1-\lambda & -1 \\ 0 & 0 & 2-\lambda \end{pmatrix} \\
&= (3-\lambda)((1-\lambda)(2-\lambda) - (-1) \times 0) - 2(0(2-\lambda) - (-1 \times 0)) + 4(0 \times 0 - 1 \times 0) \\
&= (3-\lambda)(1-\lambda)(2-\lambda).
\end{aligned}$$

Indeed this is true in general,

$$\begin{aligned}
\det(A - \lambda I_n) &= \det \begin{pmatrix} a_{11}-\lambda & a_{12} & a_{13} & \cdots & a_{1n} \\ 0 & a_{22}-\lambda & a_{23} & \cdots & a_{2n} \\ 0 & 0 & a_{33}-\lambda & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & a_{nn}-\lambda \end{pmatrix} \\
&= \prod_{i=1}^{n} (a_{ii} - \lambda).
\end{aligned}$$

Therefore, the characteristic polynomial factors completely without any further computation. So we can read off that the eigenvalues of an upper triangular matrix are simply the diagonal coefficients.

7.4 Important theory

We have established a way to identify eigenvalues and eigenvectors for an arbitrary square matrix. This method can be used to prove the existence of eigenvalues.

Theorem 7.1. *Any square $n \times n$ matrix has n complex eigenvalues (possibly not distinct).*

Proof. For any matrix the characteristic equation (7.2) is a degree n polynomial. The Fundamental Theorem of Algebra (Theorem 6.1) tells us that any degree n polynomial has n roots over the complex numbers. The n roots of the characteristic equation are the n eigenvalues. □

The Abel-Ruffini theorem states that there is no solution in the radicals for a general polynomial of degree 5 or higher with arbitrary coefficients. This implies that there is no 'nice' closed form for roots of polynomials of degree 5 or higher. So, if we want an algorithm to find eigenvalues and eigenvectors of larger matrices then we need a different approach!

7.5 Similar matrices

Let's suppose that we have an $n \times n$ matrix A and we have found n eigenvectors and n eigenvalues (all distinct). Let's call the eigenvectors by $\vec{x}_1, \ldots \vec{x}_n$ and the eigenvalues $\lambda_1, \ldots \lambda_n$ then we have the equation:

$$A\vec{x}_j = \lambda_j \vec{x}_j.$$

So if we form the matrices S to have columns equal to each eigenvector in turn and Λ (this is the capital version of λ) to be the diagonal matrix with the eigenvalues listed along the diagonal we see that we have:

$$AS = S\Lambda. \tag{7.3}$$

If S is invertible, we can multiply on the right by S^{-1} to see that we have

$$A = S\Lambda S^{-1}. \tag{7.4}$$

This formula shows another factorisation of the matrix A into simpler matrices, very much like we had when we computed the LU-factorisation matrix (Section 4.7).

The equation (7.3) is an example of a more general idea of *similar matrices*. The matrix S, above, plays the role of P in the definition here.

Definition 7.2. We say that two matrices A and B are **similar** if there exists an invertible $n \times n$ matrix P such that

$$PB = AP.$$

Since P is invertible, we can pre-multiply this equation by P^{-1} to obtain $B = P^{-1}AP$, and post-multiply by P^{-1} to obtain $A = PBP^{-1}$. Thus, we infer that being similar is a symmetric property.

Remark 7.1. The matrix form of the eigenvalue equation (7.3) says that the matrix A is similar to the diagonal matrix of eigenvalues, **if the matrix S is invertible.**

This leads to a nice theorem which we will use to help compute eigenvectors and eigenvalues of larger matrices:

Theorem 7.2. *If A and B are similar matrices then A and B have the same eigenvalues.*

Proof. Let $\vec{x}$ be an eigenvector of B with associated eigenvalue λ and $\vec{y} = P\vec{x}$. Then

$$\begin{aligned} A\vec{y} &= AP\vec{x} && \text{(definition of } \vec{y}) \\ &= PB\vec{x} && \text{(since } A \text{ and } B \text{ are similar)} \\ &= P\lambda\vec{x} && \text{(since } \vec{x} \text{ is an eigenvector of } B) \\ &= \lambda(P\vec{x}) && \text{(rearranging)} \\ &= \lambda\vec{y}. \end{aligned}$$

This shows that any eigenvalue of B is an eigenvalue of A. It also gives a formula for how eigenvectors change between B and A.

To show any eigenvalue of A is an eigenvalue of B, we repeat the calculation with A and B swapped and replace P by P^{-1}. □

The key idea behind the methods we will use to compute eigenvalues is to apply a sequence of matrices to convert a matrix A into a form similar to A for which reading off the eigenvalues is easier. However, the quality of the algorithms we apply depend heavily on properties of the matrix A.

Example 7.7. The matrices

$$A = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} \quad \text{and} \quad B = \begin{pmatrix} 1 & -2 \\ 0 & 2 \end{pmatrix}$$

are similar.

Since A is diagonal, we can read off the eigenvalues are 1 and 2 and we can take $(1,0)^T$ and $(0,1)^T$ are eigenvectors:

$$A\begin{pmatrix}1 & 0\\ 0 & 1\end{pmatrix} = \begin{pmatrix}1 & 0\\ 0 & 1\end{pmatrix}\begin{pmatrix}1 & 0\\ 0 & 2\end{pmatrix}.$$

For B, we can compute the characteristic equation:

$$0 = \det(B - \lambda I_2) = \det\begin{pmatrix}1-\lambda & -2\\ 0 & 2-\lambda\end{pmatrix} = (1-\lambda)(2-\lambda).$$

So we can see the eigenvalues are 1 and 2.

For $\lambda = 1$, we can find an eigenvalue by finding a vector $(x,y)^T$ that satisfies

$$\begin{pmatrix}0\\ 0\end{pmatrix} = (B - 1I_2)\begin{pmatrix}x\\ y\end{pmatrix} = \begin{pmatrix}0 & -2\\ 0 & 1\end{pmatrix}\begin{pmatrix}x\\ y\end{pmatrix} = \begin{pmatrix}-2y\\ y\end{pmatrix}.$$

We see this implies $y = 0$ but any value of x is allowed: so we take $x = 1$.

For $\lambda = 2$, we can find an eigenvalue by finding a vector $(x,y)^T$ that satisfies

$$\begin{pmatrix}0\\ 0\end{pmatrix} = (B - 2I_2)\begin{pmatrix}x\\ y\end{pmatrix} = \begin{pmatrix}-1 & -2\\ 0 & 0\end{pmatrix}\begin{pmatrix}x\\ y\end{pmatrix} = \begin{pmatrix}-x-2y\\ 0\end{pmatrix}.$$

The first component gives $x = -2y$. So we can take $(-2,1)$ as eigenvector.

The matrix form of eigenvalue equation gives

$$B\begin{pmatrix}1 & -2\\ 0 & 1\end{pmatrix} = \begin{pmatrix}1 & -2\\ 0 & 1\end{pmatrix}\begin{pmatrix}1 & 0\\ 0 & 2\end{pmatrix}.$$

We can identify part of the right hand side as A:

$$B\begin{pmatrix}1 & -2\\ 0 & 1\end{pmatrix} = \begin{pmatrix}1 & -2\\ 0 & 1\end{pmatrix}A.$$

Let $P = \begin{pmatrix}1 & -2\\ 0 & 1\end{pmatrix}$ - this is our candidate for the similarity definition. We need to check first that P is invertible:

$$\det P = 1 \times 1 - (-2) \times 0 = 1.$$

The determinant is nonzero so P is invertible.

To finally check, we also compute

$$PB = \begin{pmatrix}1 & -4\\ 0 & 2\end{pmatrix} = AP.$$

What about the converse direction? If two matrices A and B have the same eigenvalues, are they similar?

Example 7.8. Are these matrices similar?

$$A = \begin{pmatrix} 3 & 1 \\ 0 & 3 \end{pmatrix} \quad \text{and} \quad B = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}.$$

We saw before the eigenvalues of A are 3 and 3. We can read off the eigenvalues of B are also 3 repeated.

Assume for contradiction that A and B are similar so $PB = AP$ for an invertible matrix P. Let $P = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$\$. Then

$$PB = \begin{pmatrix} 3a & 3b \\ 3c & 3d \end{pmatrix}$$
$$AP = \begin{pmatrix} 3a + c & 3b + d \\ 3c & 3d \end{pmatrix}.$$

Equating components we see that

$$\begin{aligned} 3a &= 3a + c & &\Rightarrow c = 0 \\ 3b &= 3b + d & &\Rightarrow d = 0. \end{aligned}$$

This implies that P has the form

$$P = \begin{pmatrix} a & b \\ 0 & 0 \end{pmatrix},$$

and $\det P = 0$. This contradicts that P is invertible so A and B are not similar.

However, there is hope in special cases: We know that we can write,

$$AS = S\Lambda \quad \text{and} \quad BR = R\Lambda.$$

If R is invertible, then

$$ASR^{-1} = SR^{-1}B.$$

If S is also invertible, then SR^{-1} is also invertible and A and B are similar (with $P = SR^{-1}$). In fact, this is the idea we used in Example 7.7.

7.6 Eigenvalues of symmetric matrices

Whilst all our methods are developed for complex matrices, when we restrict to real-valued symmetric matrices, we have this nice result.

Before we state the result, we need one more definition:

Definition 7.3. We say two vectors $\vec{a}$ and $\vec{b}$ are **orthogonal** if $\langle \vec{a}, \vec{b} \rangle = 0$.

Theorem 7.3. *Let A be a symmetric matrix ($A^T = A$) with real entries. Then A has n real eigenvalues (zero imaginary part) and any eigenvectors with distinct eigenvalues are orthogonal.*

Knowing the proof of this result is not essential for this module and is only included here for interest.

Proof. Let λ be an eigenvalue of A with eigenvector $\vec{x}$. Recall that $\vec{x} \neq 0$. Then, since A has real values, we can compute that:

$$(\bar{A\vec{x}})_i = \left(\sum_{j=1}^{n} \bar{A}_{ij} x_j \right) = \sum_{j=1}^{n} A_{ij} \bar{x_j} = (A\bar{\vec{x}})_i.$$

We also note that any real, symmetric matrix is automatically Hermitian.

Then we see that

$$\begin{aligned}
\lambda \langle \vec{x}, \vec{x} \rangle &= \langle (\lambda \vec{x}), \vec{x} \rangle && \text{(from definition of Hermitian product)} \\
&= \langle (A\vec{x}), \vec{x} \rangle && \text{(definition of eigenvalue and eigenvector)} \\
&= \langle \vec{x}, A\vec{x} \rangle && \text{(symmetry of } A\text{)} \\
&= \langle \vec{x}, \lambda \vec{x} \rangle && \text{(definition of eigenvalue and eigenvector)} \\
&= \bar{\lambda} \langle \vec{x}, \vec{x} \rangle && \text{(from definition of Hermitian product).}
\end{aligned}$$

Since, $\langle \vec{x}, \vec{x} \rangle > 0$ (recall $\vec{x} \neq 0$), we can divide by $\langle \vec{x}, \vec{x} \rangle$ so infer that

$$\lambda = \bar{\lambda}.$$

Next, let $\vec{x}$ and $\vec{y}$ be eigenvectors of A with distinct, eigenvalues λ and μ, respectively. From the first part of the proof, we know that λ and μ are real. We compute that

$$\begin{aligned}
\lambda \langle \vec{x}, \vec{y} \rangle &= \langle \lambda \vec{x}, \vec{y} \rangle \\
&= \langle A\vec{x}, \vec{y} \rangle \\
&= \langle \vec{x}, A^H \vec{y} \rangle \\
&= \langle \vec{x}, A\vec{y} \rangle \\
&= \langle \vec{x}, \mu \vec{y} \rangle \\
&= \bar{\mu} \langle \vec{x}, \vec{y} \rangle \\
&= \mu \langle \vec{x}, \vec{y} \rangle.
\end{aligned}$$

Subtracting the right-hand side from the left hand side we see that

$$(\lambda - \mu)\langle \vec{x}, \vec{y} \rangle = 0.$$

This implies that if λ and μ are distinct, then $\langle \vec{x}, \vec{y} \rangle = 0$. □

Corollary 7.1. *Let A be a symmetric $n \times n$ matrix with real entries with n distinct eigenvalues. Then the eigenvectors of A form a basis of $\mathbb{R}^n$.*

Proof. We can only give an incomplete proof of this result. We will show that the eigenvectors are linearly independent. The proof is completed by showing that if you have any n linearly independent vectors in $\mathbb{R}^n$ then you must have a basis.

Denote by $\vec{x}^{(1)}, \ldots \vec{x}^{(n)}$ the n eigenvectors of A. We want to show that the eigenvectors are linearly independent. Suppose that we have real numbers $\alpha_1, \alpha_2, \cdots, \alpha_n$ such that:

$$\sum_{i=1}^{n} \alpha_i \vec{x}^{(i)} = \vec{0}. \tag{7.5}$$

To show the eigenvectors are linearly independent, we need to show that all $\alpha_i = 0$. We can do this by taking the inner product of (7.5) with $\vec{x}^{(j)}$ for *any* j:

$$0 = \vec{0} \cdot \vec{x}^{(j)} = \left(\sum_{i=1}^{n} \alpha_i \vec{x}^{(i)} \right) \cdot \vec{x}^{(j)} = \sum_{i=1}^{n} \alpha_i \left(\vec{x^{(i)}} \cdot \vec{x}^{(j)} \right) = \alpha_j \vec{x}^{(j)} \cdot \vec{x}^{(j)}.$$

Since we have that $|\vec{x}^{(j)}| > 0$, we have $\alpha_j = 0$ and we have shown that the eigenvectors are linearly independent. □

7.7 Summary

- When working with eigenvalue problems it's important to work with complex matrices and vectors.
- Similar matrices have the same eigenvalues and we have a formula for transferring eigenvectors from one matrix to another similar matrix.
- Real symmetric matrices have real eigenvalues and (if distinct) orthogonal eigenvectors. (Our method will only work for symmetric matrices).

8 Eigenvectors and eigenvalues: practical solutions

Module learning objective

- Derive and apply the Gram-Schmidt process to compute a QR factorisation of a matrix by hand.
- Implement the QR algorithm in Python and evaluate its computational feasibility on large matrices.
- Understand how the QR algorithm preserves eigenvalues using similarity

8.1 Recall: the eigenvalue problem

In the previous lecture, we defined the eigenvalue problem for a matrix A: Finding numbers λ (eigenvalues) and vectors $\vec{x}$ (eigenvectors) which satisfy the equation:

$$A\vec{x} = \lambda\vec{x}. \tag{8.1}$$

We saw one starting point for finding eigenvalues is to find the roots of the characteristic equation: a polynomial of degree n for an $n \times n$ matrix A. However, we have already seen that this approach will be infeasible for large matrices – and matrices arising in practice are often large. Recall, for example, the image-compression demonstration from the last lecture: even a modest 300×300-pixel image led to an eigenvalue problem for a 300×300 covariance matrix, and a characteristic polynomial of degree 300 is not something we can find the roots of directly. Instead, we will find a sequence of similar matrices to A such that we can read off the eigenvalues from the final matrix.

8.2 The grand strategy

In equations, we can say our "grand strategy" is to find a sequence of matrices $P_1, P_2, ...$ to form a sequence of matrices:

$$A, \quad P_1^{-1}AP_1, \quad P_2^{-1}P_1^{-1}AP_1P_2, \quad P_3^{-1}P_2^{-1}P_1^{-1}AP_1P_2P_3, ... \tag{8.2}$$

We aim to get all the way to a simple matrix where we read off the eigenvalues and eigenvectors.

For example, if at level m, say, we have transformed A into a diagonal matrix the eigenvalues are the diagonal of the matrix

$$P_m^{-1}P_{m-1}^{-1}\cdots P_2^{-1}P_1^{-1}AP_1P_2\cdots P_{m-1}P_m,$$

and the eigenvectors are the columns of the matrix

$$S_m = P_1P_2\cdots P_{m-1}P_m.$$

We have seen a similar example for upper triangular matrices (Example 7.6).

Remark 8.1. In the example below, and many others, we see that we typically want the matrices P_j to be **orthogonal**.

A (real-valued) matrix Q is orthogonal if $Q^TQ = I_n$.

In other words, if we denote by $\vec{q}^{(1)}, \ldots, \vec{q}^{(n)}$ the columns of Q, then Q is orthogonal if the columns are *orthonormal*:

$$\vec{q}^{(i)} \cdot \vec{q}^{(j)} = \begin{cases} 0 & \text{if } i \neq j \\ 1 & \text{if } i = j. \end{cases}$$

Orthogonal matrices have the nice property that $Q^{-1} = Q^T$ so we can very easily compute their inverse! They also always have a determinant of ± 1. Importantly, we can apply them (or their inverses) without worrying about adding extra problems from finite precision.

Exercise 8.1. A standard example of an orthogonal matrix is a rotation matrix (recall Example 1.3). Show that the rotation matrix $R(\theta)$ is orthogonal:

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

8.3 QR algorithm

The QR algorithm is an iterative method for computing eigenvalues and eigenvectors. At each step a matrix is factorised in a similar fashion to LU factorisation (Section 4.7). In this case, we factor a matrix, A into a product of an orthogonal matrix, Q, and an upper triangular matrix, R:

$$A = QR.$$

This is *QR factorisation*.

Given a matrix A, the algorithm repeatedly applies QR factorisation. First, we set $A^{(0)} = A$, then we successively perform for $k = 0, 1, 2, \ldots$:

1. Compute the QR factorisation of $A^{(k)}$ into an orthogonal part and upper triangular part

$$A^{(k)} = Q^{(k)}R^{(k)};$$

2. Update the matrix $A^{(k+1)}$ by recombining Q and R in the reverse order:

$$A^{(k+1)} = R^{(k)}Q^{(k)}.$$

As we take more and more steps, we hope that $A^{(k)}$ converges to an upper triangular matrix whose diagonal entries are the eigenvalues of the original matrix. For a real symmetric matrix, the limit is in fact diagonal.

Rearranging the first step within each iteration, we see that

$$R^{(k)} = (Q^{(k)})^{-1}A^{(k)} = (Q^{(k)})^{T}A^{(k)}.$$

Substituting this value of $R^{(k)}$ into the second step gives

$$A^{(k+1)} = (Q^{(k)})^{-1}A^{(k)}Q^{(k)},$$

and we see that at each step we are finding a sequence of similar matrices, all with the same eigenvalues (Theorem 7.2). For a symmetric matrix, the accumulated product

$$Q = Q^{(0)}Q^{(1)}Q^{(2)}\cdots Q^{(m)}$$

converges to a matrix whose columns are eigenvectors of A.

The hard part of the method is computing the QR factorisation. One classical way to get a QR factorisation is to use the Gram-Schmidt process. In general, the Gram-Schmidt process is used to take a sequence of vectors and form a new sequence which is orthonormal. We can apply this to the columns of A to form an orthogonal matrix. If we track this process as a matrix-matrix product, we find that the other factor is upper triangular.

8.3.1 The Gram-Schmidt process

The key idea is shown in Figure 8.1. Given a vector $\vec{a}$ (blue) and a vector $\vec{q}$ (green) with length 1. We can compute the projection of $\vec{a}$ onto the line spanned by $\vec{q}$ (orange) by

$$(\vec{a}\cdot\vec{q})\vec{q}.$$

If we subtract this term from $\vec{a}$. We end up with a vector $\vec{u}$ with $\vec{u}\cdot\vec{q} = 0$. The difference $\vec{u}$ is given by

$$\vec{u} = \vec{a} - (\vec{a}\cdot\vec{q})\vec{q},$$

and we can compute that

$$\begin{aligned}
\vec{u} \cdot \vec{q} &= (\vec{a} - (\vec{a} \cdot \vec{q})\vec{q}) \cdot \vec{q} \\
&= (\vec{a} \cdot \vec{q}) - (\vec{a} \cdot \vec{q})(\vec{q} \cdot \vec{q}) && \text{(properties of scalar product)} \\
&= (\vec{a} \cdot \vec{q}) - (\vec{a} \cdot \vec{q}) && \text{(since } \|\vec{q}\| = 1\text{)} \\
&= 0.
\end{aligned}$$

We see that we have found a vector $\vec{u}$ which is orthogonal to $\vec{q}$.

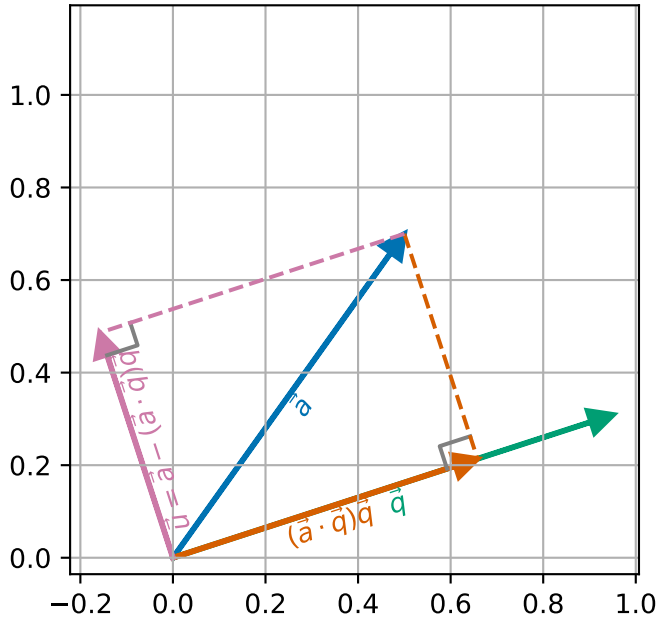


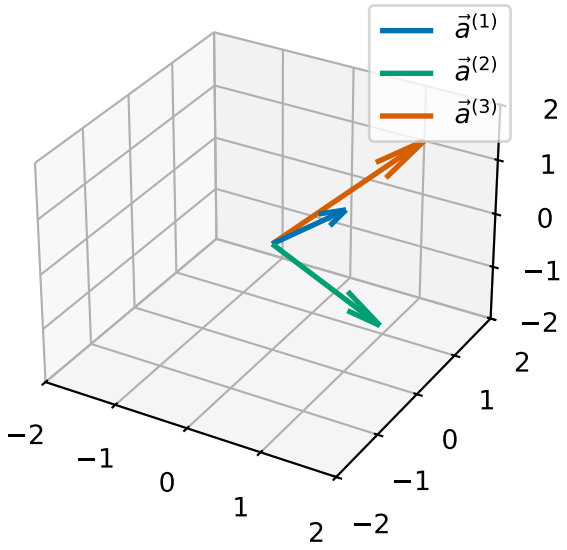
Figure 8.1: Projection of $\vec{a}$ away from $\vec{q}$ onto $\vec{u}$.

The Gram-Schmidt process works by repeatedly applying this idea to a sequence of vectors to form a new sequence of orthonormal vectors.

Example 8.1. Consider the sequence of vectors

$$\vec{a}^{(1)} = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}, \vec{a}^{(2)} = \begin{pmatrix} 2 \\ -1 \\ 0 \end{pmatrix}, \vec{a}^{(3)} = \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}.$$

We will manipulate these vectors to form three orthonormal vectors.



First, we set $\vec{q}^{(1)} = \vec{a}^{(1)}/\|\vec{a}^{(1)}\|$:

$$\|\vec{a}^{(1)}\| = \sqrt{1^2 + 0^2 + 1^2} = \sqrt{2},$$

so

$$\vec{q}^{(1)} = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ 0 \\ \frac{1}{\sqrt{2}} \end{pmatrix}.$$

Second, we want to find $\vec{q}^{(2)}$ which must satisfy that $\vec{q}^{(2)} \cdot \vec{q}^{(1)} = 0$. We can do this by subtracting from $\vec{a}^{(2)}$ the portion of $\vec{a}^{(2)}$ which points in the direction $\vec{q}^{(1)}$. We call this $\vec{u}^{(2)}$:

$$\begin{aligned}
\vec{u}^{(2)} &= \vec{a}^{(2)} - \left(\vec{a}^{(2)} \cdot \vec{q}^{(1)}\right) \vec{q}^{(1)} \\
&= \begin{pmatrix} 2 \\ -1 \\ 0 \end{pmatrix} - \left(\begin{pmatrix} 2 \\ -1 \\ 0 \end{pmatrix} \cdot \begin{pmatrix} \frac{1}{\sqrt{2}} \\ 0 \\ \frac{1}{\sqrt{2}} \end{pmatrix} \right) \begin{pmatrix} \frac{1}{\sqrt{2}} \\ 0 \\ \frac{1}{\sqrt{2}} \end{pmatrix} \\
&= \begin{pmatrix} 2 \\ -1 \\ 0 \end{pmatrix} - \frac{2}{\sqrt{2}} \begin{pmatrix} \frac{1}{\sqrt{2}} \\ 0 \\ \frac{1}{\sqrt{2}} \end{pmatrix} \\
&= \begin{pmatrix} 2 - \frac{2}{\sqrt{2}}\frac{1}{\sqrt{2}} \\ -1 - \frac{2}{\sqrt{2}}0 \\ 0 - \frac{2}{\sqrt{2}}\frac{1}{\sqrt{2}} \end{pmatrix} = \begin{pmatrix} 1 \\ -1 \\ -1 \end{pmatrix}.
\end{aligned}$$

We then normalise $\vec{u}^{(2)}$ to get a unit-length vector $\vec{q}^{(2)}$:

$$\vec{q}^{(2)} = \vec{u}^{(2)}/\|\vec{u}^{(2)}\| = \begin{pmatrix} \frac{1}{\sqrt{3}} \\ \frac{-1}{\sqrt{3}} \\ \frac{-1}{\sqrt{3}} \end{pmatrix}.$$

Third, we will find $\vec{q}^{(3)}$ which we need to check satisfies $\vec{q}^{(3)} \cdot \vec{q}^{(2)} = 0$ and $\vec{q}^{(3)} \cdot \vec{q}^{(1)} = 0$. We can do this by subtracting from $\vec{a}^{(3)}$ the portion of $\vec{a}^{(3)}$ which points in the direction $\vec{q}^{(1)}$ and the portion of $\vec{a}^{(3)}$ which points in the direction $\vec{q}^{(2)}$. We call this term $\vec{u}^{(3)}$

$$\begin{aligned}
\vec{u}^{(3)} &= \vec{a}^{(3)} - \left(\vec{a}^{(3)} \cdot \vec{q}^{(1)}\right) \vec{q}^{(1)} - \left(\vec{a}^{(3)} \cdot \vec{q}^{(2)}\right) \vec{q}^{(2)} \\
&= \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} - \left(\begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} \frac{1}{\sqrt{2}} \\ 0 \\ \frac{1}{\sqrt{2}} \end{pmatrix} \right) \begin{pmatrix} \frac{1}{\sqrt{2}} \\ 0 \\ \frac{1}{\sqrt{2}} \end{pmatrix} - \left(\begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} \frac{1}{\sqrt{3}} \\ \frac{-1}{\sqrt{3}} \\ \frac{-1}{\sqrt{3}} \end{pmatrix} \right) \begin{pmatrix} \frac{1}{\sqrt{3}} \\ \frac{-1}{\sqrt{3}} \\ \frac{-1}{\sqrt{3}} \end{pmatrix} \\
&= \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} - \left(\frac{2}{\sqrt{2}} \right) \begin{pmatrix} \frac{1}{\sqrt{2}} \\ 0 \\ \frac{1}{\sqrt{2}} \end{pmatrix} - \left(\frac{-2}{\sqrt{3}} \right) \begin{pmatrix} \frac{1}{\sqrt{3}} \\ \frac{-1}{\sqrt{3}} \\ \frac{-1}{\sqrt{3}} \end{pmatrix} \\
&= \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} - \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} - \begin{pmatrix} -2/3 \\ 2/3 \\ 2/3 \end{pmatrix} = \begin{pmatrix} 2/3 \\ 4/3 \\ -2/3 \end{pmatrix}.
\end{aligned}$$

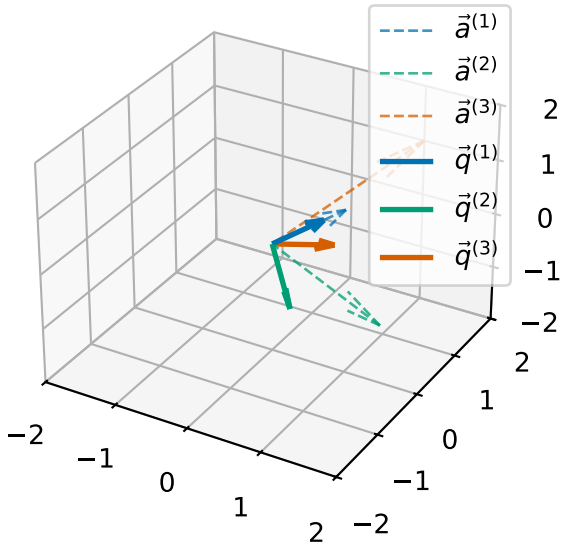
Again, we normalise $\vec{u}^{(3)}$ to get $\vec{q}^{(3)}$:

$$\begin{aligned}
\|\vec{u}^{(3)}\| &= \sqrt{(2/3)^2 + (4/3)^2 + (-2/3)^2} = \sqrt{4/9 + 16/9 + 4/9} = \sqrt{24/9} \\
&= \frac{2}{3}\sqrt{6},
\end{aligned}$$

so

$$\vec{q}^{(3)} = \begin{pmatrix} \frac{1}{\sqrt{6}} \\ \frac{2}{\sqrt{6}} \\ \frac{-1}{\sqrt{6}} \end{pmatrix}.$$

Exercise 8.2. Verify the orthonormality conditions for $\vec{q}^{(1)}, \vec{q}^{(2)}$ and $\vec{q}^{(3)}$.



Now we have applied the Gram-Schmidt process to convert from the vectors $\vec{a}^{(j)}$ to the vectors $\vec{q}^{(j)}$. We can consider the matrix Q whose columns are the vectors $\vec{q}^{(j)}$ and the matrix A whose columns are the vectors $\vec{a}^{(j)}$:

$$Q = \begin{pmatrix} \vec{q}^{(1)} & \vec{q}^{(2)} & \vec{q}^{(3)} \end{pmatrix} \quad \text{and} \quad A = \begin{pmatrix} \vec{a}^{(1)} & \vec{a}^{(2)} & \vec{a}^{(3)} \end{pmatrix}$$

Then we can compute that

$$(Q^T A)_{ij} = \vec{q}^{(i)} \cdot \vec{a}^{(j)}.$$

So

$$\begin{aligned}(Q^T A) &= \begin{pmatrix} (1,0,1)^T \cdot (\frac{1}{\sqrt{2}}, 0, \frac{1}{\sqrt{2}})^T & (2,-1,0)^T \cdot (\frac{1}{\sqrt{2}}, 0, \frac{1}{\sqrt{2}})^T & (1,2,1)^T \cdot (\frac{1}{\sqrt{2}}, 0, \frac{1}{\sqrt{2}})^T \\ (1,0,1)^T \cdot (\frac{1}{\sqrt{3}}, \frac{-1}{\sqrt{3}}, \frac{-1}{\sqrt{3}})^T & (2,-1,0)^T \cdot (\frac{1}{\sqrt{3}}, \frac{-1}{\sqrt{3}}, \frac{-1}{\sqrt{3}})^T & (1,2,1)^T \cdot (\frac{1}{\sqrt{3}}, \frac{-1}{\sqrt{3}}, \frac{-1}{\sqrt{3}})^T \\ (1,0,1)^T \cdot (\frac{1}{\sqrt{6}}, \frac{2}{\sqrt{6}}, \frac{-1}{\sqrt{6}})^T & (2,-1,0)^T \cdot (\frac{1}{\sqrt{6}}, \frac{2}{\sqrt{6}}, \frac{-1}{\sqrt{6}})^T & (1,2,1)^T \cdot (\frac{1}{\sqrt{6}}, \frac{2}{\sqrt{6}}, \frac{-1}{\sqrt{6}})^T \end{pmatrix} \\ &= \begin{pmatrix} \frac{2}{\sqrt{2}} & \frac{2}{\sqrt{2}} & \frac{2}{\sqrt{2}} \\ 0 & \frac{3}{\sqrt{3}} & \frac{-2}{\sqrt{3}} \\ 0 & 0 & \frac{4}{\sqrt{6}} \end{pmatrix}.\end{aligned}$$

Hence we have found an upper triangular matrix $R = Q^T A$. Since Q is orthogonal, we know $Q^{-1} = Q^T$ and we have a factorisation:

$$A = QR.$$

Exercise 8.3. Continue the QR-factorisation process by computing $B = RQ$ and apply the Gram-Schmidt process to the columns of B.

We can even do better than simply computing R afterwards through multiplying $Q^T A$. Let's revisit the update formula for column j:

$$\vec{u}^{(j)} = \vec{a}^{(j)} - \sum_{i=1}^{j-1} (\vec{a}^{(j)} \cdot \vec{q}^{(i)}) \vec{q}^{(i)}$$
$$\vec{q}^{(j)} = \frac{\vec{u}^{(j)}}{\|\vec{u}^{(j)}\|}.$$

Let's define a matrix R by

$$R_{ij} = \begin{cases} \vec{a}^{(j)} \cdot \vec{q}^{(i)} & \text{if} \quad i < j \\ \|\vec{u}^{(j)}\| & \text{if} \quad i = j \\ 0 & \text{if} \quad i > j, \end{cases}$$

then we have

$$\vec{u}^{(j)} = \vec{a}^{(j)} - \sum_{i=1}^{j-1} R_{ij} \vec{q}^{(i)}$$
$$\vec{q}^{(j)} = \frac{\vec{u}^{(j)}}{R_{jj}}.$$

The equations can be rearranged to:

$$\vec{a}^{(j)} = \vec{u}^{(j)} + \sum_{i=1}^{j-1} R_{ij} \vec{q}^{(i)}$$
$$\vec{u}^{(j)} = R_{jj} \vec{q}^{(j)}.$$

Substituting the value of $\vec{u}^{(j)}$ into the first equation gives:

$$\begin{aligned} \vec{a}^{(j)} &= R_{jj} \vec{q}^{(j)} + \sum_{i=1}^{j-1} R_{ij} \vec{q}^{(i)} \\ &= \sum_{i=1}^{j} R_{ij} \vec{q}^{(i)} \\ &= \sum_{i=1}^{n} R_{ij} \vec{q}^{(i)}. \end{aligned}$$

Here, in the final equation, we increased the sum up to n using the fact that $R_{ij} = 0$ for $i > j$. This equation is the same as

$$A = QR.$$

Remark. The Gram-Schmidt algorithm relies on the fact that after each projection there should be something left - i.e. $\vec{u}^{(j)}$ should be non-zero. If $\vec{a}^{(j)}$ is in the span of $\{\vec{q}^{(1)}, \ldots, \vec{q}^{(j-1)}\}$, then the projection leaves $\vec{u}^{(j)} = \vec{0}$. There are a few ways to test this, but the key idea is that if A is non-singular then we will always have $\vec{u}^{(j)} \neq \vec{0}$ – at least in exact-precision calculations…

8.3.2 Applying the QR algorithm by hand

In Exercise 8.3 above, you computed $B = RQ$ from the QR factorisation $A = QR$ found in Example 8.1. That step – factorise, then recombine in the opposite order – is *exactly* one step of the QR algorithm: it is the update rule $A^{(k+1)} = R^{(k)}Q^{(k)}$ from earlier in this section, applied with $A^{(0)}$ equal to the matrix formed from $\vec{a}^{(1)}, \vec{a}^{(2)}, \vec{a}^{(3)}$.

To see the QR algorithm actually converge, it helps to track a smaller matrix over several iterations. Let's apply it to the 2×2 matrix from Example 7.3,

$$A^{(0)} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix},$$

whose eigenvalues we already know, by direct calculation, are 4 and 2.

Step 1: factorise $A^{(0)} = Q^{(0)}R^{(0)}$. Applying the Gram-Schmidt process to the two columns $\vec{a}^{(1)} = (3,1)^T$ and $\vec{a}^{(2)} = (1,3)^T$:

$$\begin{aligned}
\vec{q}^{(1)} &= \frac{\vec{a}^{(1)}}{\|\vec{a}^{(1)}\|} = \frac{1}{\sqrt{10}} \begin{pmatrix} 3 \\ 1 \end{pmatrix}, \\
\vec{u}^{(2)} &= \vec{a}^{(2)} - \left(\vec{a}^{(2)} \cdot \vec{q}^{(1)}\right) \vec{q}^{(1)} = \begin{pmatrix} 1 \\ 3 \end{pmatrix} - \frac{6}{10} \begin{pmatrix} 3 \\ 1 \end{pmatrix} = \begin{pmatrix} -4/5 \\ 12/5 \end{pmatrix}, \\
\vec{q}^{(2)} &= \frac{\vec{u}^{(2)}}{\|\vec{u}^{(2)}\|} = \frac{1}{\sqrt{10}} \begin{pmatrix} -1 \\ 3 \end{pmatrix}.
\end{aligned}$$

So

$$Q^{(0)} = \frac{1}{\sqrt{10}} \begin{pmatrix} 3 & -1 \\ 1 & 3 \end{pmatrix}, \qquad R^{(0)} = \frac{\sqrt{10}}{5} \begin{pmatrix} 5 & 3 \\ 0 & 4 \end{pmatrix}.$$

Step 2: recombine $A^{(1)} = R^{(0)}Q^{(0)}$.

$$\begin{aligned}
A^{(1)} &= \frac{\sqrt{10}}{5} \begin{pmatrix} 5 & 3 \\ 0 & 4 \end{pmatrix} \cdot \frac{1}{\sqrt{10}} \begin{pmatrix} 3 & -1 \\ 1 & 3 \end{pmatrix} = \frac{1}{5} \begin{pmatrix} 5 & 3 \\ 0 & 4 \end{pmatrix} \begin{pmatrix} 3 & -1 \\ 1 & 3 \end{pmatrix} \\
&= \frac{1}{5} \begin{pmatrix} 18 & 4 \\ 4 & 12 \end{pmatrix} = \begin{pmatrix} 3.6 & 0.8 \\ 0.8 & 2.4 \end{pmatrix}.
\end{aligned}$$

Two things are worth checking:

- The trace is preserved: $\text{trace}(A^{(0)}) = 3 + 3 = 6$ and $\text{trace}(A^{(1)}) = 3.6 + 2.4 = 6$. This is a consequence of $A^{(1)}$ being similar to $A^{(0)}$ (Theorem 7.2): similar matrices have the same eigenvalues, and the trace is the sum of the eigenvalues, so it cannot change under a similarity transformation.
- The off-diagonal entries have shrunk, from 1 to 0.8: $A^{(1)}$ is closer to being diagonal than $A^{(0)}$ was.

Repeating the process (we will let Python do the arithmetic below), the off-diagonal entries continue to shrink – to about 0.47 after the second iteration and 0.25 after the third – and $A^{(k)}$ converges towards $\begin{pmatrix} 4 & 0 \\ 0 & 2 \end{pmatrix}$: exactly the eigenvalues we found by hand in Example 7.3.

Exercise 8.4. Perform one more iteration of the QR algorithm by hand: compute the QR factorisation of $A^{(1)}$ found above, and use it to compute $A^{(2)}$. Check that your answer is consistent with the value quoted above for the off-diagonal entries of $A^{(2)}$.

8.3.3 Python QR factorisation using Gram-Schmidt

```python
def gram_schmidt_qr(A):
    """
    Compute the QR factorisation of a square matrix using the classical
    Gram-Schmidt process.

    Parameters
    ----------
    A : numpy.ndarray
        A square 2D NumPy array of shape ``(n, n)`` representing the input
        matrix.

    Returns
    -------
    Q : numpy.ndarray
        orthogonal matrix of shape ``(n, n)`` where the columns form an
        orthonormal basis for the column space of A.
    R : numpy.ndarray
        Upper triangular matrix of shape ``(n, n)``.
    """
    n, m = A.shape
    if n != m:
        raise ValueError(f"the matrix A is not square, {A.shape=}")

    Q = np.empty_like(A)
    R = np.zeros_like(A)

    for j in range(n):
        # Start with the j-th column of A
        u = A[:, j].copy()

```

```
        # Orthogonalize against previous q vectors
        for i in range(j):
            R[i, j] = np.dot(Q[:, i], A[:, j])  # projection coefficient
            u -= R[i, j] * Q[:, i]  # subtract the projection

        # Normalize u to get q_j
        R[j, j] = np.linalg.norm(u)
        Q[:, j] = u / R[j, j]

    return Q, R
```

Let's test it with our example above:

```
A = [  1.0,  2.0,  1.0 ]
    [  0.0, -1.0,  2.0 ]
    [  1.0,  0.0,  1.0 ]
QR factorisation:
Q = [  0.70711,  0.57735,  0.40825 ]
    [  0.00000, -0.57735,  0.81650 ]
    [  0.70711, -0.57735, -0.40825 ]
R = [  1.41421,  1.41421,  1.41421 ]
    [  0.00000,  1.73205, -1.15470 ]
    [  0.00000,  0.00000,  1.63299 ]
Have we computed a factorisation? (A ~= Q @ R) True
Is Q orthogonal? (Q.T @ Q ~= np.eye(3)) True
```

Note that due to rounding errors, A is not exactly equal to QR and Q is not exactly orthogonal.

8.4 Finding eigenvalues and eigenvectors

The algorithm given above says that we use the QR factorisation to iteratively find a sequence of matrices $A^{(k)}$ which *should* converge to an upper-triangular matrix.

```
def gram_schmidt_eigen(A, maxiter=100, verbose=False):
    """
    Compute the eigenvalues and eigenvectors of a square matrix using the QR
    algorithm with classical Gram-Schmidt QR factorisation.

    This function implements the basic QR algorithm:
```

```
    1. Factorise the matrix `A` into `Q` and `R` using Gram-Schmidt QR
       factorisation.
    2. Update the matrix as:

       .. math::
           A_{k+1} = R_k Q_k

    3. Accumulate the orthogonal transformations in `V` to compute the
       eigenvectors.
    4. Iterate until `A` becomes approximately upper triangular or until the
       maximum number of iterations is reached.

    Once the iteration converges, the diagonal of `A` contains the
    eigenvalues, and the columns of `V` contain the corresponding
    eigenvectors.

    Parameters
    ----------
    A : numpy.ndarray
        A square 2D NumPy array of shape ``(n, n)`` representing the input
        matrix.
    maxiter : int, optional
        Maximum number of QR iterations to perform. Default is 100.
    verbose : bool, optional
        If ``True``, prints intermediate matrices (`A`, `Q`, `R`, and `V`) at
        each iteration. Useful for debugging and understanding convergence.
        Default is ``False``.

    Returns
    -------
    eigenvalues : numpy.ndarray
        A 1D NumPy array of length ``n`` containing the eigenvalues of `A`.
        These are the diagonal elements of the final upper triangular matrix.
    V : numpy.ndarray
        A 2D NumPy array of shape ``(n, n)`` whose columns are the normalized
        eigenvectors corresponding to the eigenvalues.
    it : int
        The number of iterations taken by the algorithm.
    """
    # identity matrix to store eigenvectors
    V = np.eye(A.shape[0])
```

```
    if verbose:
        print_array(A)

    it = -1
    for it in range(maxiter):
        if verbose:
            print(f"\n\n{it=}")

        # perform factorisation
        Q, R = gram_schmidt_qr(A)
        if verbose:
            print_array(Q)
            print_array(R)

        # update A and V in place
        A = R @ Q
        V = V @ Q

        if verbose:
            print_array(A)
            print_array(V)

        # test for convergence: is A upper triangular up to tolerance 1.0e-8?
        if np.allclose(A, np.triu(A), atol=1.0e-8):
            break

    eigenvalues = np.diag(A)
    return eigenvalues, V, it
```

We test this out in code first for the same 2×2 matrix from Example 7.3 that we iterated by hand above:

```
A = np.array([[3.0, 1.0], [1.0, 3.0]])
print_array(A)

eigenvalues, eigenvectors, it = gram_schmidt_eigen(A)
print_array(eigenvalues)
print_array(eigenvectors)
print("iterations required:", it)
```

```
A = [  3.0,  1.0 ]
```

```
    [  1.0,  3.0 ]
eigenvalues = [  4.0 ]
              [  2.0 ]
eigenvectors = [  0.7071, -0.7071 ]
               [  0.7071,  0.7071 ]
iterations required: 27
```

These values agree with those from Example 7.3. Note that this code normalises the eigenvectors to have length one, so we have slightly different values for the eigenvectors but still in the same directions.

8.5 Correctness and convergence

Let's see what happens when we try this same approach for bigger symmetric matrices. Here we have a test that samples ten different random matrices and computes the average number of iterations, average run time and maximum error in the eigenvalue equation.

```
# replicable random seed
np.random.seed(42)


def test_accuracy_of_eigensolve(A, eigenvalues, eigenvectors):
    """
    test accuracy of solution of eigenvalue problem
    """
    residuals = []
    for i in range(len(eigenvalues)):
        residual = np.linalg.norm(
            A @ eigenvectors[:, i] - eigenvalues[i] * eigenvectors[:, i]
        )
        residuals.append(residual)
    return max(residuals)


# replicable seed
repeats = 10

# restart the seed
np.random.seed(42)


```

```
def random_symmetric_matrix(n):
    # generate a random matrix
    S = special_ortho_group.rvs(n)
    D = np.diag(np.random.randint(1, 10, (n,)) / 2)
    A = S.T @ D @ S
    return A


print("size  time (s)       iter  error")

n_values = [2**j for j in range(1, 8)]
it_values = []
runtime_values = []

for n in n_values:
    runtime = 0.0
    total_its = 0
    max_error = 0.0

    for _ in range(repeats):
        # generate new random matrix
        A = random_symmetric_matrix(n)

        # do and time the solve, ensure converged
        maxiter = 1000_000
        start = time.perf_counter()
        eigenvalues, eigenvectors, it = gram_schmidt_eigen(A, maxiter=maxiter)
        end = time.perf_counter()

        if it == maxiter - 1:
            # skip
            continue

        # store the data
        runtime += end - start
        total_its += it
        max_error = max(
            max_error,
            test_accuracy_of_eigensolve(A, eigenvalues, eigenvectors),
        )

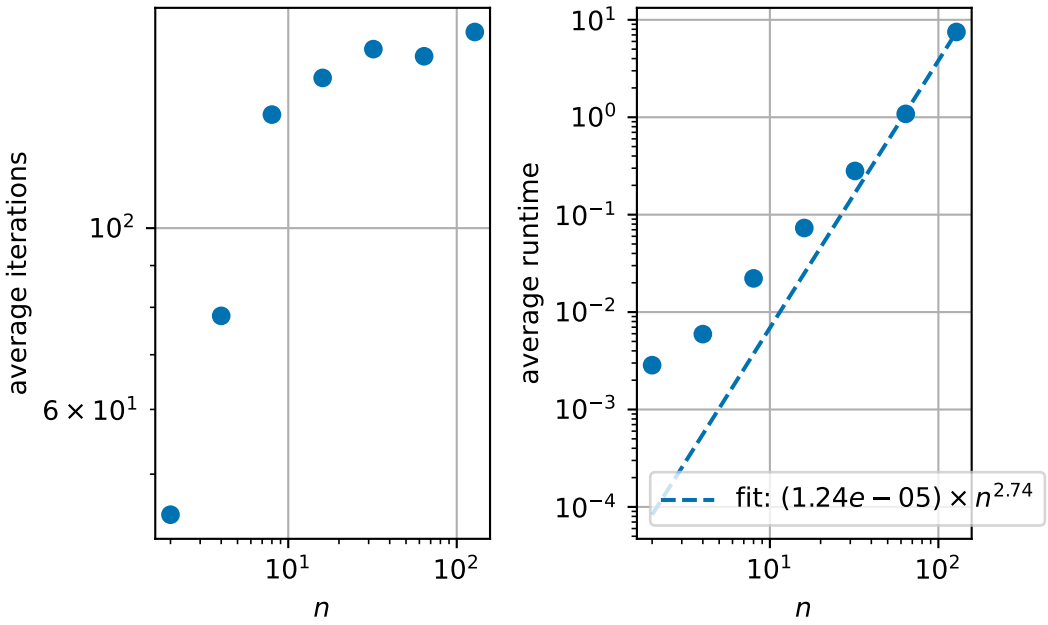
    runtime /= repeats
```

```
    average_it = total_its / repeats

    it_values.append(average_it)
    runtime_values.append(runtime)

    print(f"{n:4d}   {runtime:.6e}  {average_it:5.1f}  {max_error:.6e}")
```

```
size  time (s)       iter  error
   2   2.851427e-03   44.6  9.319111e-09
   4   5.940252e-03   78.1  9.895223e-09
   8   2.217575e-02  137.7  1.212740e-08
  16   7.297618e-02  152.7  1.346641e-08
  32   2.810952e-01  165.6  1.246268e-08
  64   1.082694e+00  162.3  1.646202e-08
 128   7.503152e+00  173.8  1.478504e-08
```



We see that the method converges reliably and to a good level of accuracy, even as the matrix size n grows – a very different picture to the characteristic-equation approach, which becomes infeasible once n is more than a handful.

8.6 Summary

- The "grand strategy" for finding eigenvalues of large matrices is to build a sequence of matrices similar to A (Theorem 7.2), aiming for a triangular (or diagonal) matrix whose eigenvalues can simply be read off the diagonal (Example 7.6).
- The QR algorithm builds this sequence by repeatedly factorising $A^{(k)} = Q^{(k)}R^{(k)}$ into an orthogonal matrix $Q^{(k)}$ and an upper triangular matrix $R^{(k)}$, then recombining them in the opposite order, $A^{(k+1)} = R^{(k)}Q^{(k)}$. Because $Q^{(k)}$ is orthogonal, each $A^{(k+1)}$ is similar to $A^{(k)}$, so every matrix in the sequence has the same eigenvalues as the original A.
- The Gram-Schmidt process gives us a concrete, implementable way to compute a QR factorisation: it turns the columns of A into an orthonormal set of vectors Q, tracking the projection coefficients used along the way in an upper triangular matrix R.
- Our Python implementation confirms, empirically, that the QR algorithm is both correct (it reproduces the eigenvalues and eigenvectors found by hand) and feasible: unlike solving the characteristic equation directly, we can apply our method to large matrices.